Bölüm 3: Bağlı Listeler

Veri Yapıları

Düğüm ve işaretçi | Tek yönlü liste | Ekleme · silme · gezinme | Ters çevirme

Çift yönlü liste | Dairesel liste | Bekçi düğümler

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

GİRİŞ

Öğrenme Çıktıları

Düğüm yapısı

Düğüm, bağlantı ve başvuru kavramlarını açıklamak.

Temel işlemler

Ekleme, silme, arama ve gezinme işlemlerini uygulamak.

Liste türleri

Tek yönlü, çift yönlü ve dairesel listeleri karşılaştırmak.

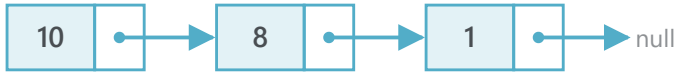
Maliyet analizi

Bağlı liste işlemlerinin zaman ve alan maliyetlerini değerlendirmek.

Bağlı Liste Türleri

Tek yönlü (singly)

Her düğüm yalnızca sonrakini gösterir; son düğüm `null`.



Çift yönlü (doubly)

Her düğüm hem öncekini hem sonrakini gösterir; iki yönde gezilir.



Dairesel (circular)

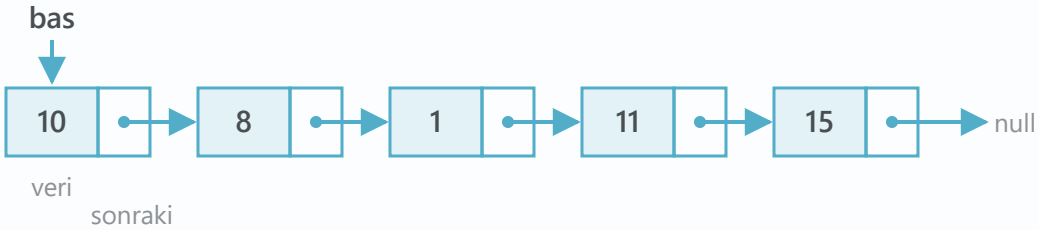
Son düğüm `null` yerine ilk düğümü gösterir; liste bir halka olur.



Üçünde de öğeler, dizideki gibi yan yana hücrelerde değil, birbirini gösteren **düğümlerde** (node) saklanır.

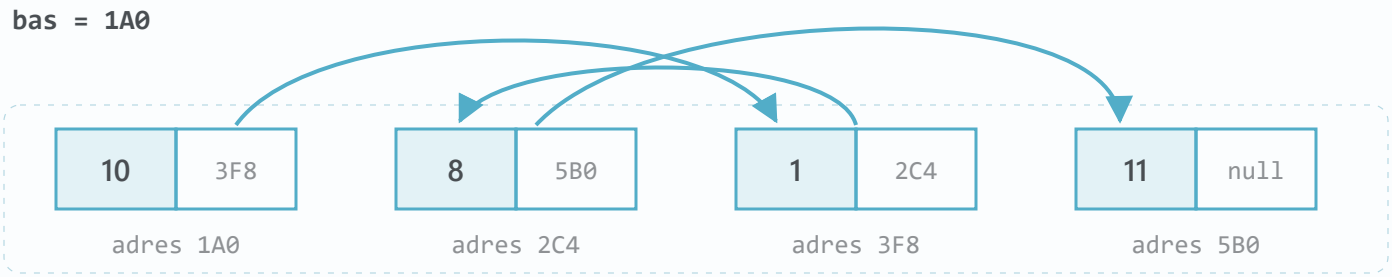
Tek Yönlü Bağlı Liste (Singly Linked List)

- Öğeler birbirine **tek yönlü** bağlantılarla bağlıdır: her öğe kendinden sonrakine bir referans içerir.
- Liste, ilk öğeyi gösteren **baş** (head, kodda `bas`) adında özel bir referansa sahiptir.
- Her düğüm iki parçadan oluşur:
  - **Veri**: düğümün taşıdığı değer (ilkel tip ya da nesne).
  - **İşaretçi (pointer)**: sonraki düğümün referansı; son düğümde `null`.



Düğümler Bellekte Nasıl Durur?

- Öğeler bellekte **ardışık konumlarda tutulmaz**; her düğüm `new` ile ayrı ayrı oluşturulur.
- Yine de **doğrusal** bir veri yapısıdır: her düğümün (sonuncusu hariç) tek bir ardılı vardır.
- Sırayı bellekteki konum değil, **sonraki** alanındaki referanslar belirler.



bellek (adres sırasıyla)

Liste sırası 10 → 1 → 8 → 11 olsa da düğümler bellekte başka sırada; oklar referansları izler.

TEMELLER

Düğüm (Node) Sınıfı

int veri

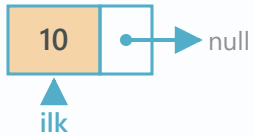
```
// Düğüm sınıfı
class Dugum {
    int veri;
    Dugum sonraki;
    Dugum(int veri) {
        this.veri = veri;
        this.sonraki = null;
    }
}
```

Jenerik (generic) sürüm

```
// Düğüm sınıfı
class Dugum<T> {
    T veri;
    Dugum<T> sonraki;
    Dugum(T veri) {
        this.veri = veri;
        this.sonraki = null;
    }
}
```

- Sınıf kendi türünden bir alan (`Dugum sonraki`) içerir: **öz-referanslı** (self-referential) yapı.
- Jenerik sürüm her tür veriyi saklar: `Dugum<String> d = new Dugum<>("Ali");`
- Yeni düğümün `sonraki` alanı başlangıçta `null` 'dır; listeye bağlanınca değişir.

Tek Yönlü Bağlı Liste Oluşturma



`new Dugum(10)` : veri = 10, sonraki = null olan bir düğüm oluşur; `ilk` onu gösterir.

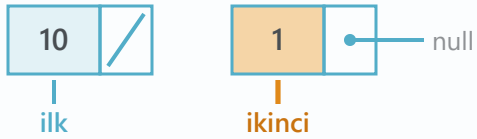
Adım 1 / 7

```
▶ Dugum ilk = new Dugum(10);
2 Dugum ikinci = new Dugum(1);
3 Dugum ucuncu = new Dugum(8);
4 Dugum dorduncu = new Dugum(11);
5 ilk.sonraki = ikinci;
6 ikinci.sonraki = ucuncu;
7 ucuncu.sonraki = dorduncu;
```

DEĞİŞKENLER

`ilk = düğüm(10)`

Tek Yönlü Bağlı Liste Oluşturma



İkinci düğüm (1) oluşur. Düğümler henüz birbirine bağlı değil.

Adım 2 / 7

```
1 Dugum ilk = new Dugum(10);
▶ Dugum ikinci = new Dugum(1);
3 Dugum ucuncu = new Dugum(8);
4 Dugum dorduncu = new Dugum(11);
5 ilk.sonraki = ikinci;
6 ikinci.sonraki = ucuncu;
7 ucuncu.sonraki = dorduncu;
```

DEĞİŞKENLER

`ilk = düğüm(10)`

`ikinci = düğüm(1)`

Tek Yönlü Bağlı Liste Oluşturma



Üçüncü düğüm (8) oluşur. Her düğüm bellekte ayrı bir yerde durabilir.

Adım 3 / 7

```
1 Dugum ilk = new Dugum(10);
2 Dugum ikinci = new Dugum(1);
▶ Dugum ucuncu = new Dugum(8);
4 Dugum dorduncu = new Dugum(11);
5 ilk.sonraki = ikinci;
6 ikinci.sonraki = ucuncu;
7 ucuncu.sonraki = dorduncu;
```

DEĞİŞKENLER

- `ilk = düğüm(10)`
- `ikinci = düğüm(1)`
- `ucuncu = düğüm(8)`

Tek Yönlü Bağlı Liste Oluşturma



Dördüncü düğüm (11) oluşur. Dört bağımsız düğüm var; hepsinin `sonraki` alanı null (/).

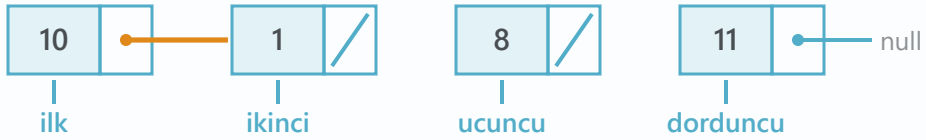
Adım 4 / 7

```
1 Dugum ilk = new Dugum(10);
2 Dugum ikinci = new Dugum(1);
3 Dugum ucuncu = new Dugum(8);
▶ Dugum dorduncu = new Dugum(11);
5 ilk.sonraki = ikinci;
6 ikinci.sonraki = ucuncu;
7 ucuncu.sonraki = dorduncu;
```

DEĞİŞKENLER

- `ilk = düğüm(10)`
- `ikinci = düğüm(1)`
- `ucuncu = düğüm(8)`
- `dorduncu = düğüm(11)`

Tek Yönlü Bağlı Liste Oluşturma



`ilk.sonraki = ikinci` : 10'un sonraki alanı artık 1'i gösterir (turuncu bağ).

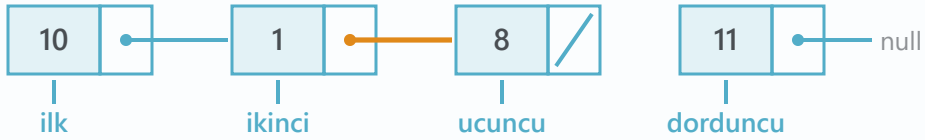
Adım 5 / 7

```
1 Dugum ilk = new Dugum(10);
2 Dugum ikinci = new Dugum(1);
3 Dugum ucuncu = new Dugum(8);
4 Dugum dorduncu = new Dugum(11);
▶ ilk.sonraki = ikinci;
6 ikinci.sonraki = ucuncu;
7 ucuncu.sonraki = dorduncu;
```

DEĞİŞKENLER

- `ilk` = **düğüm(10)**
- `ikinci` = **düğüm(1)**
- `ucuncu` = **düğüm(8)**
- `dorduncu` = **düğüm(11)**

Tek Yönlü Bağlı Liste Oluşturma



`ikinci.sonraki = ucuncu` : 1 → 8 bağlantısı kurulur.

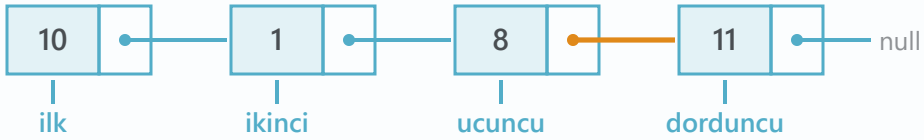
Adım 6 / 7

```
1 Dugum ilk = new Dugum(10);
2 Dugum ikinci = new Dugum(1);
3 Dugum ucuncu = new Dugum(8);
4 Dugum dorduncu = new Dugum(11);
5 ilk.sonraki = ikinci;
▶ ikinci.sonraki = ucuncu;
7 ucuncu.sonraki = dorduncu;
```

DEĞİŞKENLER

- `ilk` = **düğüm(10)**
- `ikinci` = **düğüm(1)**
- `ucuncu` = **düğüm(8)**
- `dorduncu` = **düğüm(11)**

Tek Yönlü Bağlı Liste Oluşturma



`ucuncu.sonraki = dorduncu` : liste hazır: 10 → 1 → 8 → 11 → null. Listeye erişmek için yalnızca `ilk` yeterli.

Adım 7 / 7

```
1 Dugum ilk = new Dugum(10);
2 Dugum ikinci = new Dugum(1);
3 Dugum ucuncu = new Dugum(8);
4 Dugum dorduncu = new Dugum(11);
5 ilk.sonraki = ikinci;
6 ikinci.sonraki = ucuncu;
▶ ucuncu.sonraki = dorduncu;
```

DEĞİŞKENLER

- `ilk` = **düğüm(10)**
- `ikinci` = **düğüm(1)**
- `ucuncu` = **düğüm(8)**
- `dorduncu` = **düğüm(11)**

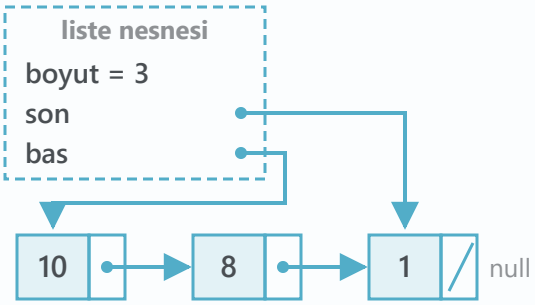
TEMELLER

Tek Yönlü Bağlı Liste İşlemleri

İşlem	Nasıl yapılır?	Karmaşıklık
Başa ekleme	Yeni düğüm eski başı gösterir, baş yeni düğüm olur	O(1)
Sona ekleme	Son düğüme kadar gezinilir (son işaretçisi varsa gezinme yok)	O(n) · O(1)
Baştan silme	Baş bir sonraki düğüme kaydırılır	O(1)
Sondan silme	Sondan bir önceki düğüm bulunmalı	O(n)
Arama · yazdırma · uzunluk	Baştan sona gezinme	O(n)
Ters çevirme · birleştirme	Listeler tek geçişte gezinilir (birleştirmede son işaretçisiyle)	O(n)

n: listedeki düğüm sayısı. i. öğeye erişim de O(n)'dir: dizideki gibi indisle doğrudan atlanamaz.

Tek Yönlü Liste ADT'si ve Java Sınıfı




```java
public class TekYonluBagliListe {
    private Dugum bas = null, son = null;
    private int boyut = 0;    // öğe sayısı
    public int boyut() { return boyut; }
    public boolean bosMu() { return boyut == 0; }
    public Integer ilk() {
        return bosMu() ? null : bas.veri; }
      // basaEkle, sonaEkle, ilkOgeyiSil …
}
```

Goodrich–Tamassia	Bu sunumda	Görev	Maliyet
`size()`, `isEmpty()`	`boyut()`, `bosMu()`	`boyut` alanı her işlemde güncellenir	O(1)
`first()`, `last()`	`ilk()`, `son.veri`	boş listede `null` döner	O(1)
`addFirst`, `addLast`	`basaEkle`, `sonaEkle`	son işaretçisiyle ikisi de sabit süre	O(1)
`removeFirst()`	`ilkOgeyiSil()`	silinen düğümü döndürür	O(1)

TEMELLER

Tek Yönlü Bağlı Liste: Avantajlar ve Dezavantajlar

Avantajları

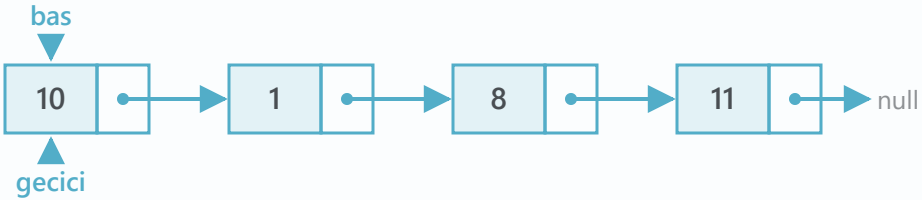
+ Kolay uygulanır.
+ Başa ekleme / baştan silme O(1); araya ekleme kaydırma gerektirmez.
+ Dinamik boyutludur: önceden kapasite ayırmaz, öğe sayısı serbestçe değişir.

Dezavantajları

– Arama yavaştır: O(n).
– Rastgele erişim yoktur; i. öğeye ulaşmak için baştan gezmek gerekir.
– Her düğüm fazladan bir referans taşır (ek bellek).
– Geriye doğru gezilemez.

Araya ekleme/silme O(1) ancak **konum (önceki düğüm) biliniyorsa** geçerlidir; konumu bulmak için gezinme yine O(n) sürer.

Liste Üzerinde Gezinme ve Yazdırma



`gecici = bas` : gezinme baştan başlar. `bas` hiç değişmez; yalnızca `gecici` ilerler.

Adım 1 / 16

```
1 void listeyiYazdir() {
▶       Dugum gecici = bas;
3       System.out.print("Bağlı Liste: ");
4       while (gecici != null) {
5           System.out.print(gecici.veri + " -> ");
6           gecici = gecici.sonraki;
7       }
8       System.out.println("null");
9   }
```

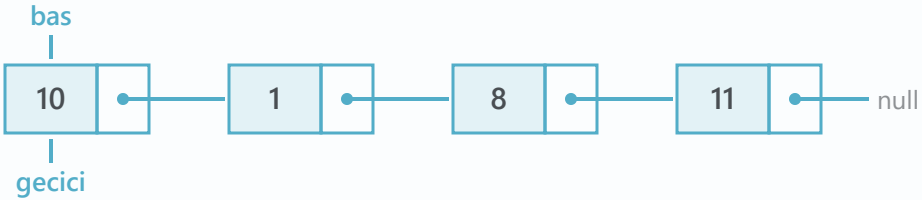
DEĞİŞKENLER

`bas = düğüm(10)`

`gecici = düğüm(10)`

ÇIKTI

Liste Üzerinde Gezinme ve Yazdırma



Başlık yazdırılır.

Adım 2 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
▶     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

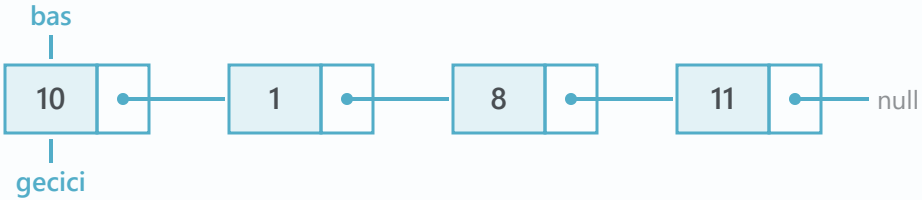
`bas = düğüm(10)`

`gecici = düğüm(10)`

ÇIKTI

```
Bağlı Liste:
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici != null` (düğüm 10) → döngü gövdesine girilir.

Adım 3 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

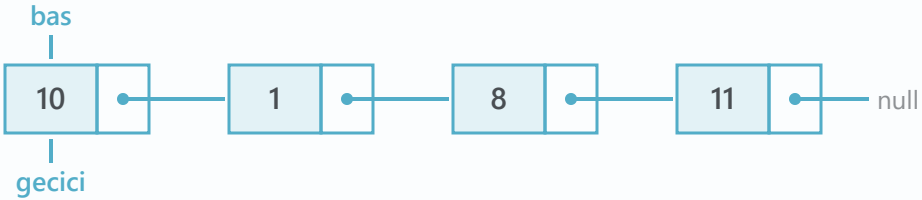
`bas = düğüm(10)`

`gecici = düğüm(10)`

ÇIKTI

```
Bağlı Liste:
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici.veri` = 10 yazdırılır.

Adım 4 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
▶         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

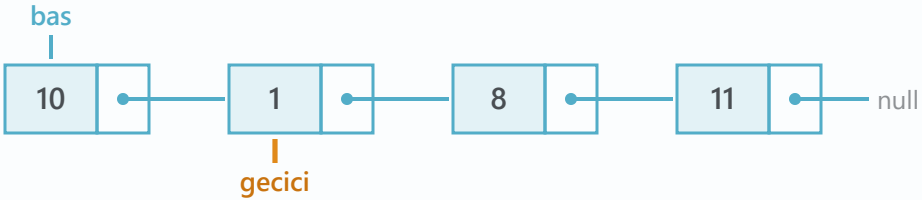
`bas` = **düğüm(10)**

`gecici` = **düğüm(10)**

ÇIKTI

```
Bağlı Liste: 10 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici = gecici.sonraki` → gecici artık düğüm(1).

Adım 5 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
▶         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

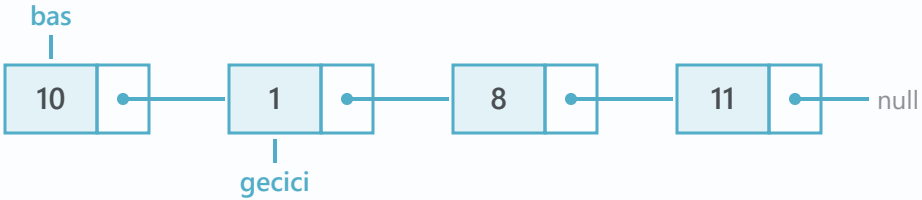
`bas = düğüm(10)`

`gecici = düğüm(1)`

ÇIKTI

```
Bağlı Liste: 10 ->
```

Liste Üzerinde Gezinme ve Yazdırma



gecici = düğüm(1), null değil → devam.

Adım 6 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

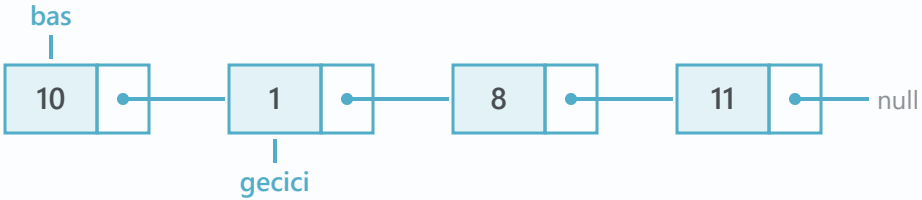
`bas = düğüm(10)`

`gecici = düğüm(1)`

ÇIKTI

```
Bağlı Liste: 10 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici.veri` = 1 yazdırılır.

Adım 7 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
▶         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

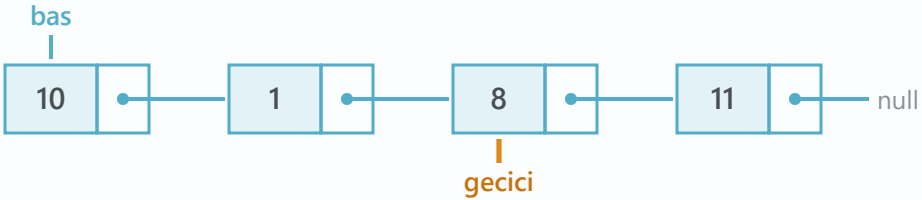
`bas` = **düğüm(10)**

`gecici` = **düğüm(1)**

ÇIKTI

```
Bağlı Liste: 10 -> 1 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici = gecici.sonraki` → gecici artık düğüm(8).

Adım 8 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
▶         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

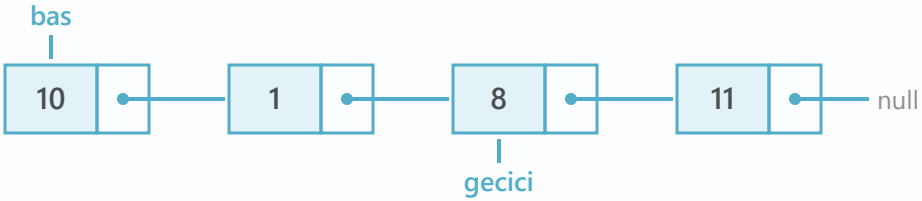
`bas = düğüm(10)`

`gecici = düğüm(8)`

ÇIKTI

```
Bağlı Liste: 10 -> 1 ->
```

Liste Üzerinde Gezinme ve Yazdırma



gecici = düğüm(8), null değil → devam.

Adım 9 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

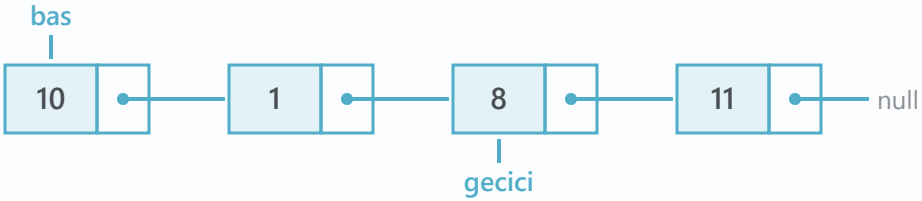
`bas = düğüm(10)`

`gecici = düğüm(8)`

ÇIKTI

```
Bağlı Liste: 10 -> 1 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici.veri` = 8 yazdırılır.

Adım 10 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
▶         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

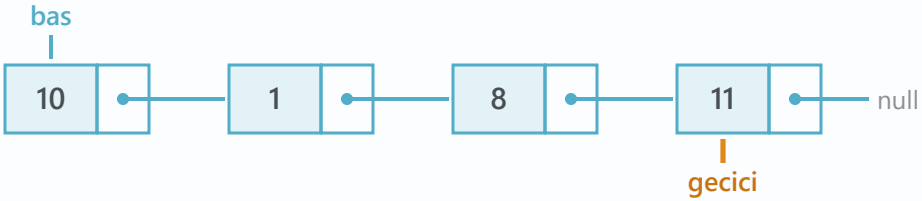
`bas` = **düğüm(10)**

`gecici` = **düğüm(8)**

ÇIKTI

```
Bağlı Liste: 10 -> 1 -> 8 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici = gecici.sonraki` → gecici artık düğüm(11).

Adım 11 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
▶         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

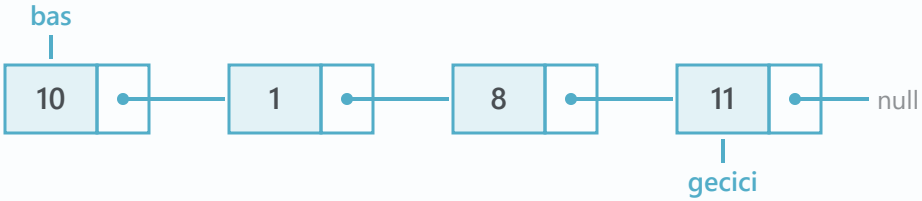
`bas = düğüm(10)`

`gecici = düğüm(11)`

ÇIKTI

```
Bağlı Liste: 10 -> 1 -> 8 ->
```

Liste Üzerinde Gezinme ve Yazdırma



gecici = düğüm(11), null değil → devam.

Adım 12 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

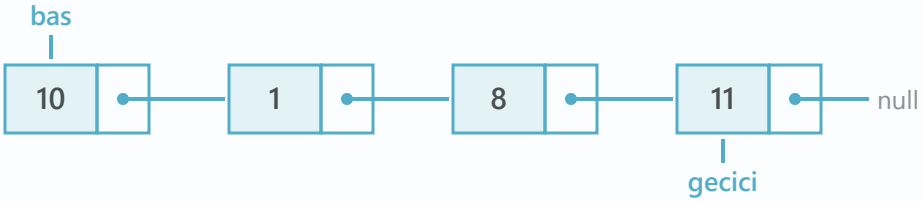
`bas = düğüm(10)`

`gecici = düğüm(11)`

ÇIKTI

```
Bağlı Liste: 10 -> 1 -> 8 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici.veri` = 11 yazdırılır.

Adım 13 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
▶         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

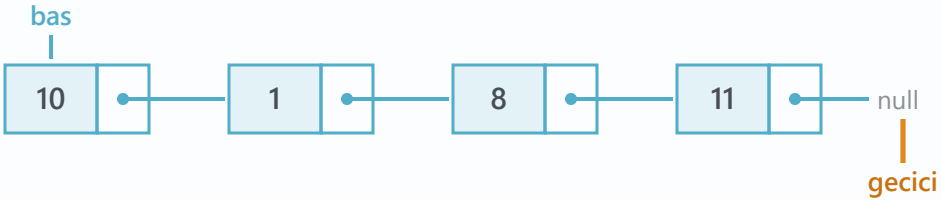
`bas` = **düğüm(10)**

`gecici` = **düğüm(11)**

ÇIKTI

```
Bağlı Liste: 10 -> 1 -> 8 -> 11 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici = gecici.sonraki` → gecici artık **null** (son düğümün sonraki alanı).

Adım 14 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
▶         gecici = gecici.sonraki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

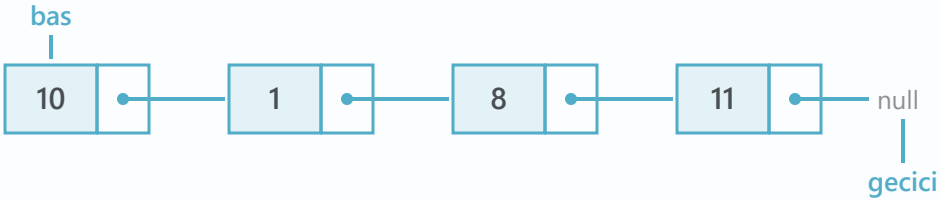
`bas = düğüm(10)`

`gecici = null`

ÇIKTI

```
Bağlı Liste: 10 -> 1 -> 8 -> 11 ->
```

Liste Üzerinde Gezinme ve Yazdırma



`gecici == null` → koşul yanlış, döngü biter.

Adım 15 / 16

```
1  void listeyiYazdir() {
2      Dugum gecici = bas;
3      System.out.print("Bağlı Liste: ");
▶      while (gecici != null) {
5          System.out.print(gecici.veri + " -> ");
6          gecici = gecici.sonraki;
7      }
8      System.out.println("null");
9  }
```

DEĞİŞKENLER

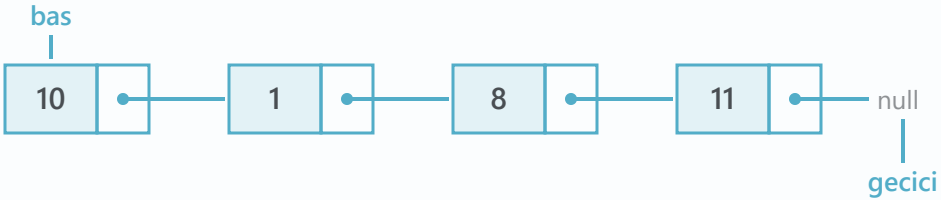
`bas = düğüm(10)`

`gecici = null`

ÇIKTI

```
Bağlı Liste: 10 -> 1 -> 8 -> 11 ->
```

Liste Üzerinde Gezinme ve Yazdırma



Sona `null` yazılır. n düğüm için döngü n kez döner: **O(n)**.

Adım 16 / 16

```
1 void listeyiYazdir() {
2     Dugum gecici = bas;
3     System.out.print("Bağlı Liste: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " -> ");
6         gecici = gecici.sonraki;
7     }
▶     System.out.println("null");
9 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

`gecici = null`

ÇIKTI

```
Bağlı Liste: 10 -> 1 -> 8 -> 11
-> null
```

Aynı Desen: Uzunluk ve Arama

O(n)

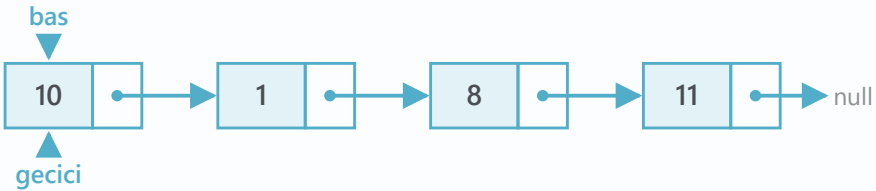
```
int listeUzunlugu() {
    int uzunluk = 0;
    Dugum gecici = bas;
    while (gecici != null) {
        uzunluk++;
        gecici = gecici.sonraki;
    }
    return uzunluk;
}
```

O(n)

```
int ogeyiAra(int aranan) {
    Dugum gecici = bas;
    int konum = 0;
    while (gecici != null) {
        if (gecici.veri == aranan)
            return konum;
        konum++;
        gecici = gecici.sonraki;
    }
    return -1;   // bulunamadı
}
```

Gezinme deseni: `gecici = bas` ile başla, `gecici != null` olduğu sürece işini yap ve `gecici = gecici.sonraki` ile ilerle. Uzunluk sık gerekiyorsa listede bir `boyut` alanı tutulup her ekleme/silmede güncellenerek O(1)'e indirilir.

Kod İzleme: `ogeyiAra(8)`



`ogeyiAra(8)` : `gecici = bas` ile baştan başlanır.

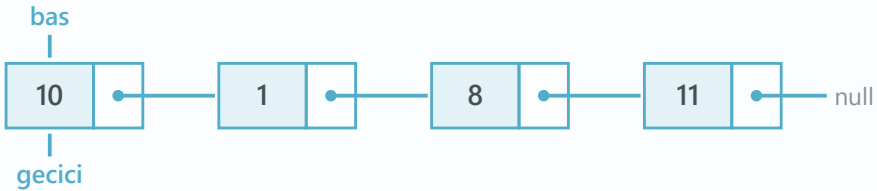
Adım 1 / 13

```
1  int ogeyiAra(int aranan) {
▶      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `aranan` = **8** `gecici` = **düğüm(10)**

Kod İzleme: `ogeyiAra(8)`



`konum = 0` : gecici'nin listedeki sırası (indis) sayılır.

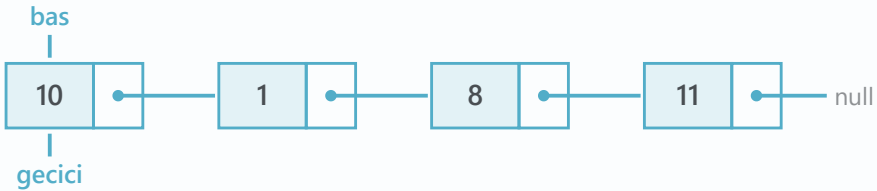
Adım 2 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
▶      int konum = 0;
4      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `aranan = 8` `gecici = düğüm(10)` `konum = 0`

Kod İzleme: `ogeyiAra(8)`



gecici = düğüm(10) null değil → döngüye girilir.

Adım 3 / 13

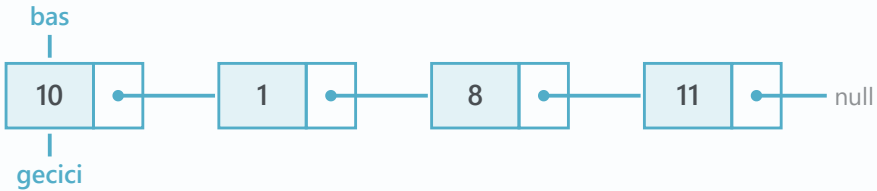
```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
▶      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `aranan` = **8**

`gecici` = **düğüm(10)** `konum` = **0**

Kod İzleme: `ogeyiAra(8)`



10 == 8 mi? **Hayır** → devam.

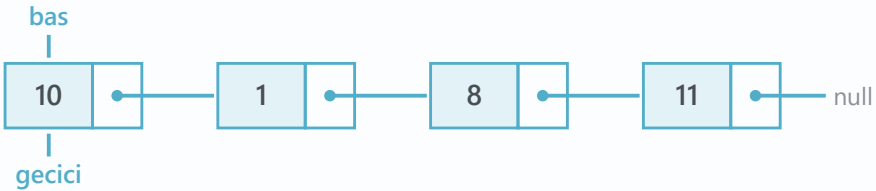
Adım 4 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
▶          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `aranan` = **8** · `gecici` = **düğüm(10)** · `konum` = **0**

Kod İzleme: `ogeyiAra(8)`



`konum++` → 1.

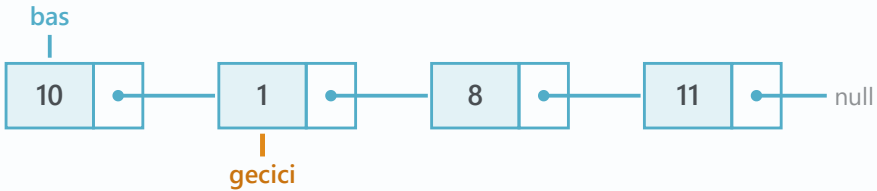
Adım 5 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
▶          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `aranan = 8`
`gecici = düğüm(10)` `konum = 1`

Kod İzleme: `ogeyiAra(8)`



`gecici = gecici.sonraki` → düğüm(1).

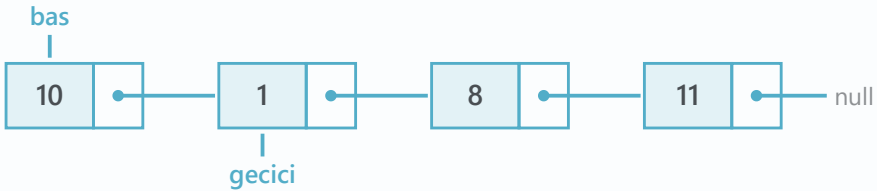
Adım 6 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
7          konum++;
▶          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `aranan` = **8** · `gecici` = **düğüm(1)** · `konum` = **1**

Kod İzleme: `ogeyiAra(8)`



gecici = düğüm(1) null değil → döngüye girilir.

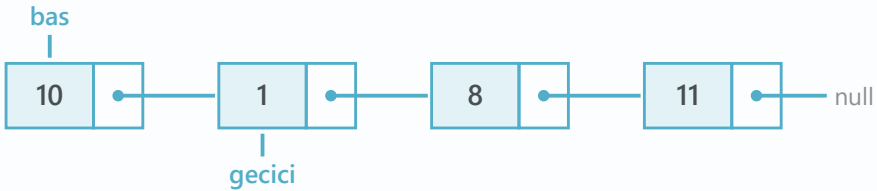
Adım 7 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
▶      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `aranan` = **8**
- `gecici` = **düğüm(1)**
- `konum` = **1**

Kod İzleme: `ogeyiAra(8)`



1 == 8 mi? **Hayır** → devam.

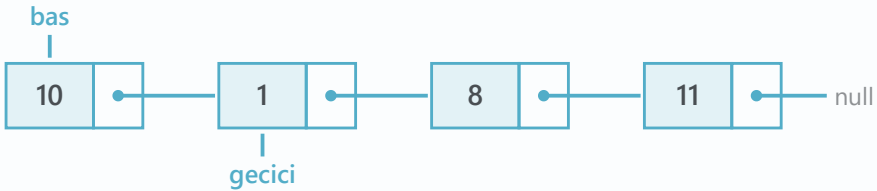
Adım 8 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
▶          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `aranan` = **8** · `gecici` = **düğüm(1)** · `konum` = **1**

Kod İzleme: `ogeyiAra(8)`



`konum++` → 2.

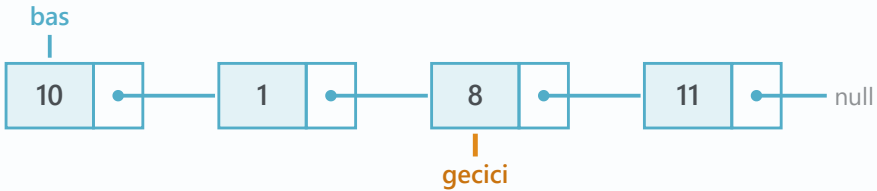
Adım 9 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
▶          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `aranan` = **8** · `gecici` = **düğüm(1)** · `konum` = **2**

Kod İzleme: `ogeyiAra(8)`



`gecici = gecici.sonraki` → düğüm(8).

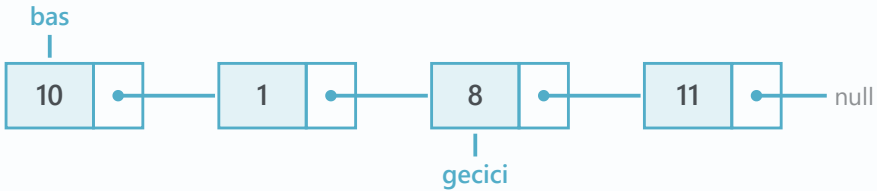
Adım 10 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
7          konum++;
▶          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `aranan` = **8** · `gecici` = **düğüm(8)** · `konum` = **2**

Kod İzleme: `ogeyiAra(8)`



gecici = düğüm(8) null değil → döngüye girilir.

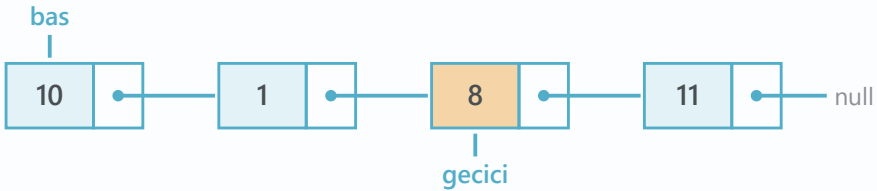
Adım 11 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
▶      while (gecici != null) {
5          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `aranan` = **8** · `gecici` = **düğüm(8)** · `konum` = **2**

Kod İzleme: `ogeyiAra(8)`



8 == 8 → aranan bulundu.

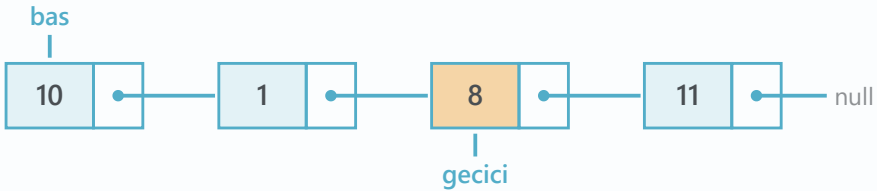
Adım 12 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
▶          if (gecici.veri == aranan)
6              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `aranan` = **8** · `gecici` = **düğüm(8)** · `konum` = **2**

Kod İzleme: `ogeyiAra(8)`



return 2. 3 düğüme bakıldı; en kötü durumda (yoksa) n düğümün hepsine bakılır: **O(n)**, sonuç −1.

Adım 13 / 13

```
1  int ogeyiAra(int aranan) {
2      Dugum gecici = bas;
3      int konum = 0;
4      while (gecici != null) {
5          if (gecici.veri == aranan)
▶              return konum;
7          konum++;
8          gecici = gecici.sonraki;
9      }
10     return -1;   // bulunamadı
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `aranan` = **8**

`gecici` = **düğüm(8)** `konum` = **2**

Başa Düğüm Ekleme — O(1)



`basaEkle(11)` çağrılır. Liste boş: `bas = null`.

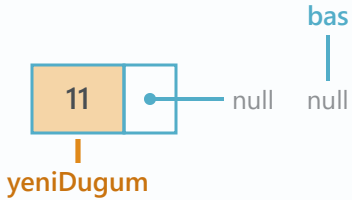
Adım 1 / 12

```
▶ void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     yeniDugum.sonraki = bas;
4     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

`bas = null` `veri = 11`

Başa Düğüm Ekleme — O(1)



Yeni düğüm (11) oluşturulur; sonraki alanı null.

Adım 2 / 12

```
1 void basaEkle(int veri) {
▶       Dugum yeniDugum = new Dugum(veri);
3       yeniDugum.sonraki = bas;
4       bas = yeniDugum;
5 }
```

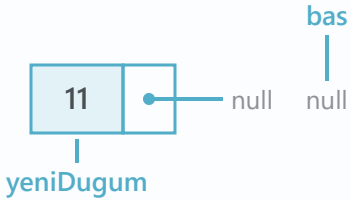
DEĞİŞKENLER

bas = **null**

veri = **11**

yeniDugum = **düğüm(11)**

Başa Düğüm Ekleme — O(1)



`yeniDugum.sonraki = bas` : bas null olduğundan 11'in sonrakisi yine null.

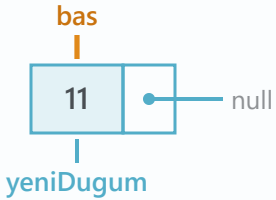
Adım 3 / 12

```
1 void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
▶     yeniDugum.sonraki = bas;
4     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

`bas = null` `veri = 11` `yeniDugum = düğüm(11)`

Başa Düğüm Ekleme — O(1)



`bas = yeniDugum` : baş 11'e taşındı: 11 → null.

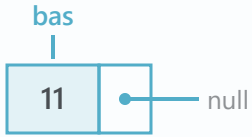
Adım 4 / 12

```
1 void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     yeniDugum.sonraki = bas;
▶     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

`bas = düğüm(11)` `veri = 11` `yeniDugum = düğüm(11)`

Başa Düğüm Ekleme — O(1)



`basaEkle(8)` çağrılır. Liste: 11 → null.

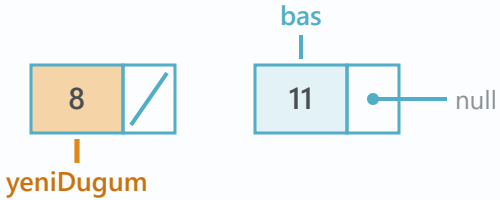
Adım 5 / 12

```
▶ void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     yeniDugum.sonraki = bas;
4     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

`bas = düğüm(11)` `veri = 8`

Başa Düğüm Ekleme — O(1)



Yeni düğüm (8) oluşturulur; sonraki alanı null.

Adım 6 / 12

```
1 void basaEkle(int veri) {
▶       Dugum yeniDugum = new Dugum(veri);
3       yeniDugum.sonraki = bas;
4       bas = yeniDugum;
5 }
```

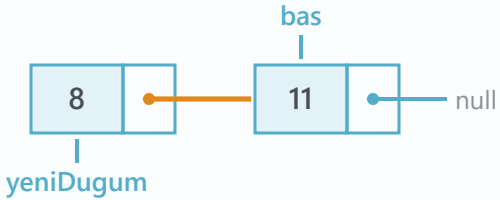
DEĞİŞKENLER

bas = **düğüm(11)**

veri = **8**

yeniDugum = **düğüm(8)**

Başa Düğüm Ekleme — O(1)



`yeniDugum.sonraki = bas` : 8, eski başı (11) gösterir. Önce bas değişseydi 11'e giden yol kaybolurdu.

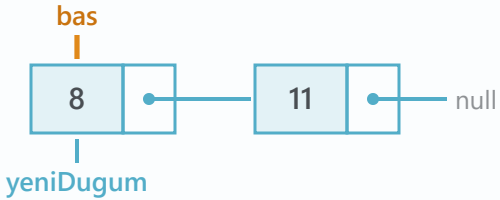
Adım 7 / 12

```
1 void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
▶     yeniDugum.sonraki = bas;
4     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

- bas = **düğüm(11)**
- veri = **8**
- yeniDugum = **düğüm(8)**

Başa Düğüm Ekleme — O(1)



`bas = yeniDugum` : baş 8'e taşındı: 8 → 11 → null.

Adım 8 / 12

```
1 void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     yeniDugum.sonraki = bas;
▶     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

`bas = düğüm(8)` `veri = 8` `yeniDugum = düğüm(8)`

Başa Düğüm Ekleme — O(1)



`basaEkle(1)` çağrılır. Liste: 8 → 11 → null.

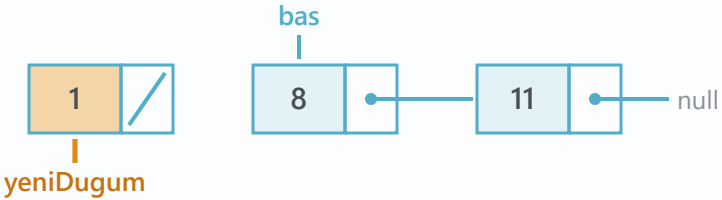
Adım 9 / 12

```
▶ void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     yeniDugum.sonraki = bas;
4     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

`bas = düğüm(8)` `veri = 1`

Başa Düğüm Ekleme — O(1)



Yeni düğüm (1) oluşturulur; sonraki alanı null.

Adım 10 / 12

```
1 void basaEkle(int veri) {
▶       Dugum yeniDugum = new Dugum(veri);
3       yeniDugum.sonraki = bas;
4       bas = yeniDugum;
5 }
```

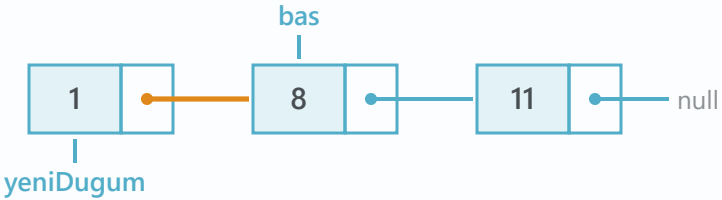
DEĞİŞKENLER

bas = **düğüm(8)**

veri = **1**

yeniDugum = **düğüm(1)**

Başa Düğüm Ekleme — O(1)



`yeniDugum.sonraki = bas` : 1, eski başı (8) gösterir. Önce bas değişseydi 8'e giden yol kaybolurdu.

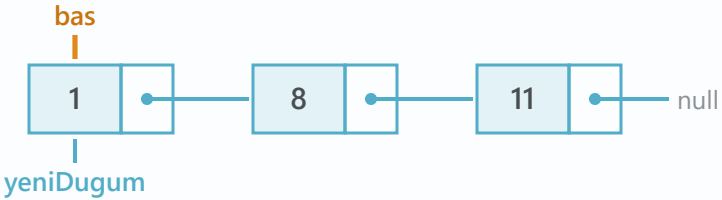
Adım 11 / 12

```
1 void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
      yeniDugum.sonraki = bas;
4     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

- bas = **düğüm(8)**
- veri = **1**
- yeniDugum = **düğüm(1)**

Başa Düğüm Ekleme — O(1)



`bas = yeniDugum` : baş 1'e taşındı: 1 → 8 → 11 → null. Liste ne kadar uzun olursa olsun yalnızca iki atama: **O(1)**.

Adım 12 / 12

```
1 void basaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     yeniDugum.sonraki = bas;
▶     bas = yeniDugum;
5 }
```

DEĞİŞKENLER

`bas = düğüm(1)` `veri = 1` `yeniDugum = düğüm(1)`

Sona Düğüm Ekleme — O(n)



`sonaEkle(10)` : yeni düğüm oluşturulur.

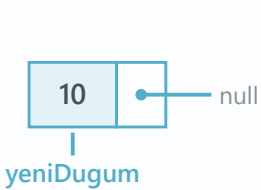
Adım 1 / 16

```
1  void sonaEkle(int veri) {
▶       Dugum yeniDugum = new Dugum(veri);
3       if (bas == null) {
4           bas = yeniDugum;
5           return;
6       }
7       Dugum gecici = bas;
8       while (gecici.sonraki != null)
9           gecici = gecici.sonraki;
10      gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas = null` `veri = 10` `yeniDugum = düğüm(10)`

Sona Düğüm Ekleme — O(n)





`bas == null` → liste boş.

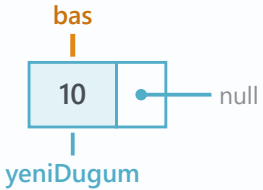
Adım 2 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
▶      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas = null` `veri = 10` `yeniDugum = düğüm(10)`

Sona Düğüm Ekleme — O(n)



`bas = yeniDugum` : tek düğümlü liste.

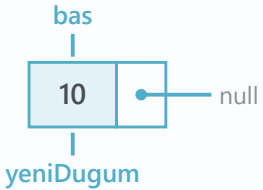
Adım 3 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
▶          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `veri = 10` `yeniDugum = düğüm(10)`

Sona Düğüm Ekleme — O(n)



`return` : gezinmeye gerek yok.

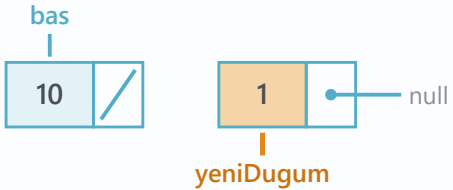
Adım 4 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
▶          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `veri` = **10** `yeniDugum` = **düğüm(10)**

Sona Düğüm Ekleme — O(n)



`sonaEkle(1)` : yeni düğüm oluşturulur.

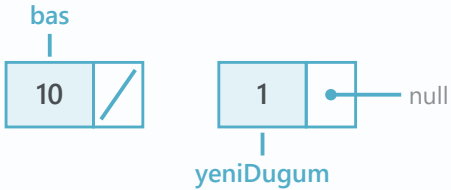
Adım 5 / 16

```
1  void sonaEkle(int veri) {
▶      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `veri = 1` `yeniDugum = düğüm(1)`

Sona Düğüm Ekleme — O(n)



Liste boş değil → sona kadar gezinilecek.

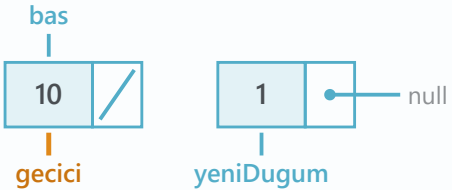
Adım 6 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
▶      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `veri` = **1** `yeniDugum` = **düğüm(1)**

Sona Düğüm Ekleme — O(n)



`gecici = bas` : son düğümü bulmak için baştan başlanır.

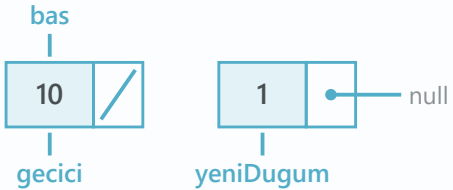
Adım 7 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
▶      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `veri = 1`
- `yeniDugum = düğüm(1)`
- `gecici = düğüm(10)`

Sona Düğüm Ekleme — O(n)



düğüm(10).sonraki == null → gecici son düğümde, döngü biter.

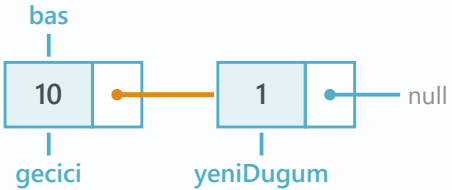
Adım 8 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
▶      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `veri = 1`
- `yeniDugum = düğüm(1)`
- `gecici = düğüm(10)`

Sona Düğüm Ekleme — O(n)



`gecici.sonraki = yeniDugum` : 10 → 1 → null.

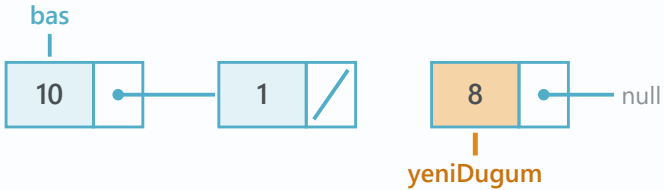
Adım 9 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
▶      gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `veri = 1`
- `yeniDugum = düğüm(1)`
- `gecici = düğüm(10)`

Sona Düğüm Ekleme — O(n)



`sonaEkle(8)` : yeni düğüm oluşturulur.

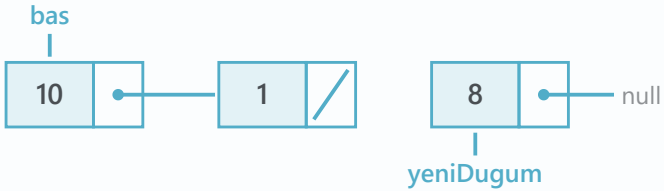
Adım 10 / 16

```
1  void sonaEkle(int veri) {
▶       Dugum yeniDugum = new Dugum(veri);
3       if (bas == null) {
4           bas = yeniDugum;
5           return;
6       }
7       Dugum gecici = bas;
8       while (gecici.sonraki != null)
9           gecici = gecici.sonraki;
10      gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `veri = 8` `yeniDugum = düğüm(8)`

Sona Düğüm Ekleme — O(n)



Liste boş değil → sona kadar gezinilecek.

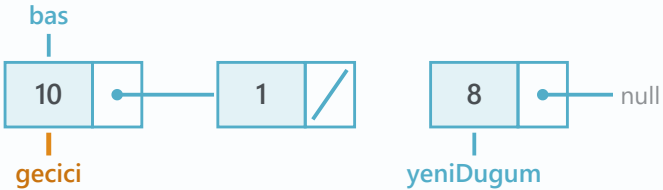
Adım 11 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
▶      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `veri` = **8** `yeniDugum` = **düğüm(8)**

Sona Düğüm Ekleme — O(n)



`gecici = bas` : son düğümü bulmak için baştan başlanır.

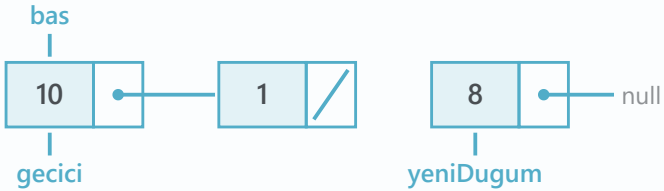
Adım 12 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
▶      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `veri = 8`
- `yeniDugum = düğüm(8)`
- `gecici = düğüm(10)`

Sona Düğüm Ekleme — O(n)



düğüm(10).sonraki null değil → ilerle.

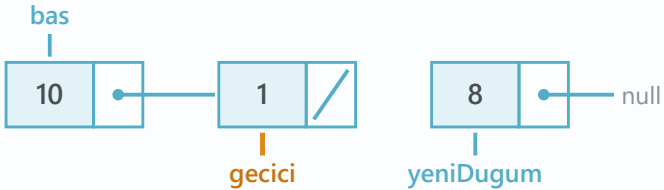
Adım 13 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
▶      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `veri = 8`
- `yeniDugum = düğüm(8)`
- `gecici = düğüm(10)`

Sona Düğüm Ekleme — O(n)



`gecici = gecici.sonraki` → düğüm(1).

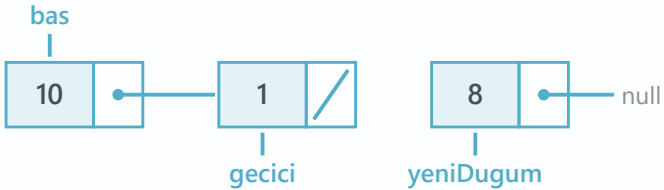
Adım 14 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
▶          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `veri` = **8**
- `yeniDugum` = **düğüm(8)**
- `gecici` = **düğüm(1)**

Sona Düğüm Ekleme — O(n)



düğüm(1).sonraki == null → gecici son düğümde, döngü biter.

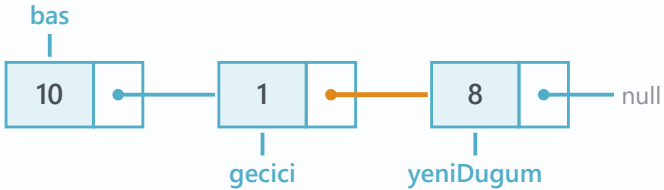
Adım 15 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
▶      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `veri = 8`
- `yeniDugum = düğüm(8)`
- `gecici = düğüm(1)`

Sona Düğüm Ekleme — O(n)



`gecici.sonraki = yeniDugum` : 10 → 1 → 8 → null. Her ekleme sona kadar gezinir: **O(n)**.

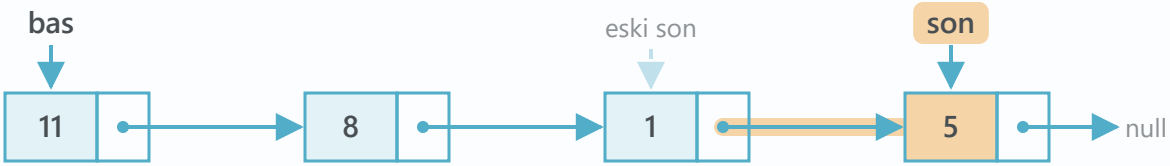
Adım 16 / 16

```
1  void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki != null)
9          gecici = gecici.sonraki;
▶      gecici.sonraki = yeniDugum;
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `veri` = **8**
- `yeniDugum` = **düğüm(8)**
- `gecici` = **düğüm(1)**

Son (Tail) İşaretçisi ile Sona Ekleme — O(1)




```
class TekYonluBagliListe {
    Dugum bas, son;   // baş ve son
    void sonaEkle(int veri) {
        Dugum yeniDugum = new Dugum(veri);
        if (bas == null) {
            bas = son = yeniDugum;
        } else {
            son.sonraki = yeniDugum;
            son = yeniDugum;
        }
    }
}
```

Gezinme yok: O(1)

Son düğüm doğrudan bilinir; iki atama yeter.

Bedeli

Her işlem `son`'u güncel tutmalı (boş listeye ekleme, son düğümü silme).

Sondan silme hâlâ O(n)

Tek yönlü listede `son`'dan geriye gidilemez.

Kod İzleme: `son` ile `sonaEkle`



`sonaEkle(10)` : yeni düğüm oluşturulur.

Adım 1 / 11

```
1 void sonaEkle(int veri) {
▶     Dugum yeniDugum = new Dugum(veri);
3     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas = null`
- `son = null`
- `veri = 10`
- `yeniDugum = düğüm(10)`

Kod İzleme: `son` ile `sonaEkle`



`bas == null` → liste boş.

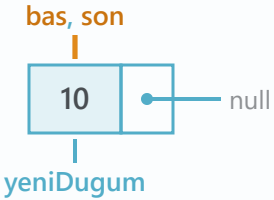
Adım 2 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
▶     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas = null`
- `son = null`
- `veri = 10`
- `yeniDugum = düğüm(10)`

Kod İzleme: `son` ile `sonaEkle`



`bas = son = yeniDugum` : tek düğüm hem baş hem son.

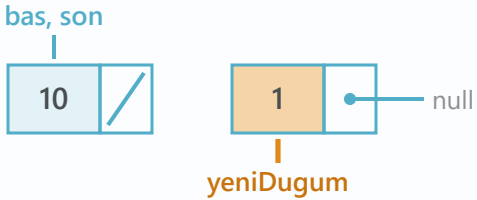
Adım 3 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     if (bas == null) {
▶         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(10)**
- `veri` = **10**
- `yeniDugum` = **düğüm(10)**

Kod İzleme: `son` ile `sonaEkle`



`sonaEkle(1)` : yeni düğüm oluşturulur.

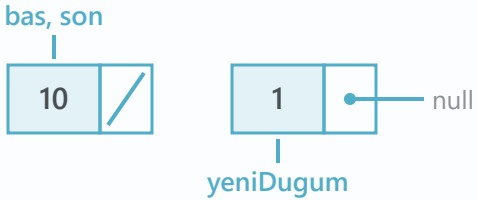
Adım 4 / 11

```
1 void sonaEkle(int veri) {
▶       Dugum yeniDugum = new Dugum(veri);
3       if (bas == null) {
4           bas = son = yeniDugum;
5       } else {
6           son.sonraki = yeniDugum;
7           son = yeniDugum;
8       }
9   }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(10)**
- `veri` = **1**
- `yeniDugum` = **düğüm(1)**

Kod İzleme: `son` ile `sonaEkle`



Liste boş değil; son düğüm zaten biliniyor: düğüm(10).

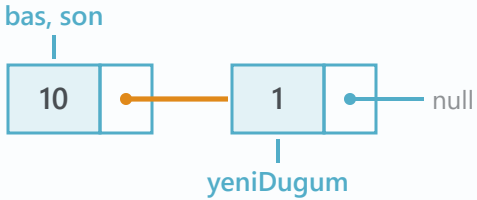
Adım 5 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
▶     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(10)**
- `veri` = **1**
- `yeniDugum` = **düğüm(1)**

Kod İzleme: `son` ile `sonaEkle`



`son.sonraki = yeniDugum` : 10 → 1 bağı kurulur. Gezinme yok.

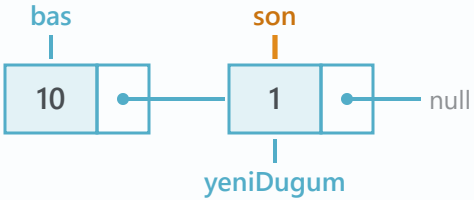
Adım 6 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
▶           son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(10)**
- `veri` = **1**
- `yeniDugum` = **düğüm(1)**

Kod İzleme: `son` ile `sonaEkle`



`son = yeniDugum` : son artık 1.

Adım 7 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
▶         son = yeniDugum;
8     }
9 }
```

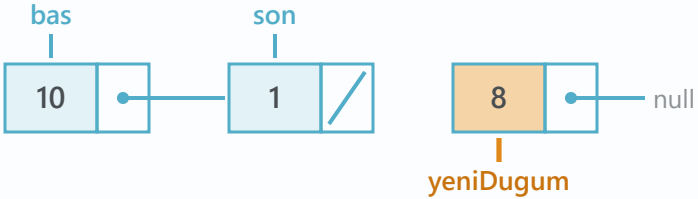
DEĞİŞKENLER

`bas = düğüm(10)`

`son = düğüm(1)` `veri = 1`

`yeniDugum = düğüm(1)`

Kod İzleme: `son` ile `sonaEkle`



`sonaEkle(8)` : yeni düğüm oluşturulur.

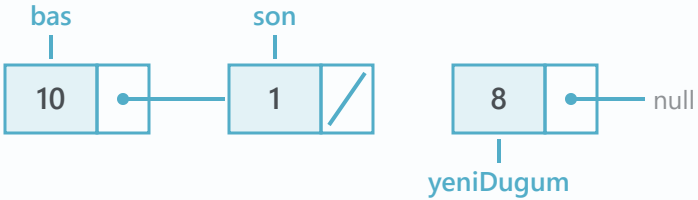
Adım 8 / 11

```
1 void sonaEkle(int veri) {
▶     Dugum yeniDugum = new Dugum(veri);
3     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(1)**
- `veri` = **8**
- `yeniDugum` = **düğüm(8)**

Kod İzleme: `son` ile `sonaEkle`



Liste boş değil; son düğüm zaten biliniyor: düğüm(1).

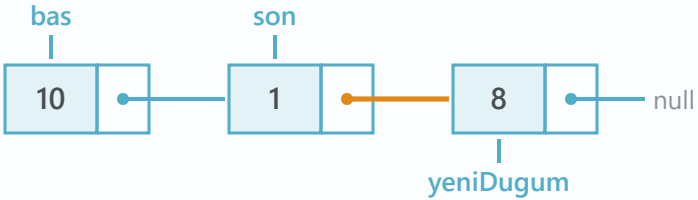
Adım 9 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
▶     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(1)**
- `veri` = **8**
- `yeniDugum` = **düğüm(8)**

Kod İzleme: `son` ile `sonaEkle`



`son.sonraki = yeniDugum` : 1 → 8 bağı kurulur. Gezinme yok.

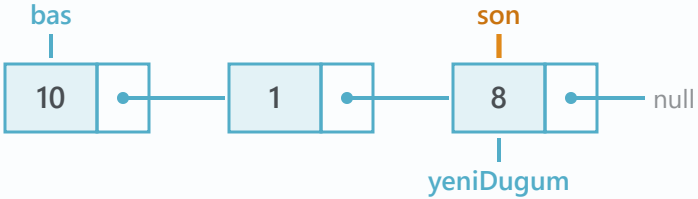
Adım 10 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
▶           son.sonraki = yeniDugum;
7         son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(1)**
- `veri` = **8**
- `yeniDugum` = **düğüm(8)**

Kod İzleme: `son` ile `sonaEkle`



`son = yeniDugum` : son artık 8. Her çağrı iki atama: **O(1)**.

Adım 11 / 11

```
1 void sonaEkle(int veri) {
2     Dugum yeniDugum = new Dugum(veri);
3     if (bas == null) {
4         bas = son = yeniDugum;
5     } else {
6         son.sonraki = yeniDugum;
          son = yeniDugum;
8     }
9 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `son = düğüm(8)`
- `veri = 8`
- `yeniDugum = düğüm(8)`

TEK YÖNLÜ: GEZİNME VE EKLEME

Belirli Bir Konuma Düğüm Ekleme

```
void konumaEkle(int veri, int indis) {   // indis 0'dan başlar
    if (indis == 0) { basaEkle(veri); return; }
    Dugum gecici = bas;
    for (int i = 0; i < indis - 1 && gecici != null; i++)
        gecici = gecici.sonraki;
    if (gecici == null) {          // liste indis'ten kısa
        System.out.println("Belirtilen konum bulunamadı.");
        return;
    }
    Dugum yeniDugum = new Dugum(veri);
    yeniDugum.sonraki = gecici.sonraki;   // (1)
    gecici.sonraki = yeniDugum;           // (2)
}
```

indis 0

Başa ekleme ile aynıdır; ayrıca ele alınır.

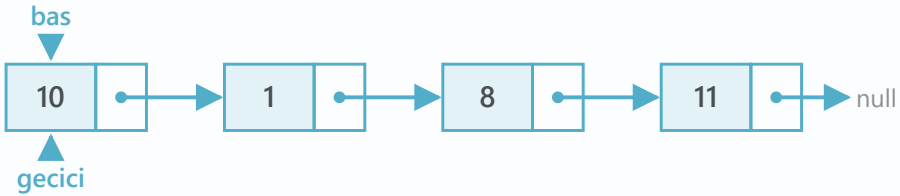
Sıra önemli

Önce (1), sonra (2); tersi listenin kalanını koparır.

Maliyet

Konuma yürümek O(indis), en kötü O(n); bağlama O(1).

Kod İzleme: `konumaEkle(5, 2)`



`konumaEkle(5, 2)` : indis ≠ 0, başa ekleme değil. `gecici = bas` ; indis − 1 = 1 adım ilerlenecek.

Adım 1 / 8

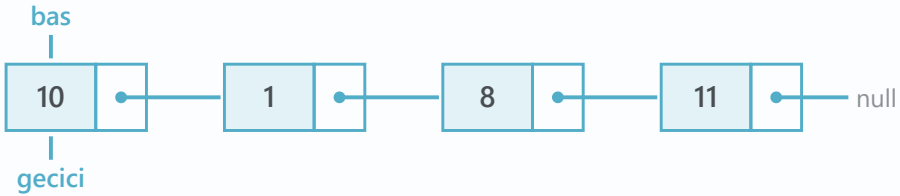
```
▶ Dugum gecici = bas;
2 for (int i = 0; i < indis - 1 && gecici != null; i++)
3     gecici = gecici.sonraki;
4 if (gecici == null) {          // liste indis'ten kısa
5     System.out.println("Belirtilen konum bulunamadı.");
6     return;
7 }
8 Dugum yeniDugum = new Dugum(veri);
9 yeniDugum.sonraki = gecici.sonraki;   // (1)
10 gecici.sonraki = yeniDugum;           // (2)
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `veri` = **5**

`indis` = **2** `gecici` = **düğüm(10)**

Kod İzleme: `konumaEkle(5, 2)`



i = 0: 0 < 1 ve gecici != null → döngü gövdesi.

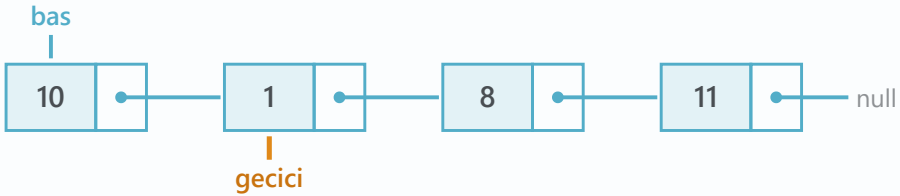
Adım 2 / 8

```
1  Dugum gecici = bas;
▶  for (int i = 0; i < indis - 1 && gecici != null; i++)
3      gecici = gecici.sonraki;
4  if (gecici == null) {          // liste indis'ten kısa
5      System.out.println("Belirtilen konum bulunamadı.");
6      return;
7  }
8  Dugum yeniDugum = new Dugum(veri);
9  yeniDugum.sonraki = gecici.sonraki;   // (1)
10 gecici.sonraki = yeniDugum;           // (2)
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `veri` = **5** · `indis` = **2** · `gecici` = **düğüm(10)** · `i` = **0**

Kod İzleme: `konumaEkle(5, 2)`



`gecici = gecici.sonraki` → düğüm(1).

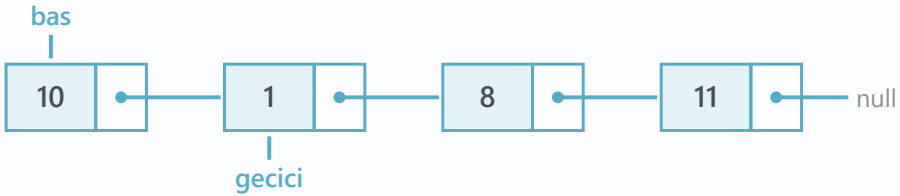
Adım 3 / 8

```
1  Dugum gecici = bas;
2  for (int i = 0; i < indis - 1 && gecici != null; i++)
▶      gecici = gecici.sonraki;
4  if (gecici == null) {          // liste indis'ten kısa
5      System.out.println("Belirtilen konum bulunamadı.");
6      return;
7  }
8  Dugum yeniDugum = new Dugum(veri);
9  yeniDugum.sonraki = gecici.sonraki;   // (1)
10 gecici.sonraki = yeniDugum;           // (2)
```

DEĞİŞKENLER

`bas = düğüm(10)` `veri = 5` `indis = 2` `gecici = düğüm(1)` `i = 0`

Kod İzleme: `konumaEkle(5, 2)`



i = 1: 1 < 1 yanlış → döngü biter. gecici, indis 1'deki düğümde (eklenecek yerin öncesi).

Adım 4 / 8

```
1  Dugum gecici = bas;
▶  for (int i = 0; i < indis - 1 && gecici != null; i++)
3      gecici = gecici.sonraki;
4  if (gecici == null) {          // liste indis'ten kısa
5      System.out.println("Belirtilen konum bulunamadı.");
6      return;
7  }
8  Dugum yeniDugum = new Dugum(veri);
9  yeniDugum.sonraki = gecici.sonraki;   // (1)
10 gecici.sonraki = yeniDugum;           // (2)
```

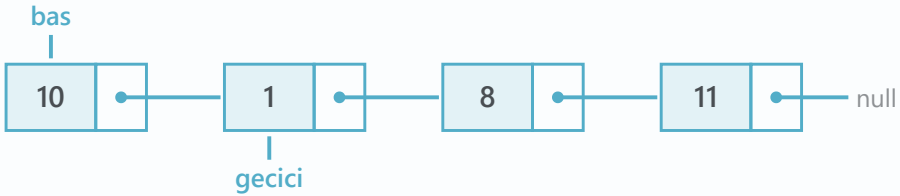
DEĞİŞKENLER

`bas` = **düğüm(10)** `veri` = **5**

`indis` = **2** `gecici` = **düğüm(1)**

`i` = **1**

Kod İzleme: `konumaEkle(5, 2)`



gecici null değil → konum geçerli, hata mesajı yok.

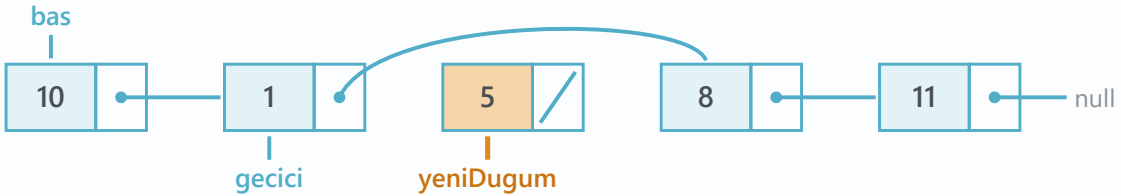
Adım 5 / 8

```
1  Dugum gecici = bas;
2  for (int i = 0; i < indis - 1 && gecici != null; i++)
3      gecici = gecici.sonraki;
▶  if (gecici == null) {          // liste indis'ten kısa
5      System.out.println("Belirtilen konum bulunamadı.");
6      return;
7  }
8  Dugum yeniDugum = new Dugum(veri);
9  yeniDugum.sonraki = gecici.sonraki;   // (1)
10 gecici.sonraki = yeniDugum;           // (2)
```

DEĞİŞKENLER

`bas` = **düğüm(10)** · `veri` = **5** · `indis` = **2** · `gecici` = **düğüm(1)**

Kod İzleme: `konumaEkle(5, 2)`



Yeni düğüm (5) oluşturulur. (Çizimde 8 ve 11 sağa kaydırıldı; bellekte hiçbir öğe kaymaz.) 1 hâlâ 8'i gösteriyor.

Adım 6 / 8

```
1  Dugum gecici = bas;
2  for (int i = 0; i < indis - 1 && gecici != null; i++)
3      gecici = gecici.sonraki;
4  if (gecici == null) {          // liste indis'ten kısa
5      System.out.println("Belirtilen konum bulunamadı.");
6      return;
7  }
▶  Dugum yeniDugum = new Dugum(veri);
9  yeniDugum.sonraki = gecici.sonraki;   // (1)
10 gecici.sonraki = yeniDugum;           // (2)
```

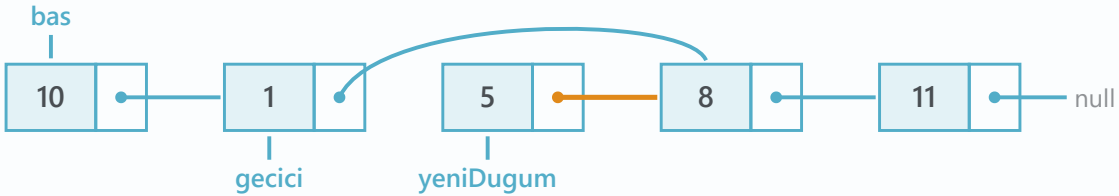
DEĞİŞKENLER

`bas = düğüm(10)` `veri = 5`

`indis = 2` `gecici = düğüm(1)`

`yeniDugum = düğüm(5)`

Kod İzleme: `konumaEkle(5, 2)`



(1) `yeniDugum.sonraki = gecici.sonraki` : 5, 8'i gösterir. Liste henüz değişmedi.

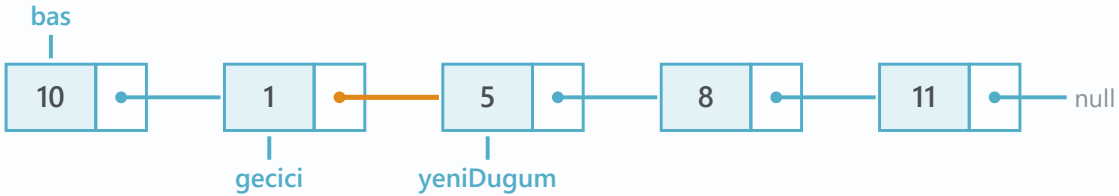
Adım 7 / 8

```
1  Dugum gecici = bas;
2  for (int i = 0; i < indis - 1 && gecici != null; i++)
3      gecici = gecici.sonraki;
4  if (gecici == null) {          // liste indis'ten kısa
5      System.out.println("Belirtilen konum bulunamadı.");
6      return;
7  }
8  Dugum yeniDugum = new Dugum(veri);
▶  yeniDugum.sonraki = gecici.sonraki;   // (1)
10 gecici.sonraki = yeniDugum;           // (2)
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `veri` = **5**
- `indis` = **2**
- `gecici` = **düğüm(1)**
- `yeniDugum` = **düğüm(5)**

Kod İzleme: `konumaEkle(5, 2)`



(2) `gecici.sonraki = yeniDugum` : 10 → 1 → 5 → 8 → 11. Yalnızca **2 bağlantı** değişti; sıra ters olsaydı 8'e yol kaybolurdu.

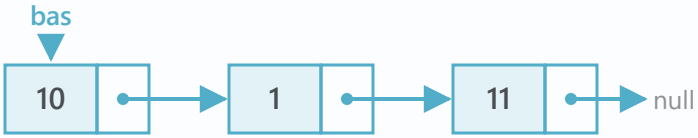
Adım 8 / 8

```
1 Dugum gecici = bas;
2 for (int i = 0; i < indis - 1 && gecici != null; i++)
3     gecici = gecici.sonraki;
4 if (gecici == null) {          // liste indis'ten kısa
5     System.out.println("Belirtilen konum bulunamadı.");
6     return;
7 }
8 Dugum yeniDugum = new Dugum(veri);
9 yeniDugum.sonraki = gecici.sonraki;   // (1)
▶ gecici.sonraki = yeniDugum;           // (2)
```

DEĞİŞKENLER

`bas = düğüm(10)` `veri = 5` `indis = 2` `gecici = düğüm(1)` `yeniDugum = düğüm(5)`

Baştan Düğüm Silme — O(1)



1. çağrı: `bas` null değil (10 → 1 → 11 → null).

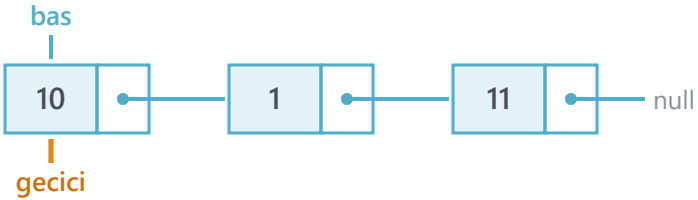
Adım 1 / 16

```
1 Dugum ilkOgeyiSil() {
▶     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

Baştan Düğüm Silme — O(1)



`gecici = bas` : silinecek düğüm (10) kaybolmasın diye saklanır.

Adım 2 / 16

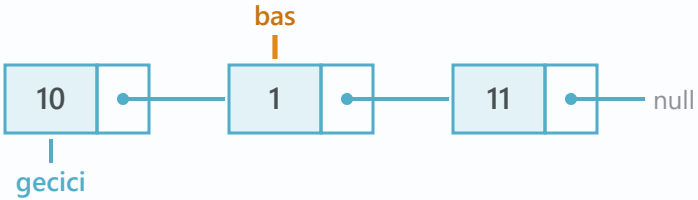
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
▶     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

`gecici = düğüm(10)`

Baştan Düğüm Silme — O(1)



`bas = bas.sonraki` : baş düğüm(1); 10'a artık listeden ulaşılamaz.

Adım 3 / 16

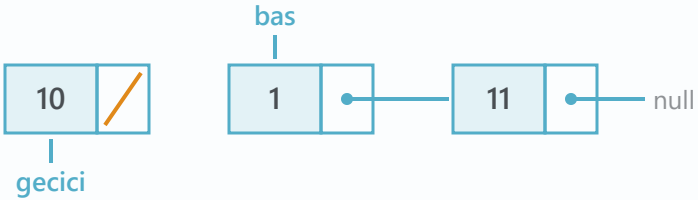
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
▶     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(1)`

`gecici = düğüm(10)`

Baştan Düğüm Silme — O(1)



`gecici.sonraki = null` : 10 listeden tamamen koparılır.

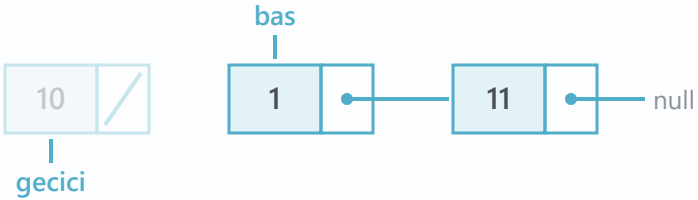
Adım 4 / 16

```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
▶     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

- `bas = düğüm(1)`
- `gecici = düğüm(10)`

Baştan Düğüm Silme — O(1)



`return gecici` : 10 çağırana döner. Kimse tutmazsa çöp toplayıcı (GC) belleği geri alır. **O(1)**.

Adım 5 / 16

```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
▶     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(1)`

`gecici = düğüm(10)`

Baştan Düğüm Silme — O(1)



2. çağrı: `bas` null değil (1 → 11 → null).

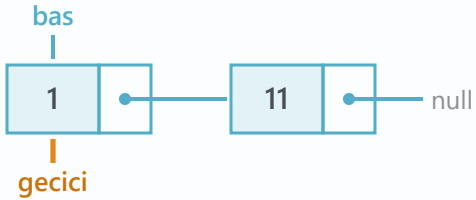
Adım 6 / 16

```
1 Dugum ilkOgeyiSil() {
▶     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(1)`

Baştan Düğüm Silme — O(1)



`gecici = bas` : silinecek düğüm (1) kaybolmasın diye saklanır.

Adım 7 / 16

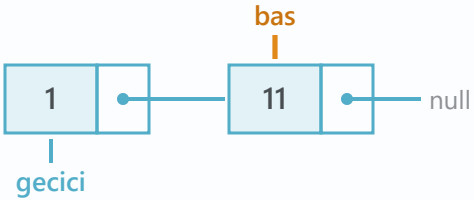
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
▶     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(1)`

`gecici = düğüm(1)`

Baştan Düğüm Silme — O(1)



`bas = bas.sonraki` : baş düğüm(11); 1'e artık listeden ulaşılamaz.

Adım 8 / 16

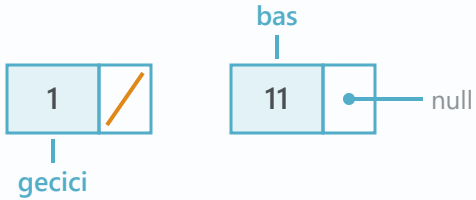
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
▶     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

bas = **düğüm(11)**

gecici = **düğüm(1)**

Baştan Düğüm Silme — O(1)



`gecici.sonraki = null` : 1 listeden tamamen koparılır.

Adım 9 / 16

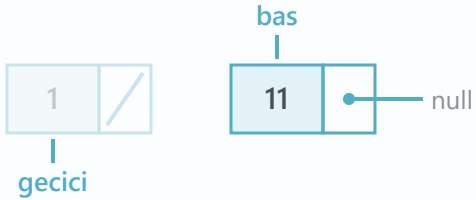
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
▶     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(11)`

`gecici = düğüm(1)`

Baştan Düğüm Silme — O(1)



`return gecici` : 1 çağırana döner. Kimse tutmazsa çöp toplayıcı (GC) belleği geri alır. **O(1)**.

Adım 10 / 16

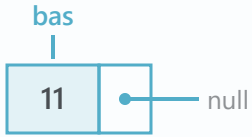
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
▶     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(11)`

`gecici = düğüm(1)`

Baştan Düğüm Silme — O(1)



3. çağrı: `bas` null değil (11 → null).

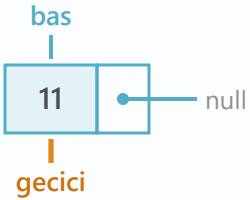
Adım 11 / 16

```
1 Dugum ilkOgeyiSil() {
▶     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(11)`

Baştan Düğüm Silme — O(1)



`gecici = bas` : silinecek düğüm (11) kaybolmasın diye saklanır.

Adım 12 / 16

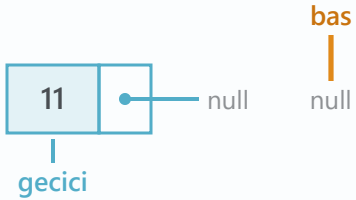
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
▶     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = düğüm(11)`

`gecici = düğüm(11)`

Baştan Düğüm Silme — O(1)



`bas = bas.sonraki` : baş **null**, liste boşaldı; 11'e artık listeden ulaşılamaz.

Adım 13 / 16

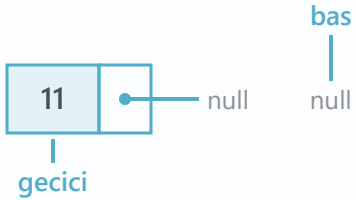
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
▶     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = null`

`gecici = düğüm(11)`

Baştan Düğüm Silme — O(1)



`gecici.sonraki = null` : 11 listeden tamamen koparılır.

Adım 14 / 16

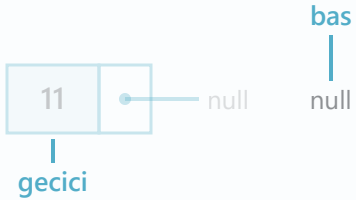
```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
▶     gecici.sonraki = null;  boyut--;
7     return gecici;
8 }
```

DEĞİŞKENLER

`bas = null`

`gecici = düğüm(11)`

Baştan Düğüm Silme — O(1)



`return gecici` : 11 çağırana döner. Kimse tutmazsa çöp toplayıcı (GC) belleği geri alır. **O(1)**.

Adım 15 / 16

```
1 Dugum ilkOgeyiSil() {
2     if (bas == null)
3         return null;
4     Dugum gecici = bas;
5     bas = bas.sonraki;  if (bas == null) son = null;
6     gecici.sonraki = null;  boyut--;
▶     return gecici;
8 }
```

DEĞİŞKENLER

`bas = null`

`gecici = düğüm(11)`

Baştan Düğüm Silme — O(1)



Boş listede çağrılırsa `bas == null` → **return null**. Bu kontrol olmasa `bas.sonraki` NullPointerException verirdi.

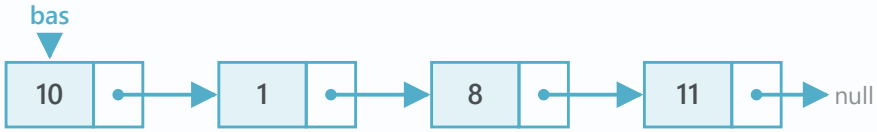
Adım 16 / 16

```
1 Dugum ilkOgeyiSil() {
▶       if (bas == null)
▶           return null;
4       Dugum gecici = bas;
5       bas = bas.sonraki;  if (bas == null) son = null;
6       gecici.sonraki = null;  boyut--;
7       return gecici;
8   }
```

DEĞİŞKENLER

`bas = null`

Sondan Düğüm Silme — O(n)



Liste boş değil.

Adım 1 / 9

```
1  void sonOgeyiSil() {
▶      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

Sondan Düğüm Silme — O(n)



`bas.sonraki` null değil → birden çok düğüm var.

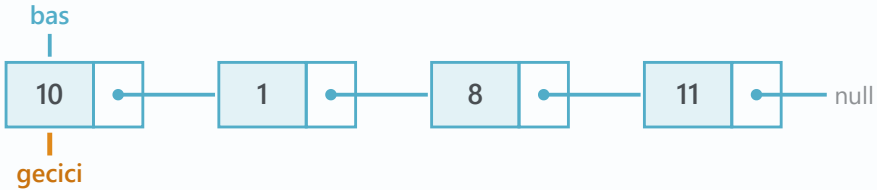
Adım 2 / 9

```
1  void sonOgeyiSil() {
2      if (bas == null) return;
▶      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

Sondan Düğüm Silme — O(n)



`gecici = bas` . Amaç: **sondan bir önceki** düğümü bulmak (onun bağı değişecek).

Adım 3 / 9

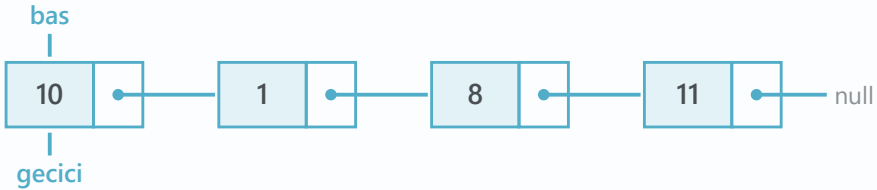
```
1  void sonOgeyiSil() {
2      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
▶      Dugum gecici = bas;
8      while (gecici.sonraki.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

`gecici = düğüm(10)`

Sondan Düğüm Silme — O(n)



düğüm(10).sonraki.sonraki = 8 ≠ null → ilerle.

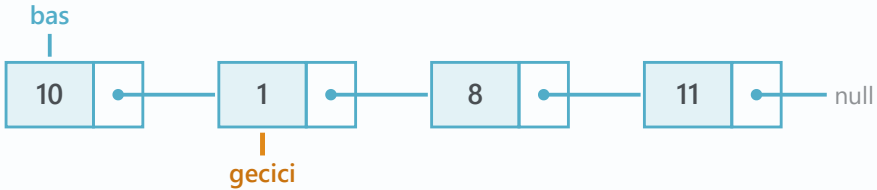
Adım 4 / 9

```
1  void sonOgeyiSil() {
2      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
▶      while (gecici.sonraki.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `gecici = düğüm(10)`

Sondan Düğüm Silme — O(n)



`gecici = gecici.sonraki` → düğüm(1).

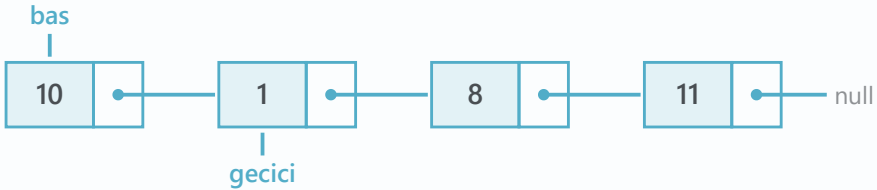
Adım 5 / 9

```
1  void sonOgeyiSil() {
2      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki.sonraki != null)
▶          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `gecici = düğüm(1)`

Sondan Düğüm Silme — O(n)



düğüm(1).sonraki.sonraki = 11 ≠ null → ilerle.

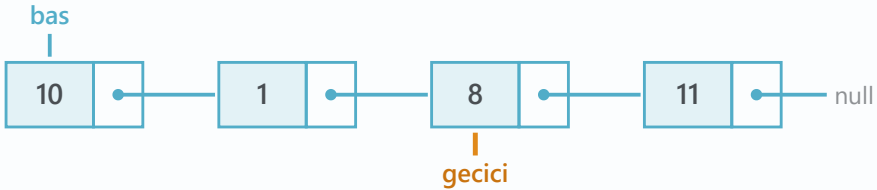
Adım 6 / 9

```
1  void sonOgeyiSil() {
2      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
▶      while (gecici.sonraki.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `gecici = düğüm(1)`

Sondan Düğüm Silme — O(n)



`gecici = gecici.sonraki` → düğüm(8).

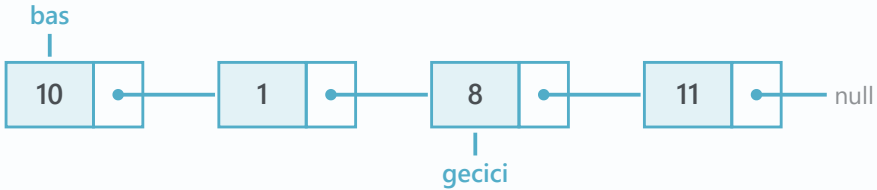
Adım 7 / 9

```
1  void sonOgeyiSil() {
2      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki.sonraki != null)
▶          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `gecici` = **düğüm(8)**

Sondan Düğüm Silme — O(n)



düğüm(8).sonraki.sonraki == null → gecici sondan bir önceki düğümde.

Adım 8 / 9

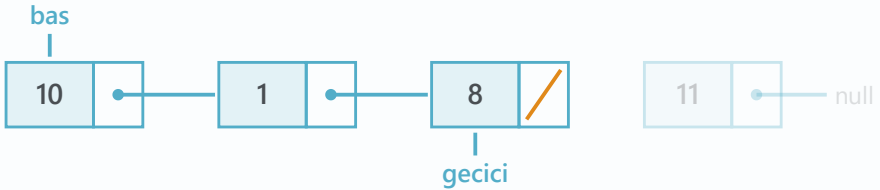
```
1  void sonOgeyiSil() {
2      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
▶      while (gecici.sonraki.sonraki != null)
9          gecici = gecici.sonraki;
10     gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

`gecici = düğüm(8)`

Sondan Düğüm Silme — O(n)



`gecici.sonraki = null` : 11 listeden çıkar → 10 → 1 → 8 → null. **O(n)**; `son` işaretçisi olsa bile geri gidilemediği için hızlanmaz.

Adım 9 / 9

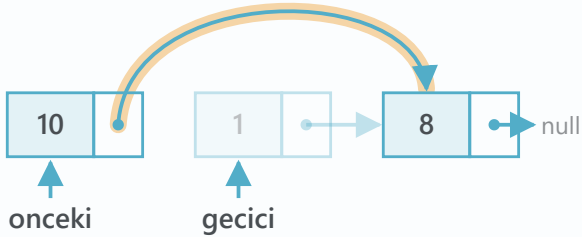
```
1  void sonOgeyiSil() {
2      if (bas == null) return;
3      if (bas.sonraki == null) {  // tek düğüm
4          bas = son = null;  boyut--;
5          return;
6      }
7      Dugum gecici = bas;
8      while (gecici.sonraki.sonraki != null)
9          gecici = gecici.sonraki;
▶      gecici.sonraki = null;  son = gecici;  boyut--;
11 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `gecici = düğüm(8)`

Belirli Bir Öğeyi Silme

```
void ogeyiSil(int silinecek) {
    if (bas == null) return;
    if (bas.veri == silinecek) {
        bas = bas.sonraki;  boyut--;
        if (bas == null) son = null;  return;
    }
    Dugum onceki = bas;
    Dugum gecici = bas.sonraki;
    while (gecici != null) {
        if (gecici.veri == silinecek) {
            onceki.sonraki = gecici.sonraki;
            if (gecici == son) son = onceki;  boyut--;
            return;
        }
        onceki = gecici;
        gecici = gecici.sonraki;
    }
    System.out.println("Bulunamadı.");
}
```

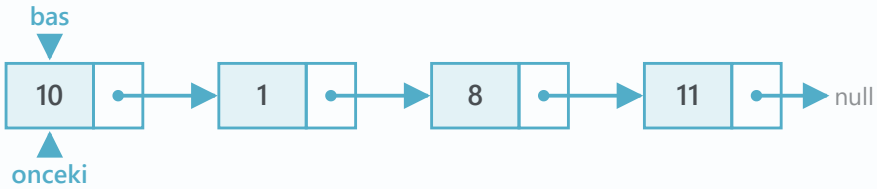


`onceki.sonraki = gecici.sonraki`

silinecek düğüm "atlanır"

- İki işaretçi birlikte yürür: silmek için **öncekine** ihtiyaç var.
- Baş siliniyorsa ayrı durum.
- Arama O(n) + bağlama O(1) → **O(n)**.

Kod İzleme: `ogeyiSil(8)`



`ogeyiSil(8)` : liste boş değil ve baştaki 10 ≠ 8 (ilk iki `if` geçildi). `onceki = bas` .

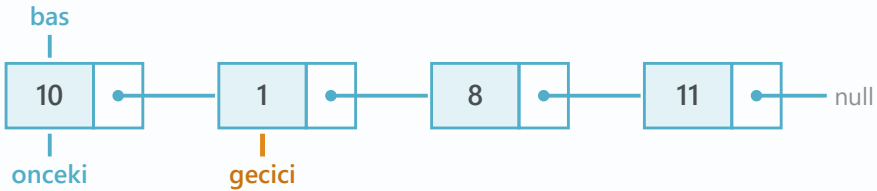
Adım 1 / 10

```
▶ Dugum onceki = bas;
2 Dugum gecici = bas.sonraki;
3 while (gecici != null) {
4     if (gecici.veri == silinecek) {
5         onceki.sonraki = gecici.sonraki;
6         if (gecici == son) son = onceki;  boyut--;
7         return;
8     }
9     onceki = gecici;
10    gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `silinecek` = **8** `onceki` = **düğüm(10)**

Kod İzleme: `ogeyiSil(8)`



`gecici = bas.sonraki` : iki işaretçi yan yana yürür; silmek için **öncekine** ihtiyaç var.

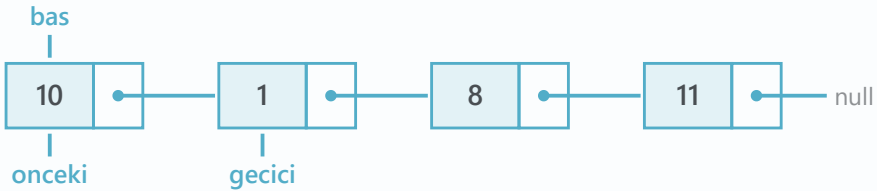
Adım 2 / 10

```
1  Dugum onceki = bas;
▶ 2  Dugum gecici = bas.sonraki;
3  while (gecici != null) {
4      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
9      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `silinecek = 8`
- `onceki = düğüm(10)`
- `gecici = düğüm(1)`

Kod İzleme: `ogeyiSil(8)`



gecici = düğüm(1) null değil → döngü.

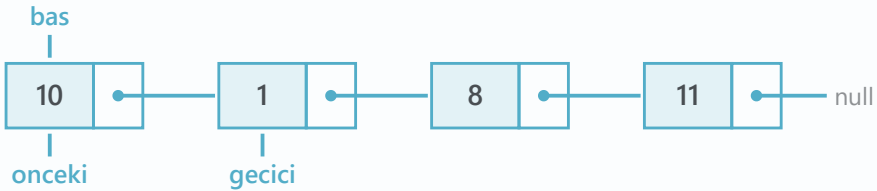
Adım 3 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
▶  while (gecici != null) {
4      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
9      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `silinecek = 8`
- `onceki = düğüm(10)`
- `gecici = düğüm(1)`

Kod İzleme: `ogeyiSil(8)`



1 == 8 mi? Hayır.

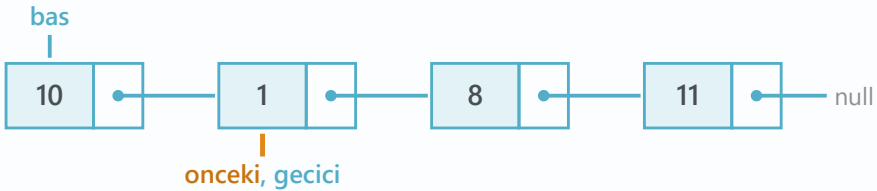
Adım 4 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
3  while (gecici != null) {
▶      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
9      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `silinecek` = **8**
- `onceki` = **düğüm(10)**
- `gecici` = **düğüm(1)**

Kod İzleme: `ogeyiSil(8)`



`onceki = gecici` → düğüm(1).

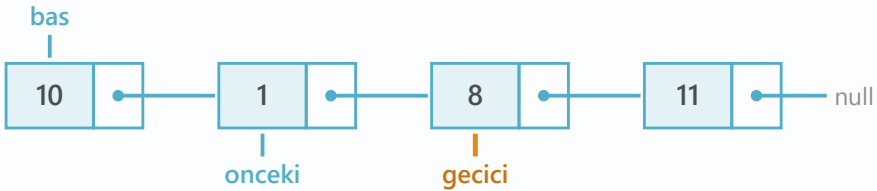
Adım 5 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
3  while (gecici != null) {
4      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
▶      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `silinecek = 8`
- `onceki = düğüm(1)`
- `gecici = düğüm(1)`

Kod İzleme: `ogeyiSil(8)`



`gecici = gecici.sonraki` → düğüm(8).

Adım 6 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
3  while (gecici != null) {
4      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
9      onceki = gecici;
▶      gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

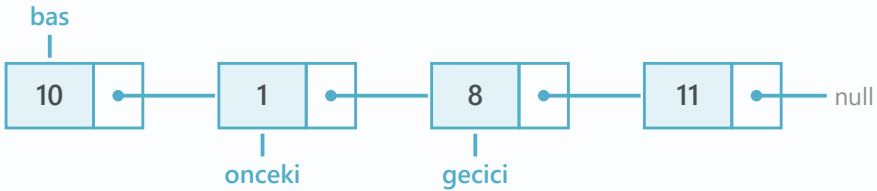
DEĞİŞKENLER

`bas = düğüm(10)` `silinecek = 8`

`onceki = düğüm(1)`

`gecici = düğüm(8)`

Kod İzleme: `ogeyiSil(8)`



gecici = düğüm(8) null değil → döngü.

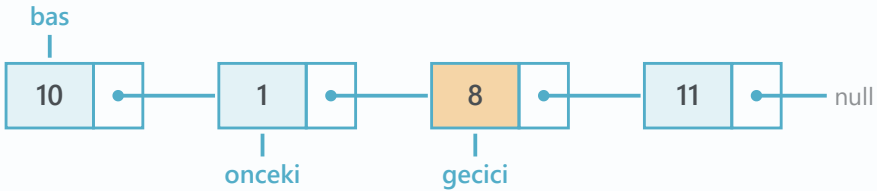
Adım 7 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
▶  while (gecici != null) {
4      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
9      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `silinecek = 8`
- `onceki = düğüm(1)`
- `gecici = düğüm(8)`

Kod İzleme: `ogeyiSil(8)`



8 == 8 → silinecek düğüm bulundu.

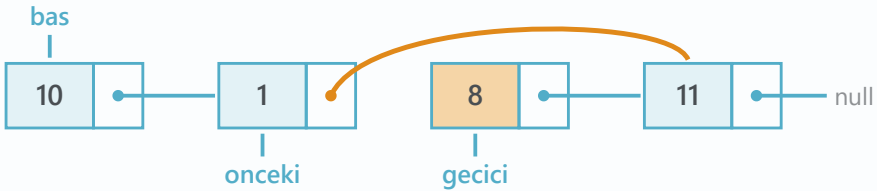
Adım 8 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
3  while (gecici != null) {
▶      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
9      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `silinecek` = **8**
- `onceki` = **düğüm(1)**
- `gecici` = **düğüm(8)**

Kod İzleme: `ogeyiSil(8)`



`onceki.sonraki = gecici.sonraki` : 1 artık 11'i gösterir; 8 "atlanır".

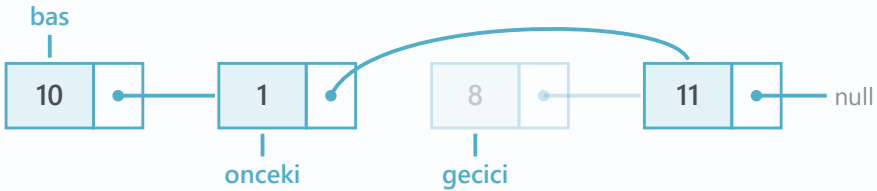
Adım 9 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
3  while (gecici != null) {
4      if (gecici.veri == silinecek) {
▶          onceki.sonraki = gecici.sonraki;
▶          if (gecici == son) son = onceki;  boyut--;
7          return;
8      }
9      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `silinecek` = **8**
- `onceki` = **düğüm(1)**
- `gecici` = **düğüm(8)**

Kod İzleme: `ogeyiSil(8)`



`return`. Liste: 10 → 1 → 11 → null. Arama O(n) + bağlama O(1) → **O(n)**.

Adım 10 / 10

```
1  Dugum onceki = bas;
2  Dugum gecici = bas.sonraki;
3  while (gecici != null) {
4      if (gecici.veri == silinecek) {
5          onceki.sonraki = gecici.sonraki;
6          if (gecici == son) son = onceki;  boyut--;
▶          return;
8      }
9      onceki = gecici;
10     gecici = gecici.sonraki;
11 }
12 System.out.println("Bulunamadı.");
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `silinecek` = **8**

`onceki` = **düğüm(1)**

`gecici` = **düğüm(8)**

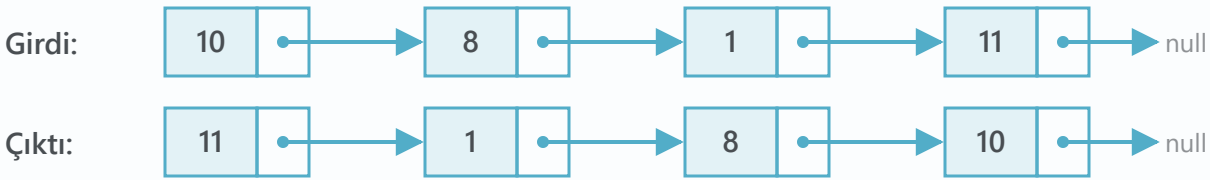
Listeyi Ters Çevirme

```
void listeyiTersCevir() {
    son = bas;                  // eski baş yeni son olur
    Dugum onceki = null;
    Dugum suanki = bas;
    Dugum sonraki = null;
    while (suanki != null) {
        sonraki = suanki.sonraki;  // 1) sonrakini sakla
        suanki.sonraki = onceki;   // 2) bağı ters çevir
        onceki = suanki;           // 3) onceki ilerler
        suanki = sonraki;          // 4) suanki ilerler
    }
    bas = onceki;
}
```

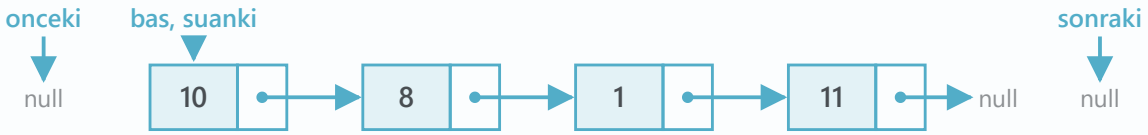
Üç işaretçi
onceki, suanki, sonraki

Yeni düğüm yok
Her turda tek bağ ters çevrilir.

O(n) zaman
O(1) ek bellek



Kod İzleme: `listeyiTersCevir()`



Başlangıç: `onceki = null` , `suanki = bas` (10), `sonraki = null` . Yeni düğüm oluşturulmayacak.

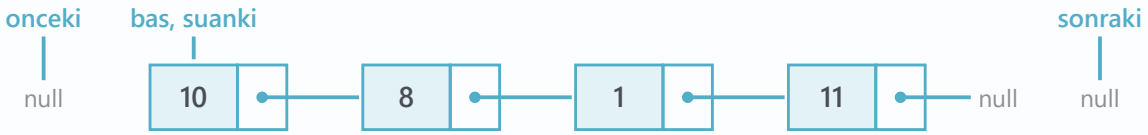
Adım 1 / 19

```
son = bas;                  // eski baş yeni son olur
Dugum onceki = null;
Dugum suanki = bas;
Dugum sonraki = null;
while (suanki != null) {
    sonraki = suanki.sonraki;  // 1) sonrakini sakla
    suanki.sonraki = onceki;   // 2) bağı ters çevir
    onceki = suanki;           // 3) onceki ilerler
    suanki = sonraki;          // 4) suanki ilerler
}
bas = onceki;
```

DEĞİŞKENLER

`bas = düğüm(10)` `onceki = null` `suanki = düğüm(10)` `sonraki = null`

Kod İzleme: `listeyiTersCevir()`



1. tur: suanki = düğüm(10) null değil.

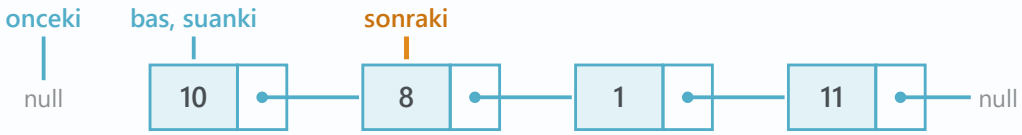
Adım 2 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
▶ 5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `onceki = null`
- `suanki = düğüm(10)`
- `sonraki = null`

Kod İzleme: `listeyiTersCevir()`



1) `sonraki = suanki.sonraki` = düğüm(8): bağ koparılınca listenin kalanı kaybolmasın.

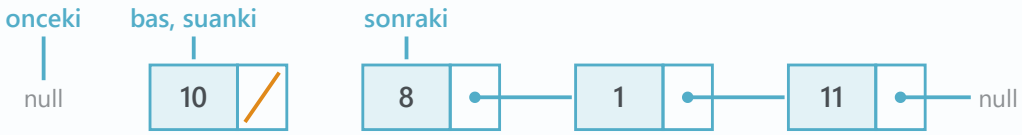
Adım 3 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
▶      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `onceki = null`
- `suanki = düğüm(10)`
- `sonraki = düğüm(8)`

Kod İzleme: `listeyiTersCevir()`



2) `suanki.sonraki = onceki` : 10 artık null'u gösterir (bağ ters döndü).

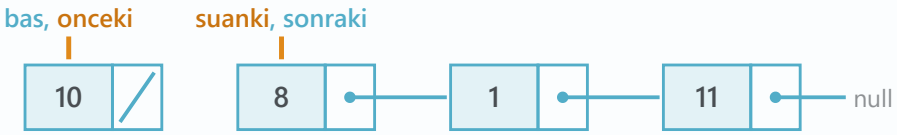
Adım 4 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
▶      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `onceki = null`
- `suanki = düğüm(10)`
- `sonraki = düğüm(8)`

Kod İzleme: `listeyiTersCevir()`



3–4) `onceki = suanki; suanki = sonraki` : iki işaretçi bir düğüm ilerler.

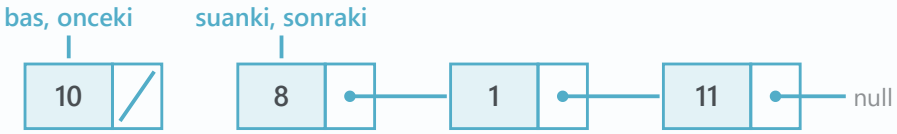
Adım 5 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
▶      onceki = suanki;           // 3) onceki ilerler
▶      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(10)**
- `suanki` = **düğüm(8)**
- `sonraki` = **düğüm(8)**

Kod İzleme: `listeyiTersCevir()`



2. tur: suanki = düğüm(8) null değil.

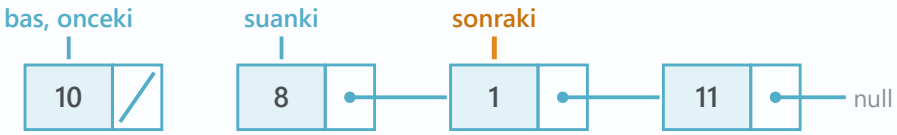
Adım 6 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
▶  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- bas = **düğüm(10)**
- onceki = **düğüm(10)**
- suanki = **düğüm(8)**
- sonraki = **düğüm(8)**

Kod İzleme: `listeyiTersCevir()`



1) `sonraki = suanki.sonraki` = düğüm(1): bağ koparılınca listenin kalanı kaybolmasın.

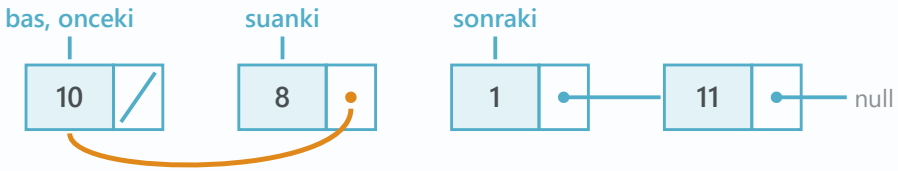
Adım 7 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
▶      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(10)**
- `suanki` = **düğüm(8)**
- `sonraki` = **düğüm(1)**

Kod İzleme: `listeyiTersCevir()`



2) `suanki.sonraki = onceki` : 8 artık düğüm(10)'u gösterir (bağ ters döndü).

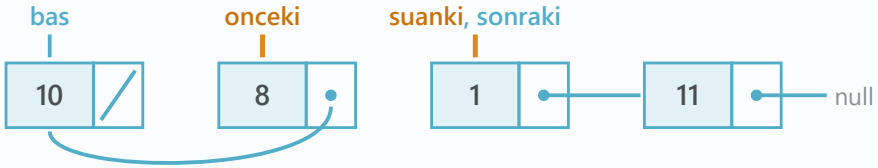
Adım 8 / 19

```
1  son = bas;                 // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
▶      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(10)**
- `suanki` = **düğüm(8)**
- `sonraki` = **düğüm(1)**

Kod İzleme: `listeyiTersCevir()`



3–4) `onceki = suanki; suanki = sonraki` : iki işaretçi bir düğüm ilerler.

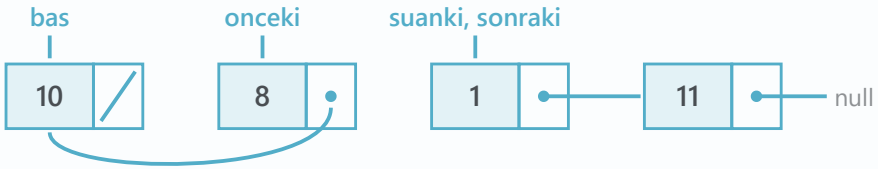
Adım 9 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
▶      onceki = suanki;           // 3) onceki ilerler
▶      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(8)**
- `suanki` = **düğüm(1)**
- `sonraki` = **düğüm(1)**

Kod İzleme: `listeyiTersCevir()`



3. tur: suanki = düğüm(1) null değil.

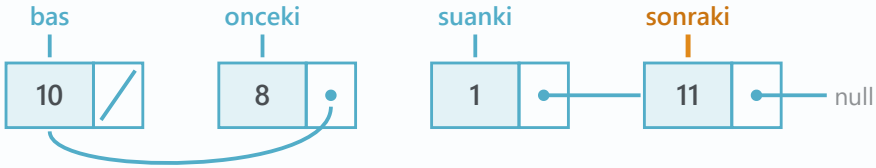
Adım 10 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
▶  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(8)**
- `suanki` = **düğüm(1)**
- `sonraki` = **düğüm(1)**

Kod İzleme: `listeyiTersCevir()`



1) `sonraki = suanki.sonraki` = düğüm(11): bağ koparılınca listenin kalanı kaybolmasın.

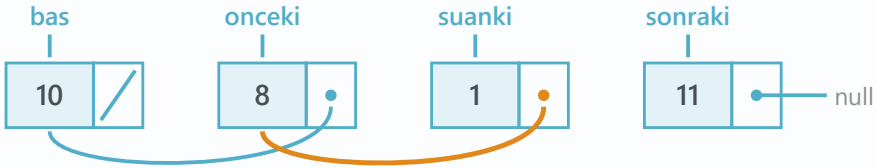
Adım 11 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
▶      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- bas = **düğüm(10)**
- onceki = **düğüm(8)**
- suanki = **düğüm(1)**
- sonraki = **düğüm(11)**

Kod İzleme: `listeyiTersCevir()`



2) `suanki.sonraki = onceki` : 1 artık düğüm(8)'i gösterir (bağ ters döndü).

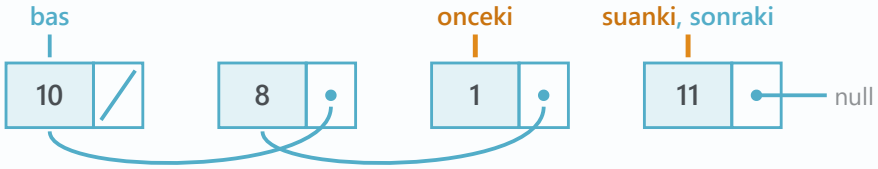
Adım 12 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;   // 1) sonrakini sakla
▶      suanki.sonraki = onceki;    // 2) bağı ters çevir
8      onceki = suanki;            // 3) onceki ilerler
9      suanki = sonraki;           // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(8)**
- `suanki` = **düğüm(1)**
- `sonraki` = **düğüm(11)**

Kod İzleme: `listeyiTersCevir()`



3–4) `onceki = suanki; suanki = sonraki` : iki işaretçi bir düğüm ilerler.

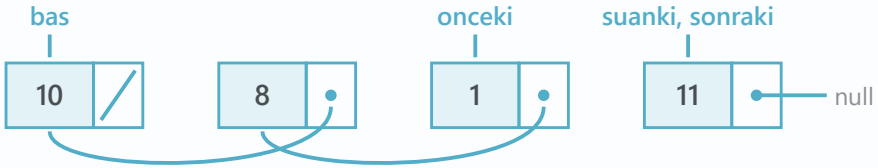
Adım 13 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
▶      onceki = suanki;           // 3) onceki ilerler
▶      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(1)**
- `suanki` = **düğüm(11)**
- `sonraki` = **düğüm(11)**

Kod İzleme: `listeyiTersCevir()`



4. tur: suanki = düğüm(11) null değil.

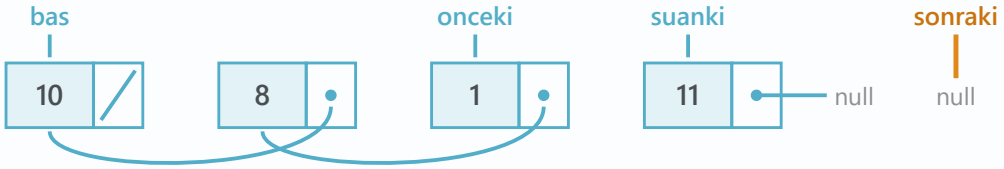
Adım 14 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
▶  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(1)**
- `suanki` = **düğüm(11)**
- `sonraki` = **düğüm(11)**

Kod İzleme: `listeyiTersCevir()`



1) `sonraki = suanki.sonraki` = null: bağ koparılınca listenin kalanı kaybolmasın.

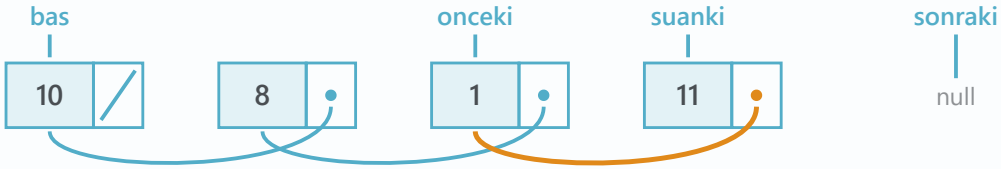
Adım 15 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
▶      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- bas = **düğüm(10)**
- onceki = **düğüm(1)**
- suanki = **düğüm(11)**
- sonraki = **null**

Kod İzleme: `listeyiTersCevir()`



2) `suanki.sonraki = onceki` : 11 artık düğüm(1)'i gösterir (bağ ters döndü).

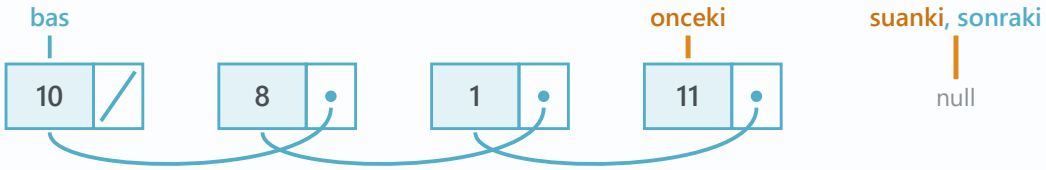
Adım 16 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
▶      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `onceki` = **düğüm(1)**
- `suanki` = **düğüm(11)**
- `sonraki` = **null**

Kod İzleme: `listeyiTersCevir()`



3–4) `onceki = suanki; suanki = sonraki` : iki işaretçi bir düğüm ilerler.

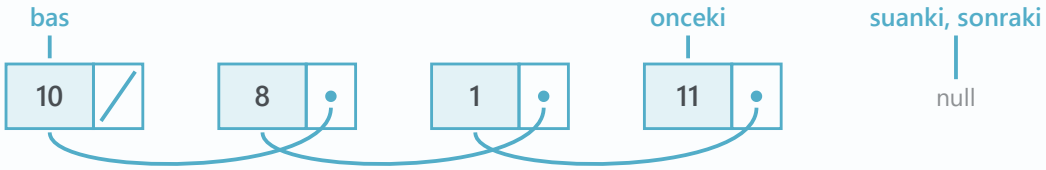
Adım 17 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
▶      onceki = suanki;           // 3) onceki ilerler
▶      suanki = sonraki;          // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `onceki = düğüm(11)`
- `suanki = null`
- `sonraki = null`

Kod İzleme: `listeyiTersCevir()`



`suanki == null` → döngü biter; `onceki` eski son düğümde (11).

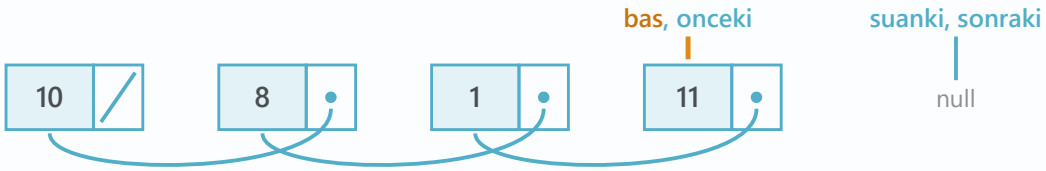
Adım 18 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
▶  while (suanki != null) {
6      sonraki = suanki.sonraki;   // 1) sonrakini sakla
7      suanki.sonraki = onceki;    // 2) bağı ters çevir
8      onceki = suanki;            // 3) onceki ilerler
9      suanki = sonraki;           // 4) suanki ilerler
10 }
11 bas = onceki;
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `onceki = düğüm(11)`
- `suanki = null`
- `sonraki = null`

Kod İzleme: `listeyiTersCevir()`



`bas = onceki` : yeni liste 11 → 1 → 8 → 10 → null. **O(n)** zaman, **O(1)** ek bellek.

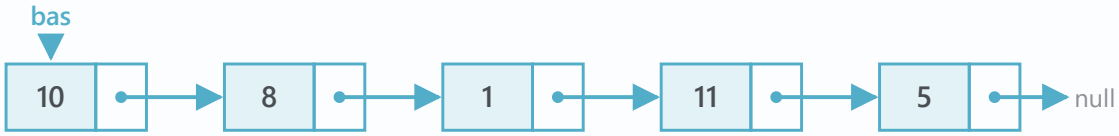
Adım 19 / 19

```
1  son = bas;                  // eski baş yeni son olur
2  Dugum onceki = null;
3  Dugum suanki = bas;
4  Dugum sonraki = null;
5  while (suanki != null) {
6      sonraki = suanki.sonraki;  // 1) sonrakini sakla
7      suanki.sonraki = onceki;   // 2) bağı ters çevir
8      onceki = suanki;           // 3) onceki ilerler
9      suanki = sonraki;          // 4) suanki ilerler
10 }
▶  bas = onceki;
```

DEĞİŞKENLER

- `bas = düğüm(11)`
- `onceki = düğüm(11)`
- `suanki = null`
- `sonraki = null`

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



Liste boş değil (boşsa istisna fırlatılır).

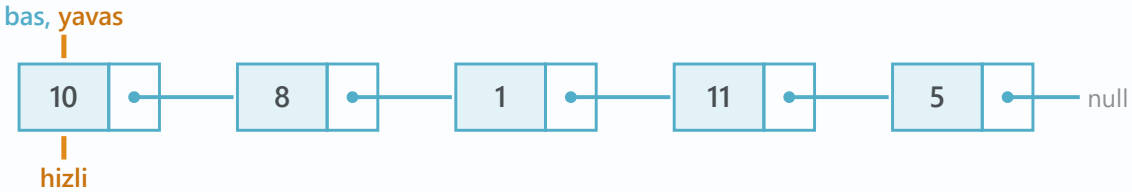
Adım 1 / 10

```
1  int ortaOgeyiBul() {
▶      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
5      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

`bas = düğüm(10)`

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



İki işaretçi de baştan başlar. Uzunluğu bilmeden ortayı bulacağız.

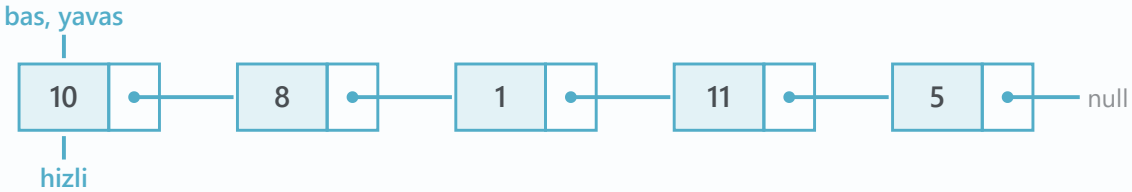
Adım 2 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
▶      Dugum yavas = bas, hizli = bas;
5      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `yavas = düğüm(10)`
- `hizli = düğüm(10)`

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



hizli = düğüm(10) ve hizli.sonraki = düğüm(8) null değil → 1. tur.

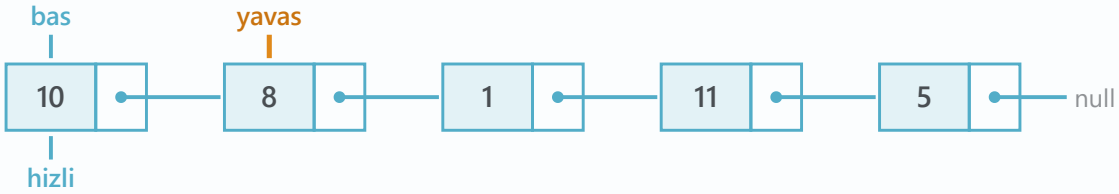
Adım 3 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
▶      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `yavas = düğüm(10)`
- `hizli = düğüm(10)`

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



`yavas` 1 adım → düğüm(8).

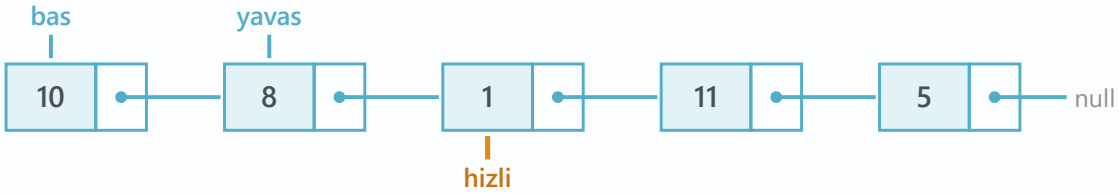
Adım 4 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
5      while (hizli != null && hizli.sonraki != null) {
▶          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `yavas` = **düğüm(8)**
- `hizli` = **düğüm(10)**

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



`hizli` 2 adım → düğüm(1).

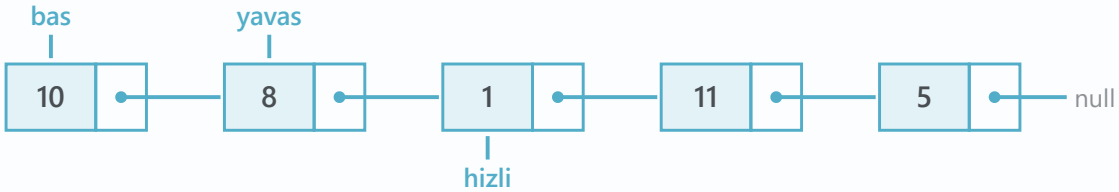
Adım 5 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
5      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
▶          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `yavas` = **düğüm(8)**
- `hizli` = **düğüm(1)**

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



hizli = düğüm(1) ve hizli.sonraki = düğüm(11) null değil → 2. tur.

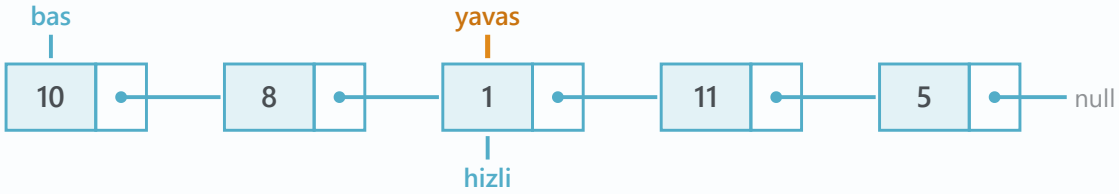
Adım 6 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
▶      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `yavas` = **düğüm(8)**
- `hizli` = **düğüm(1)**

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



`yavas` 1 adım → düğüm(1).

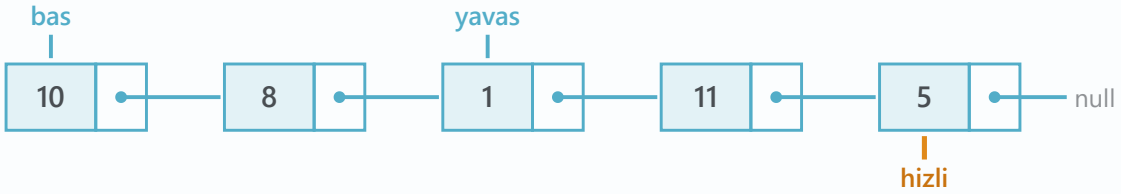
Adım 7 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
5      while (hizli != null && hizli.sonraki != null) {
▶          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `yavas` = **düğüm(1)**
- `hizli` = **düğüm(1)**

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



`hizli` 2 adım → düğüm(5).

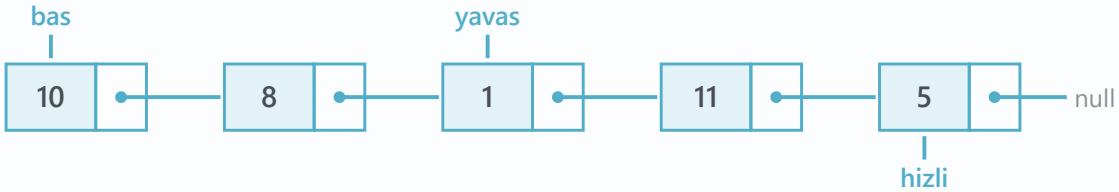
Adım 8 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
5      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
▶          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `yavas` = **düğüm(1)**
- `hizli` = **düğüm(5)**

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



hizli.sonraki == null → hızlı sona vardı, döngü biter. 2 turda yavas 2, hizli 4 adım attı.

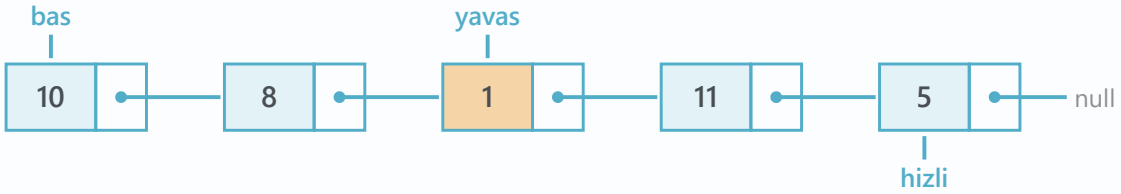
Adım 9 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
▶      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
9      return yavas.veri;
10 }
```

DEĞİŞKENLER

- bas = **düğüm(10)**
- yavas = **düğüm(1)**
- hizli = **düğüm(5)**

Orta Öğeyi Bulma: Yavaş ve Hızlı İşaretçi



return 1: yavaş işaretçi ortada. Tek geçiş, **O(n)**. Çift uzunlukta ikinci orta döner (10, 8, 1, 11 → 1).

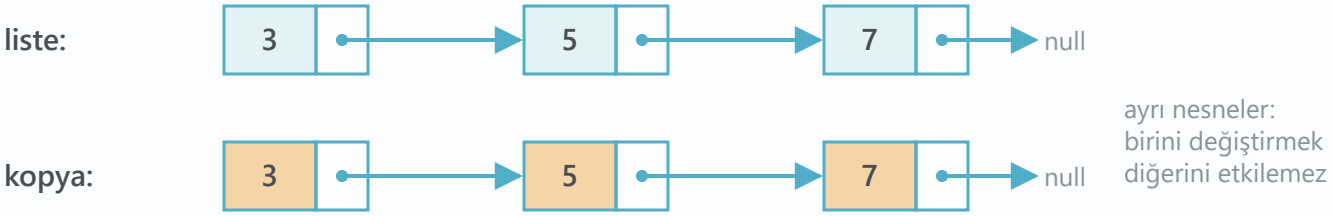
Adım 10 / 10

```
1  int ortaOgeyiBul() {
2      if (bas == null)
3          throw new IllegalStateException("Liste boş");
4      Dugum yavas = bas, hizli = bas;
5      while (hizli != null && hizli.sonraki != null) {
6          yavas = yavas.sonraki;          // 1 adım
7          hizli = hizli.sonraki.sonraki;  // 2 adım
8      }
▶      return yavas.veri;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `yavas` = **düğüm(1)**
- `hizli` = **düğüm(5)**

Listeyi Kopyalama




```
TekYonluBagliListe kopyala() {
    TekYonluBagliListe kopya =
        new TekYonluBagliListe();
    Dugum gecici = bas;
    while (gecici != null) {
        kopya.sonaEkle(gecici.veri);
        gecici = gecici.sonraki;
    }
    return kopya;
}
```

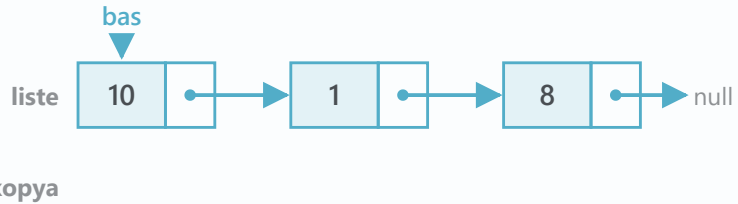
Derin kopya

Yeni düğümler oluşturulur; `kopya.bas = bas` yazmak yalnızca aynı listeye ikinci bir referans olurdu.

Maliyet

`sonaEkle` her seferinde sona kadar gezinirse toplam $O(n^2)$; son işaretçisiyle $O(n)$.

Kod İzleme: `kopyala()`



Yeni, boş bir liste oluşturulur. `kopya.bas = bas` yazmak kopya olmazdı: aynı düğümlere ikinci bir referans olurdu.

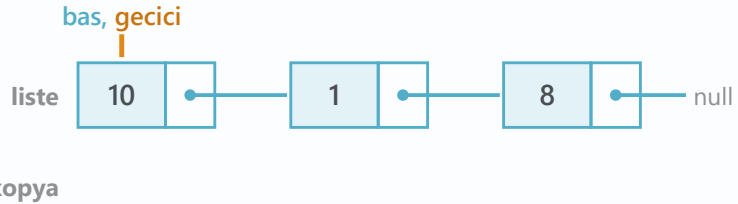
Adım 1 / 13

```
1  TekYonluBagliListe kopyala() {
▶      TekYonluBagliListe kopya =
▶          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `kopya = boş`

Kod İzleme: `kopyala()`



`gecici = bas` : orijinal liste baştan gezilecek.

Adım 2 / 13

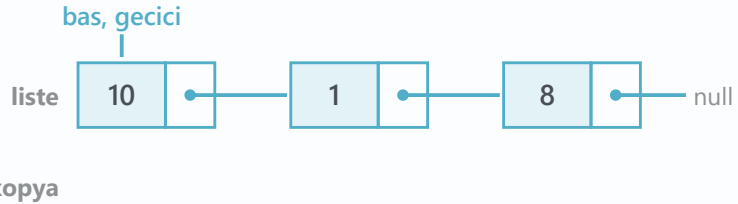
```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
▶      Dugum gecici = bas;
5      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `kopya = boş`

`gecici = düğüm(10)`

Kod İzleme: `kopyala()`



gecici = düğüm(10) null değil.

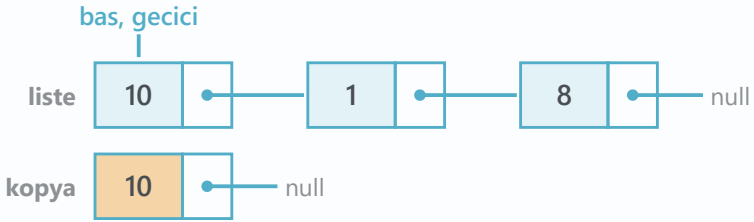
Adım 3 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
▶      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `kopya` = **boş** `gecici` = **düğüm(10)**

Kod İzleme: `kopyala()`



`kopya.sonaEkle(10)` : **yeni** bir düğüm oluşturulup kopyanın sonuna bağlanır.

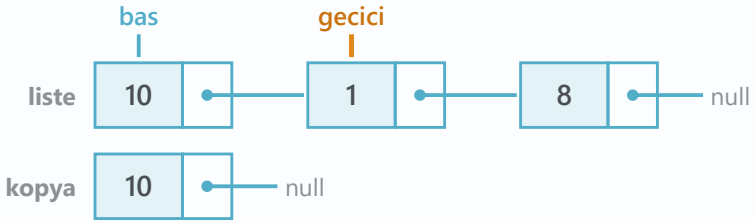
Adım 4 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
▶          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `kopya = 10` `gecici = düğüm(10)`

Kod İzleme: `kopyala()`



`gecici = gecici.sonraki` → düğüm(1).

Adım 5 / 13

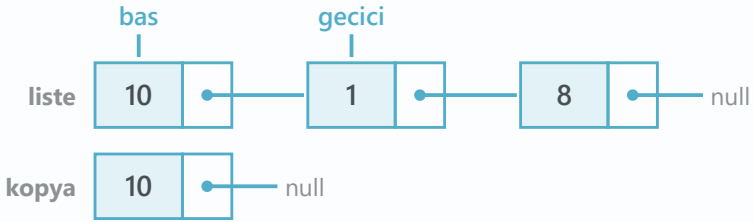
```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
▶          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `kopya = 10`

`gecici = düğüm(1)`

Kod İzleme: `kopyala()`



gecici = düğüm(1) null değil.

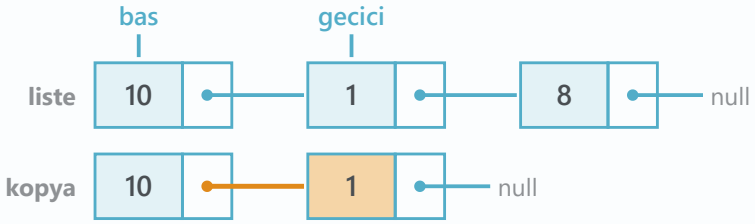
Adım 6 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
▶      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `kopya = 10` `gecici = düğüm(1)`

Kod İzleme: `kopyala()`



`kopya.sonaEkle(1)` : **yeni** bir düğüm oluşturulup kopyanın sonuna bağlanır.

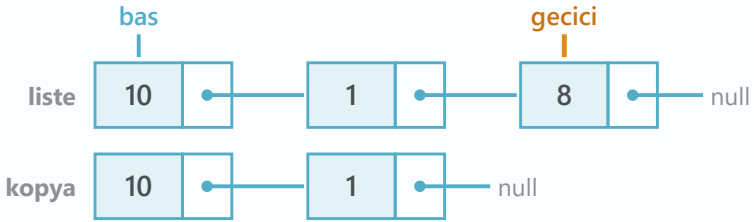
Adım 7 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
▶          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas = düğüm(10)` `kopya = 10→1` `gecici = düğüm(1)`

Kod İzleme: `kopyala()`



`gecici = gecici.sonraki` → düğüm(8).

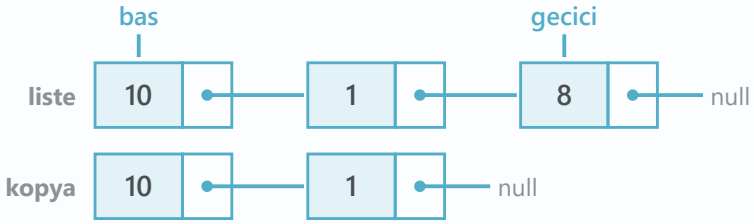
Adım 8 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
▶          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `kopya` = **10→1** `gecici` = **düğüm(8)**

Kod İzleme: `kopyala()`



gecici = düğüm(8) null değil.

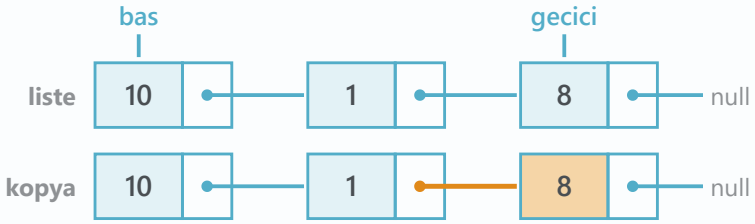
Adım 9 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
▶      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)** `kopya` = **10→1** `gecici` = **düğüm(8)**

Kod İzleme: `kopyala()`



`kopya.sonaEkle(8)` : **yeni** bir düğüm oluşturulup kopyanın sonuna bağlanır.

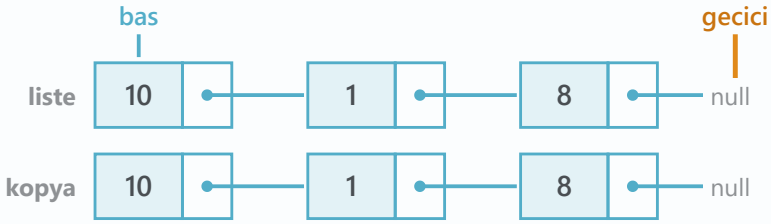
Adım 10 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
▶          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

- `bas = düğüm(10)`
- `kopya = 10→1→8`
- `gecici = düğüm(8)`

Kod İzleme: `kopyala()`



`gecici = gecici.sonraki` → null.

Adım 11 / 13

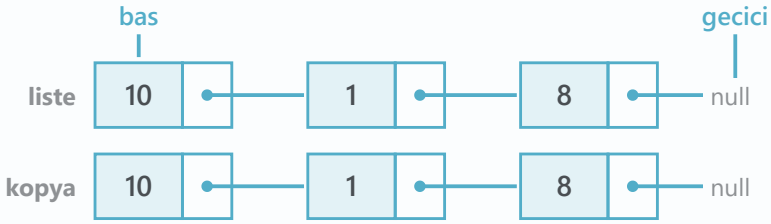
```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
▶          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)**

`kopya` = **10→1→8** `gecici` = **null**

Kod İzleme: `kopyala()`



gecici == null → döngü biter.

Adım 12 / 13

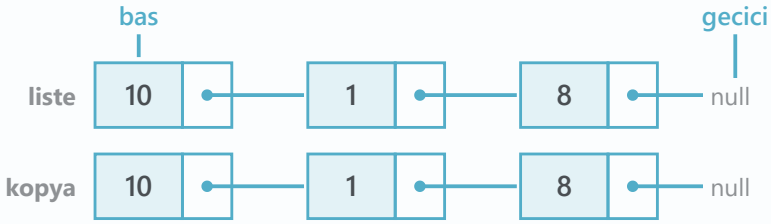
```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
▶      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
9      return kopya;
10 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)**

`kopya` = **10→1→8** `gecici` = **null**

Kod İzleme: `kopyala()`



`return kopya` : aynı değerler, farklı düğümler (derin kopya). sonaEkle O(n) ise toplam O(n^2); son işaretçisiyle **O(n)**.

Adım 13 / 13

```
1  TekYonluBagliListe kopyala() {
2      TekYonluBagliListe kopya =
3          new TekYonluBagliListe();
4      Dugum gecici = bas;
5      while (gecici != null) {
6          kopya.sonaEkle(gecici.veri);
7          gecici = gecici.sonraki;
8      }
▶      return kopya;
10 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)**

`kopya` = **10→1→8**

`gecici` = **null**

Listeden Tekrarlayanları Kaldırma

```
void tekrarlayanlariKaldir() {
    TekYonluBagliListe gorulen =
        new TekYonluBagliListe();
    Dugum onceki = null, gecici = bas;
    while (gecici != null) {
        int v = gecici.veri;
        if (gorulen.iceriyor(v)) {
            onceki.sonraki =
                gecici.sonraki;   // atla
            if (gecici == son) son = onceki;  boyut--;
        } else {
            gorulen.basaEkle(v);
            onceki = gecici;
        }
        gecici = gecici.sonraki;
    }
}
```

Fikir

Görülen değerler ikinci bir listede tutulur; tekrar eden düğüm "atlanır".

Maliyet

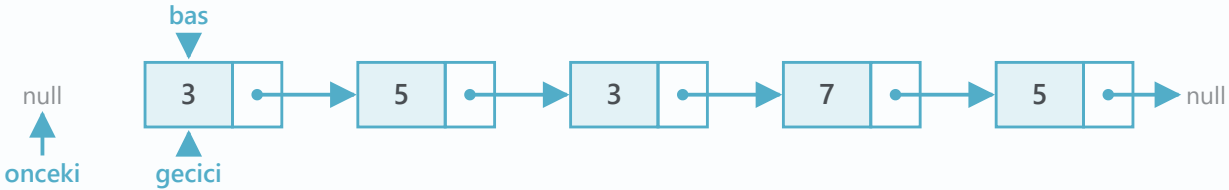
Her düğüm için `gorulen` listesinde arama: $O(n^2)$. Bir hash kümesi ile $O(n)$.

Not

İlk düğüm hiçbir zaman tekrar olmaz, bu yüzden `onceki` null iken silme olmaz.

`iceriyor(v)`: `ogeyiAra(v) != -1`. Örnek: 3 → 5 → 3 → 7 → 5 ⇒ 3 → 5 → 7

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



Başlangıç (önceki satırlar): `gorulen` boş, `onceki = null`, `gecici = bas`. gecici = düğüm(3) null değil.

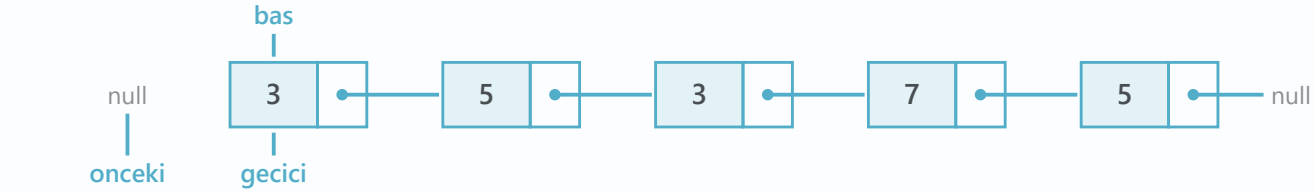
Adım 1 / 15

```
▶ while (gecici != null) {
2     int v = gecici.veri;
3     if (gorulen.iceriyor(v)) {
4         onceki.sonraki =
5             gecici.sonraki;   // atla
6         if (gecici == son) son = onceki;  boyut--;
7     } else {
8         gorulen.basaEkle(v);
9         onceki = gecici;
10    }
11    gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = boş`
- `onceki = null`
- `gecici = düğüm(3)`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`v = 3` .

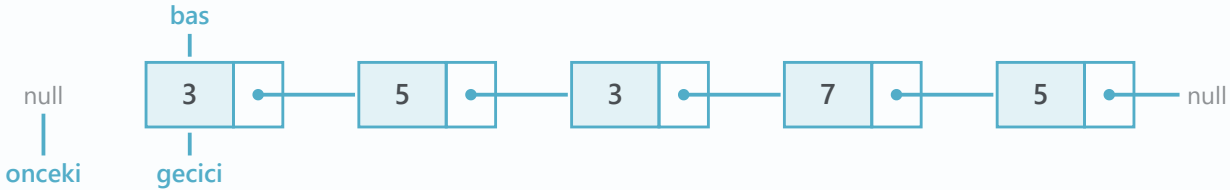
Adım 2 / 15

```
1  while (gecici != null) {
▶       int v = gecici.veri;
3       if (gorulen.iceriyor(v)) {
4           onceki.sonraki =
5               gecici.sonraki;    // atla
6           if (gecici == son) son = onceki;  boyut--;
7       } else {
8           gorulen.basaEkle(v);
9           onceki = gecici;
10      }
11      gecici = gecici.sonraki;
12  }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = boş`
- `onceki = null`
- `gecici = düğüm(3)`
- `v = 3`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gorulen.iceriyor(3)` → false: 3 ilk kez görülüyor.

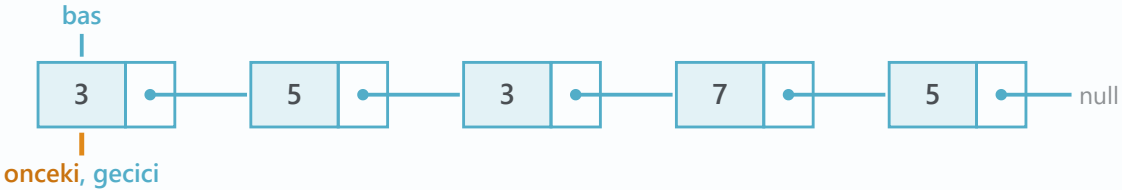
Adım 3 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
▶      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

`bas = düğüm(3)` `gorulen = boş` `onceki = null` `gecici = düğüm(3)` `v = 3`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gorulen.basaEkle(3)` ; `onceki = gecici` : düğüm kalıyor, onceki ona ilerler.

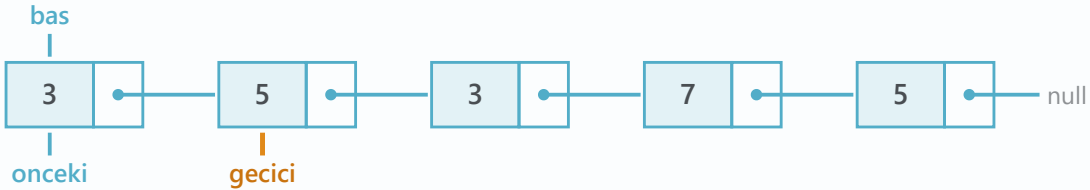
Adım 4 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
▶          gorulen.basaEkle(v);
▶          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

bas = **düğüm(3)** gorulen = **3** onceki = **düğüm(3)** gecici = **düğüm(3)** v = **3**

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gecici = gecici.sonraki` → düğüm(5).

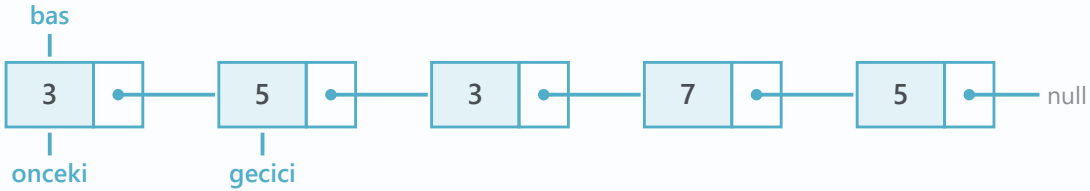
Adım 5 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
▶      gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

`bas = düğüm(3)` `gorulen = 3`
`onceki = düğüm(3)`
`gecici = düğüm(5)` `v = 3`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



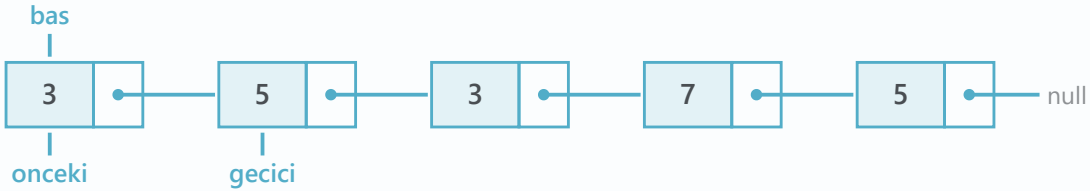
gecici = düğüm(5) null değil.


```
▶ while (gecici != null) {
2     int v = gecici.veri;
3     if (gorulen.iceriyor(v)) {
4         onceki.sonraki =
5             gecici.sonraki;   // atla
6         if (gecici == son) son = onceki;  boyut--;
7     } else {
8         gorulen.basaEkle(v);
9         onceki = gecici;
10    }
11    gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- bas = **düğüm(3)**
- gorulen = **3**
- onceki = **düğüm(3)**
- gecici = **düğüm(5)**

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`v = 5` .

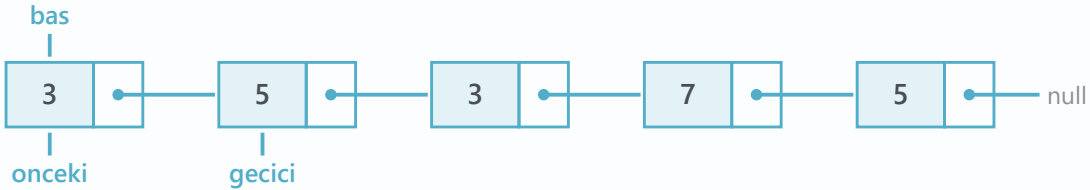
Adım 7 / 15

```
1  while (gecici != null) {
▶       int v = gecici.veri;
3       if (gorulen.iceriyor(v)) {
4           onceki.sonraki =
5               gecici.sonraki;   // atla
6           if (gecici == son) son = onceki;  boyut--;
7       } else {
8           gorulen.basaEkle(v);
9           onceki = gecici;
10      }
11      gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

`bas = düğüm(3)` `gorulen = 3`
`onceki = düğüm(3)`
`gecici = düğüm(5)` `v = 5`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gorulen.iceriyor(5)` → false: 5 ilk kez görülüyor.

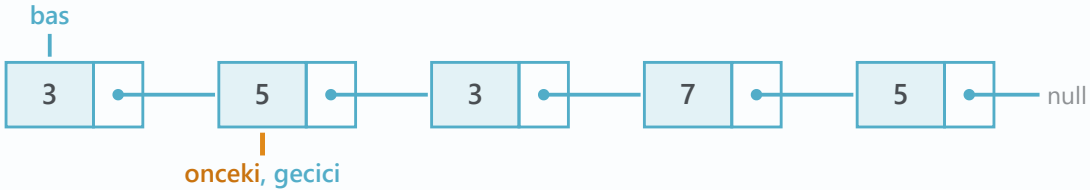
Adım 8 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
▶      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas` = **düğüm(3)**
- `gorulen` = **3**
- `onceki` = **düğüm(3)**
- `gecici` = **düğüm(5)**
- `v` = **5**

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gorulen.basaEkle(5)` ; `onceki = gecici` : düğüm kalıyor, onceki ona ilerler.

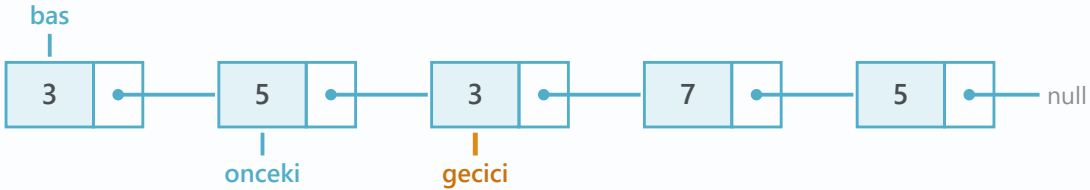
Adım 9 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
▶          gorulen.basaEkle(v);
▶          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = 5→3`
- `onceki = düğüm(5)`
- `gecici = düğüm(5)`
- `v = 5`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gecici = gecici.sonraki` → düğüm(3).

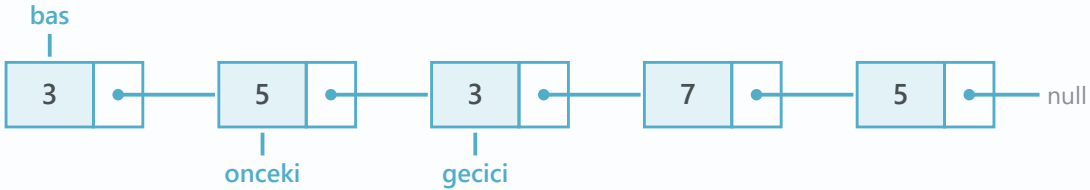
Adım 10 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
▶      gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = 5→3`
- `onceki = düğüm(5)`
- `gecici = düğüm(3)`
- `v = 5`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



gecici = düğüm(3) null değil.

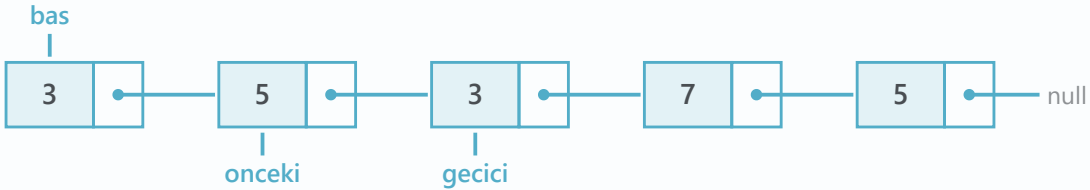
Adım 11 / 15

```
▶ while (gecici != null) {
2     int v = gecici.veri;
3     if (gorulen.iceriyor(v)) {
4         onceki.sonraki =
5             gecici.sonraki;   // atla
6         if (gecici == son) son = onceki;  boyut--;
7     } else {
8         gorulen.basaEkle(v);
9         onceki = gecici;
10    }
11    gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- bas = **düğüm(3)**
- gorulen = **5→3**
- onceki = **düğüm(5)**
- gecici = **düğüm(3)**

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`v = 3` .

Adım 12 / 15

```
1  while (gecici != null) {
▶       int v = gecici.veri;
3       if (gorulen.iceriyor(v)) {
4           onceki.sonraki =
5               gecici.sonraki;   // atla
6           if (gecici == son) son = onceki;  boyut--;
7       } else {
8           gorulen.basaEkle(v);
9           onceki = gecici;
10      }
11      gecici = gecici.sonraki;
12 }
```

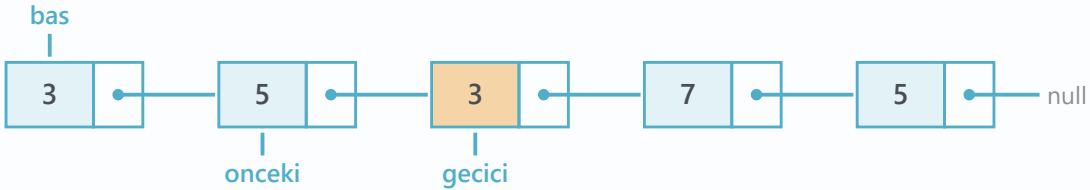
DEĞİŞKENLER

`bas = düğüm(3)` `gorulen = 5→3`

`onceki = düğüm(5)`

`gecici = düğüm(3)` `v = 3`

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gorulen.iceriyor(3)` → **true**: 3 daha önce görüldü, bu düğüm tekrar.

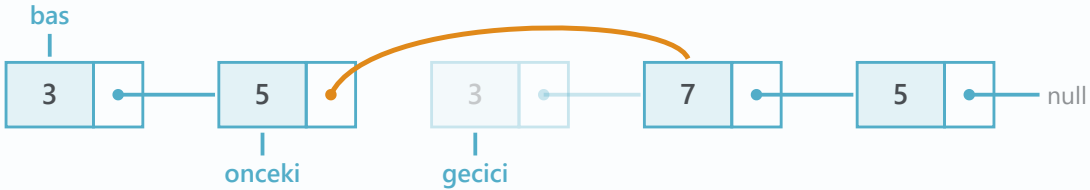
Adım 13 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
▶      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas` = **düğüm(3)**
- `gorulen` = **5→3**
- `onceki` = **düğüm(5)**
- `gecici` = **düğüm(3)**
- `v` = **3**

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`onceki.sonraki = gecici.sonraki` : 5 → düğüm(7); tekrar eden 3 atlanır. `onceki` yerinde kalır.

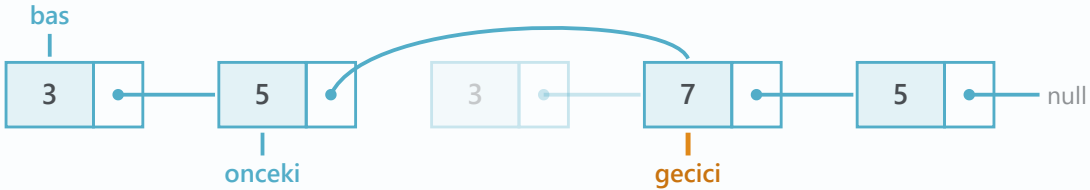
Adım 14 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
▶          onceki.sonraki =
▶              gecici.sonraki;   // atla
▶          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas` = **düğüm(3)**
- `gorulen` = **5→3**
- `onceki` = **düğüm(5)**
- `gecici` = **düğüm(3)**
- `v` = **3**

Kod İzleme: Tekrarları Kaldırma (3, 5, 3, 7, 5)



`gecici = gecici.sonraki` → düğüm(7).

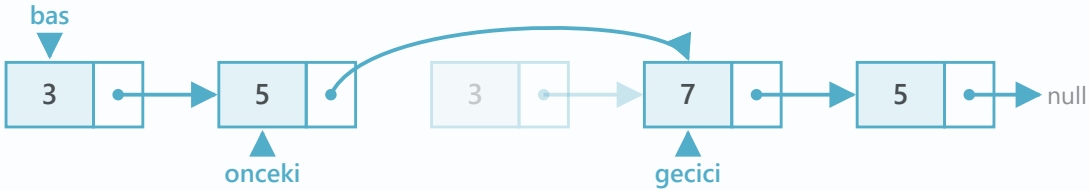
Adım 15 / 15

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
▶      gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

`bas = düğüm(3)` `gorulen = 5→3`
`onceki = düğüm(5)`
`gecici = düğüm(7)` `v = 3`

Kod İzleme: Tekrarları Kaldırma (devam)



gecici = düğüm(7) null değil.

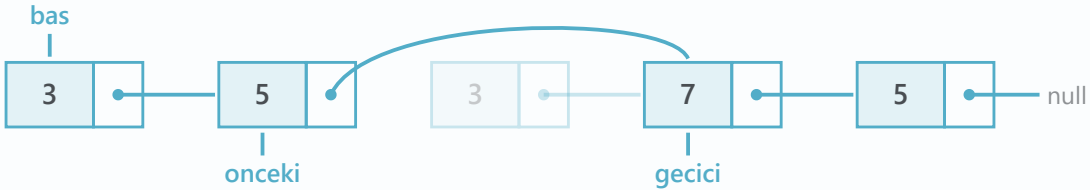
Adım 1 / 11

```
▶ while (gecici != null) {
2     int v = gecici.veri;
3     if (gorulen.iceriyor(v)) {
4         onceki.sonraki =
5             gecici.sonraki;   // atla
6         if (gecici == son) son = onceki;  boyut--;
7     } else {
8         gorulen.basaEkle(v);
9         onceki = gecici;
10    }
11    gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- bas = **düğüm(3)**
- gorulen = **5→3**
- onceki = **düğüm(5)**
- gecici = **düğüm(7)**

Kod İzleme: Tekrarları Kaldırma (devam)



`v = 7` .

Adım 2 / 11

```
1 while (gecici != null) {
▶       int v = gecici.veri;
3       if (gorulen.iceriyor(v)) {
4           onceki.sonraki =
5               gecici.sonraki;   // atla
6           if (gecici == son) son = onceki;  boyut--;
7       } else {
8           gorulen.basaEkle(v);
9           onceki = gecici;
10      }
11      gecici = gecici.sonraki;
12  }
```

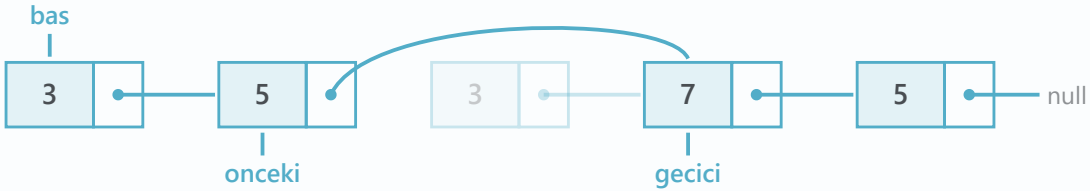
DEĞİŞKENLER

`bas = düğüm(3)` `gorulen = 5→3`

`onceki = düğüm(5)`

`gecici = düğüm(7)` `v = 7`

Kod İzleme: Tekrarları Kaldırma (devam)



`gorulen.iceriyor(7)` → false: 7 ilk kez görülüyor.

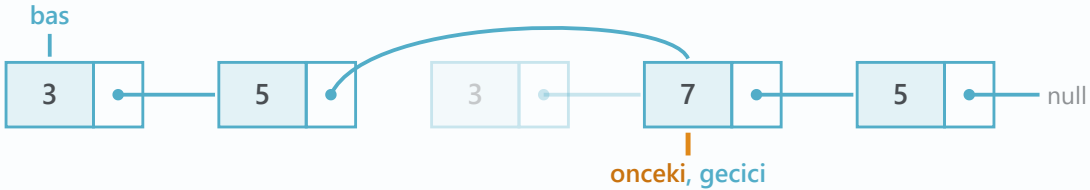
Adım 3 / 11

```
1  while (gecici != null) {
2      int v = gecici.veri;
▶      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas` = **düğüm(3)**
- `gorulen` = **5→3**
- `onceki` = **düğüm(5)**
- `gecici` = **düğüm(7)**
- `v` = **7**

Kod İzleme: Tekrarları Kaldırma (devam)



`gorulen.basaEkle(7)` ; `onceki = gecici` : düğüm kalıyor, onceki ona ilerler.

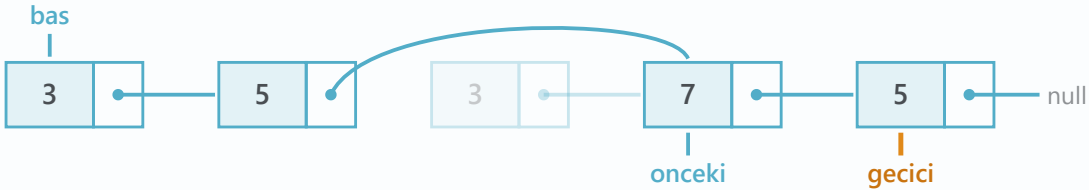
Adım 4 / 11

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
▶          gorulen.basaEkle(v);
▶          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas` = **düğüm(3)**
- `gorulen` = **7→5→3**
- `onceki` = **düğüm(7)**
- `gecici` = **düğüm(7)**
- `v` = **7**

Kod İzleme: Tekrarları Kaldırma (devam)



`gecici = gecici.sonraki` → düğüm(5).

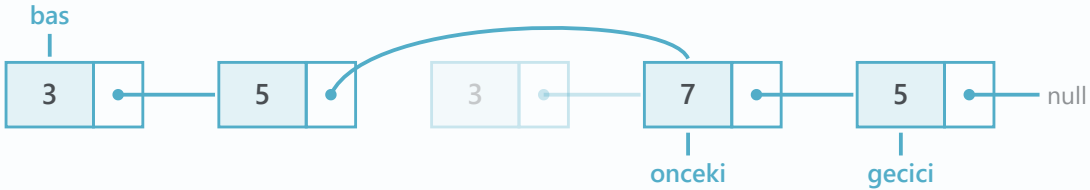
Adım 5 / 11

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
▶      gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = 7→5→3`
- `onceki = düğüm(7)`
- `gecici = düğüm(5)`
- `v = 7`

Kod İzleme: Tekrarları Kaldırma (devam)



gecici = düğüm(5) null değil.

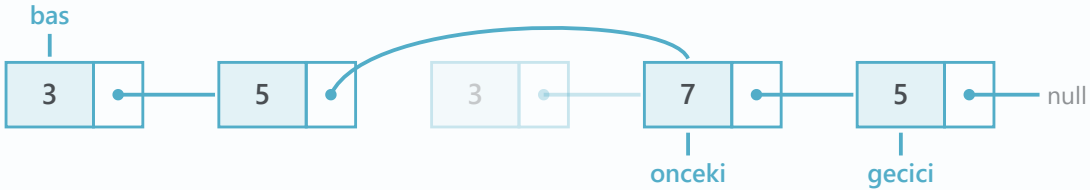
Adım 6 / 11

```
▶ while (gecici != null) {
2     int v = gecici.veri;
3     if (gorulen.iceriyor(v)) {
4         onceki.sonraki =
5             gecici.sonraki;   // atla
6         if (gecici == son) son = onceki;  boyut--;
7     } else {
8         gorulen.basaEkle(v);
9         onceki = gecici;
10    }
11    gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- bas = **düğüm(3)**
- gorulen = **7→5→3**
- onceki = **düğüm(7)**
- gecici = **düğüm(5)**

Kod İzleme: Tekrarları Kaldırma (devam)



`v = 5` .

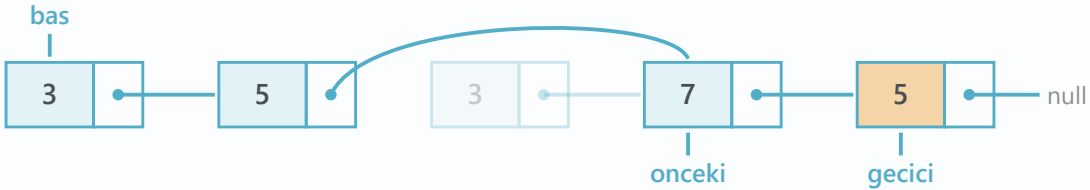
Adım 7 / 11

```
1  while (gecici != null) {
▶      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = 7→5→3`
- `onceki = düğüm(7)`
- `gecici = düğüm(5)` `v = 5`

Kod İzleme: Tekrarları Kaldırma (devam)



`gorulen.iceriyor(5)` → **true**: 5 daha önce görüldü, bu düğüm tekrar.

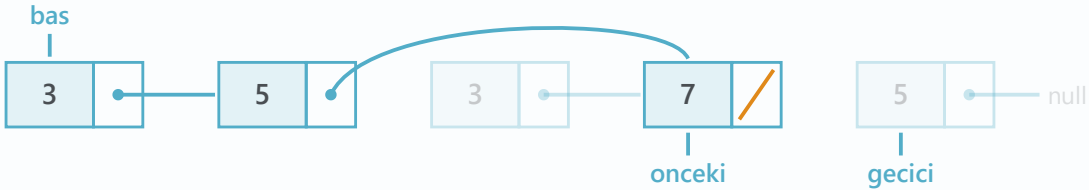
Adım 8 / 11

```
1  while (gecici != null) {
2      int v = gecici.veri;
▶      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas` = **düğüm(3)**
- `gorulen` = **7→5→3**
- `onceki` = **düğüm(7)**
- `gecici` = **düğüm(5)**
- `v` = **5**

Kod İzleme: Tekrarları Kaldırma (devam)



`onceki.sonraki = gecici.sonraki` : 7 → null; tekrar eden 5 atlanır. `onceki` yerinde kalır.

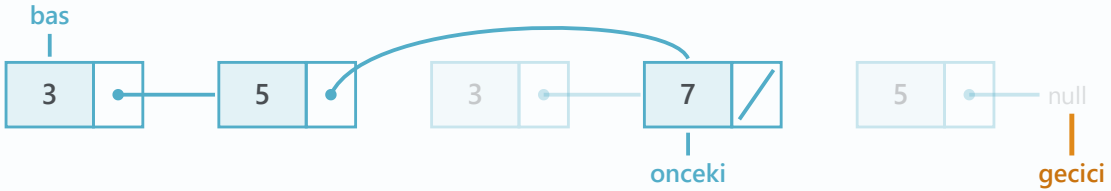
Adım 9 / 11

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
▶          onceki.sonraki =
▶              gecici.sonraki;   // atla
▶          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
11     gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = 7→5→3`
- `onceki = düğüm(7)`
- `gecici = düğüm(5)` `v = 5`

Kod İzleme: Tekrarları Kaldırma (devam)



`gecici = gecici.sonraki` → null.

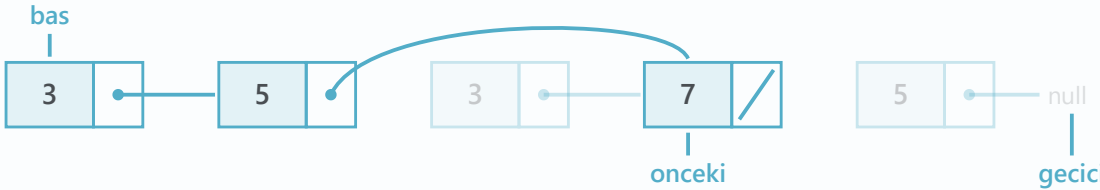
Adım 10 / 11

```
1  while (gecici != null) {
2      int v = gecici.veri;
3      if (gorulen.iceriyor(v)) {
4          onceki.sonraki =
5              gecici.sonraki;   // atla
6          if (gecici == son) son = onceki;  boyut--;
7      } else {
8          gorulen.basaEkle(v);
9          onceki = gecici;
10     }
▶      gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- `bas = düğüm(3)`
- `gorulen = 7→5→3`
- `onceki = düğüm(7)`
- `gecici = null`
- `v = 5`

Kod İzleme: Tekrarları Kaldırma (devam)



gecici == null → döngü biter. Sonuç: 3 → 5 → 7 → null. Her düğümde gorulen'de arama: **O(n²)**; hash kümesiyle O(n).

Adım 11 / 11

```
▶ while (gecici != null) {
2     int v = gecici.veri;
3     if (gorulen.iceriyor(v)) {
4         onceki.sonraki =
5             gecici.sonraki;   // atla
6         if (gecici == son) son = onceki;  boyut--;
7     } else {
8         gorulen.basaEkle(v);
9         onceki = gecici;
10    }
11    gecici = gecici.sonraki;
12 }
```

DEĞİŞKENLER

- bas = **düğüm(3)**
- gorulen = **7→5→3**
- onceki = **düğüm(7)**
- gecici = **null**

İki Sıralı Listeyi Birleştirme

```
TekYonluBagliListe birlestir(
        TekYonluBagliListe l1,
        TekYonluBagliListe l2) {
    TekYonluBagliListe sonuc =
        new TekYonluBagliListe();
    Dugum g1 = l1.bas, g2 = l2.bas;
    while (g1 != null && g2 != null) {
        if (g1.veri <= g2.veri) {
            sonuc.sonaEkle(g1.veri);
            g1 = g1.sonraki;
        } else {
            sonuc.sonaEkle(g2.veri);
            g2 = g2.sonraki;
        }
    }
    // kalan öğeleri ekle
    while (g1 != null) {
        sonuc.sonaEkle(g1.veri);
        g1 = g1.sonraki;
    }
    while (g2 != null) {
        sonuc.sonaEkle(g2.veri);
        g2 = g2.sonraki;
    }
    return sonuc;
}
```

1 → 4 → 7 ve 2 → 3 → 8

⇒ **1 → 2 → 3 → 4 → 7 → 8**

`sonaEkle` O(1) ise (son işaretçisi) toplam **O(n + m)** (n, m: liste uzunlukları); her ekleme sona gezinirse $O((n + m)^2)$.

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



sonuc

`sonuc` boş yeni liste. `g1` ve `g2` iki sıralı listenin başında.

Adım 1 / 17

```
▶ Dugum g1 = l1.bas, g2 = l2.bas;
2 while (g1 != null && g2 != null) {
3     if (g1.veri <= g2.veri) {
4         sonuc.sonaEkle(g1.veri);
5         g1 = g1.sonraki;
6     } else {
7         sonuc.sonaEkle(g2.veri);
8         g2 = g2.sonraki;
9     }
10 }
```

DEĞİŞKENLER

g1 = **düğüm(1)** g2 = **düğüm(2)**

sonuc = **boş**

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



sonuc

İki liste de bitmedi → karşılaştır.

Adım 2 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
▶  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

g1 = **düğüm(1)**

g2 = **düğüm(2)**

sonuc = **boş**

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



sonuc

1 ≤ 2 → **doğru**: küçük olan l1'de.

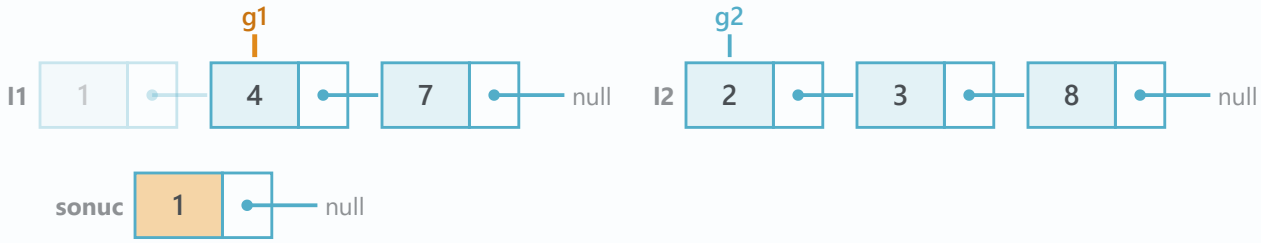
Adım 3 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
▶      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

g1 = **düğüm(1)** g2 = **düğüm(2)** sonuc = **boş**

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



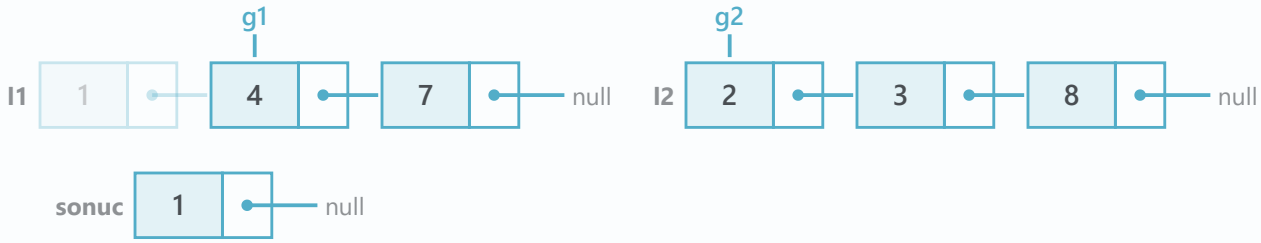
1 sonuca eklenir, `g1` ilerler.


```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
▶          sonuc.sonaEkle(g1.veri);
▶          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

`g1 = düğüm(4)` `g2 = düğüm(2)` `sonuc = 1`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



İki liste de bitmedi → karşılaştır.

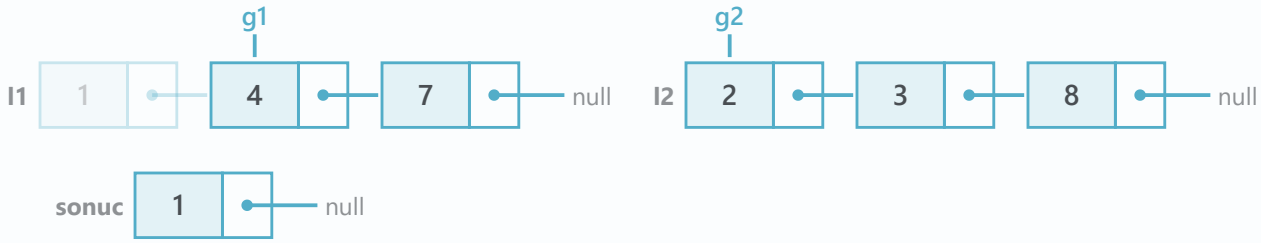
Adım 5 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
▶  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

`g1 = düğüm(4)` `g2 = düğüm(2)` `sonuc = 1`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



4 ≤ 2 → **yanlış**: küçük olan l2'de.

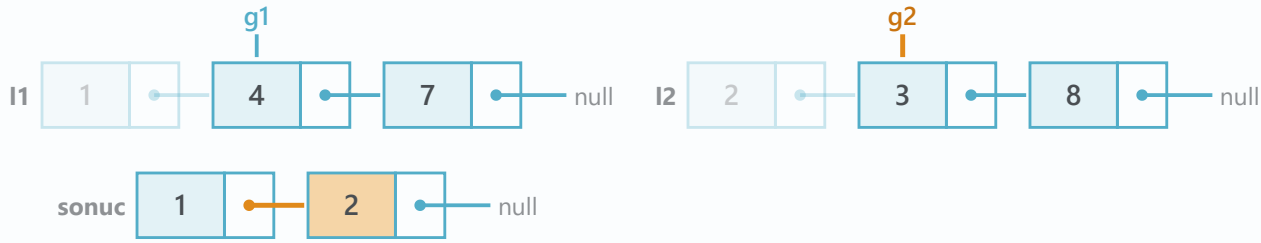
Adım 6 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
▶      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- g1 = **düğüm(4)**
- g2 = **düğüm(2)**
- sonuc = **1**

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



2 sonuca eklenir, `g2` ilerler.

Adım 7 / 17

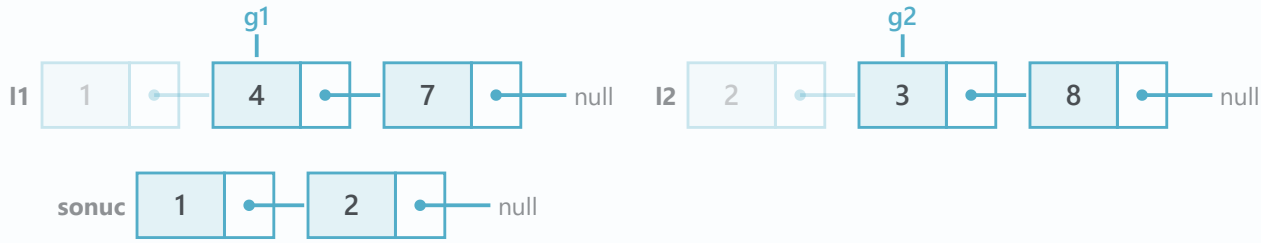
```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
▶          sonuc.sonaEkle(g2.veri);
▶          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

`g1 = düğüm(4)` `g2 = düğüm(3)`

`sonuc = 1→2`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



İki liste de bitmedi → karşılaştır.

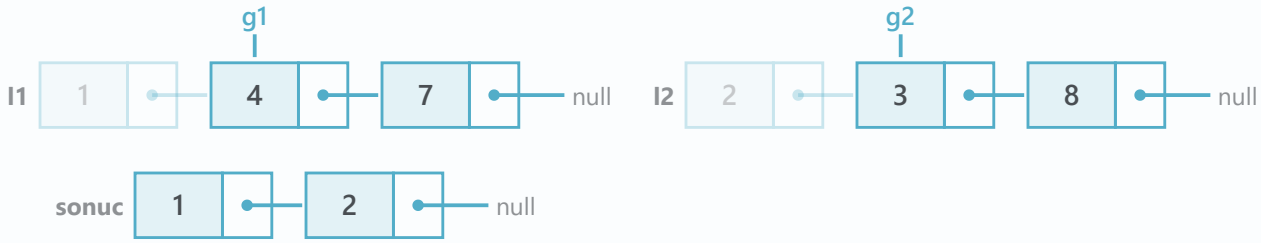
Adım 8 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
▶  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- `g1 = düğüm(4)`
- `g2 = düğüm(3)`
- `sonuc = 1→2`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



4 ≤ 3 → **yanlış**: küçük olan l2'de.

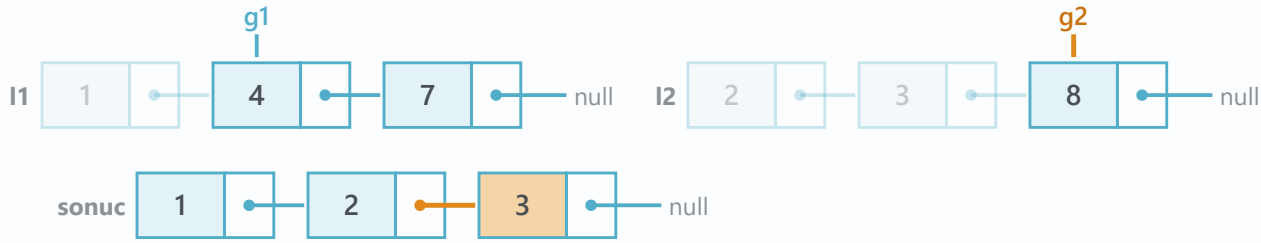
Adım 9 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
▶      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- g1 = **düğüm(4)**
- g2 = **düğüm(3)**
- sonuc = **1→2**

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



3 sonuca eklenir, `g2` ilerler.

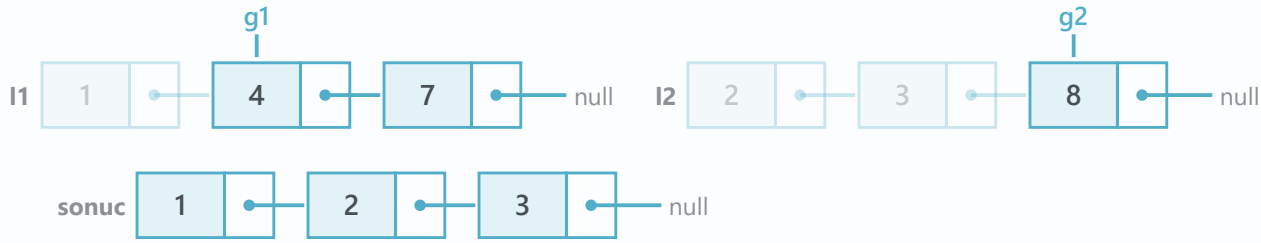
Adım 10 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
▶          sonuc.sonaEkle(g2.veri);
▶          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- `g1 = düğüm(4)`
- `g2 = düğüm(8)`
- `sonuc = 1→2→3`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



İki liste de bitmedi → karşılaştır.

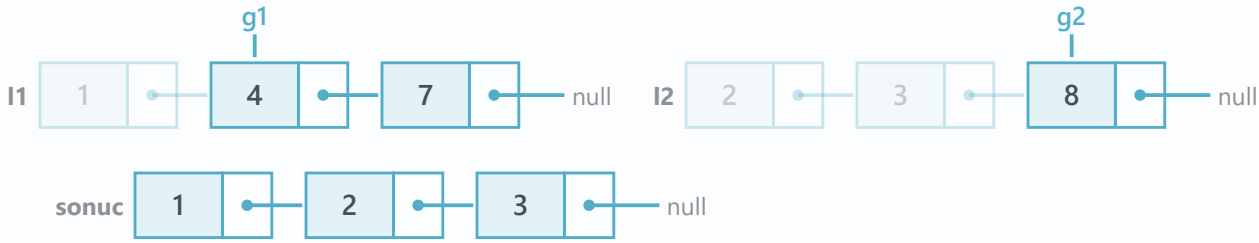
Adım 11 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
▶  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- `g1 = düğüm(4)`
- `g2 = düğüm(8)`
- `sonuc = 1→2→3`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



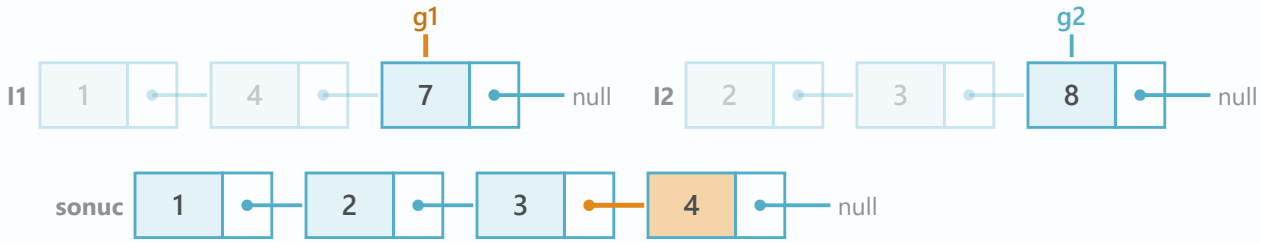
4 ≤ 8 → **doğru**: küçük olan l1'de.


```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
▶      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- g1 = **düğüm(4)**
- g2 = **düğüm(8)**
- sonuc = **1→2→3**

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



4 sonuca eklenir, `g1` ilerler.

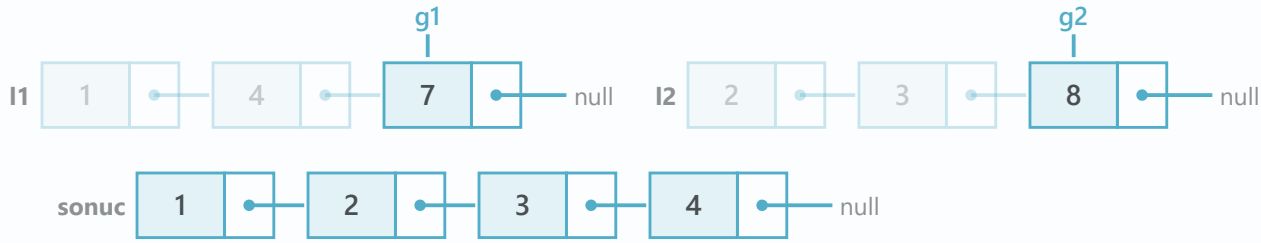
Adım 13 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
▶          sonuc.sonaEkle(g1.veri);
▶          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- `g1 = düğüm(7)`
- `g2 = düğüm(8)`
- `sonuc = 1→2→3→4`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



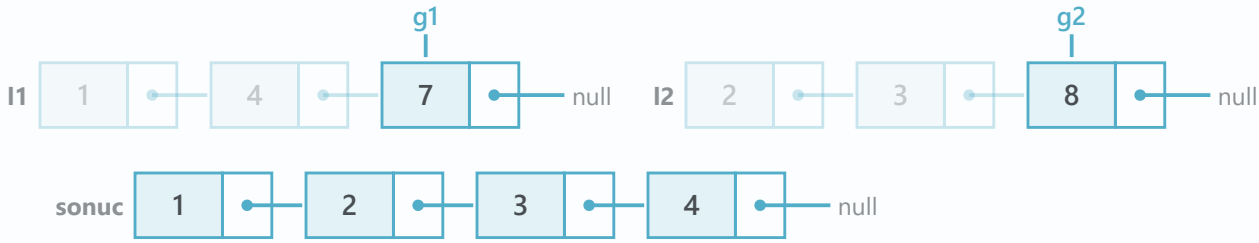
İki liste de bitmedi → karşılaştır.


```
1  Dugum g1 = l1.bas, g2 = l2.bas;
▶  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- `g1 = düğüm(7)`
- `g2 = düğüm(8)`
- `sonuc = 1→2→3→4`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



7 ≤ 8 → **doğru**: küçük olan l1'de.

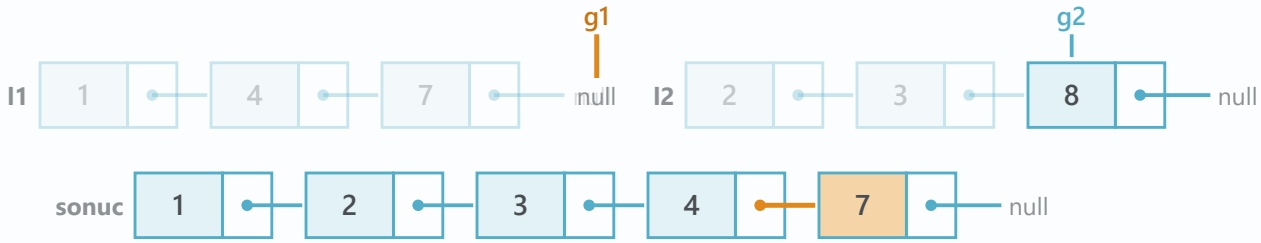
Adım 15 / 17

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
▶      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

- g1 = **düğüm(7)**
- g2 = **düğüm(8)**
- sonuc = **1→2→3→4**

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



7 sonuca eklenir, `g1` ilerler.

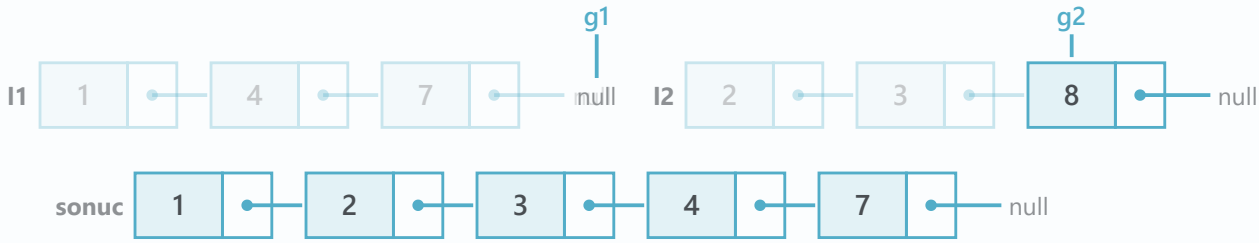

```
1  Dugum g1 = l1.bas, g2 = l2.bas;
2  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
▶          sonuc.sonaEkle(g1.veri);
▶          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

`g1 = null` `g2 = düğüm(8)`

`sonuc = 1→2→3→4→7`

Kod İzleme: `birlestir` (1, 4, 7 + 2, 3, 8)



`g1 == null` : l1 bitti → ana döngü biter. l2'de kalanlar (8) zaten sıralı.

Adım 17 / 17

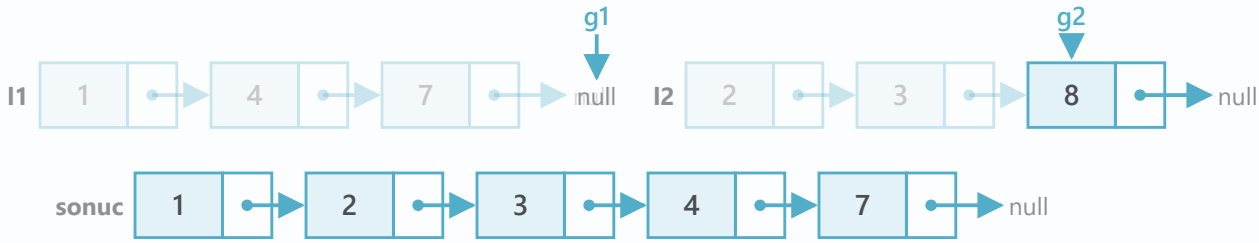
```
1  Dugum g1 = l1.bas, g2 = l2.bas;
▶  while (g1 != null && g2 != null) {
3      if (g1.veri <= g2.veri) {
4          sonuc.sonaEkle(g1.veri);
5          g1 = g1.sonraki;
6      } else {
7          sonuc.sonaEkle(g2.veri);
8          g2 = g2.sonraki;
9      }
10 }
```

DEĞİŞKENLER

`g1 = null` `g2 = düğüm(8)`

`sonuc = 1→2→3→4→7`

Kod İzleme: `birlestir` (devam)



Kalanlar: `g1 == null` → ilk döngü hiç çalışmaz.

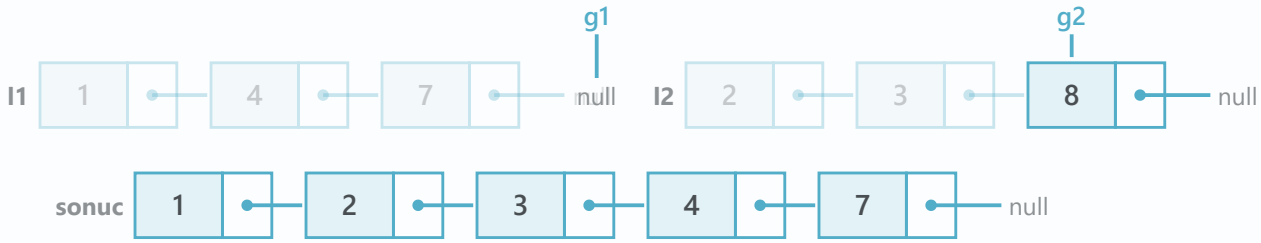
Adım 1 / 5

```
1  // kalan öğeleri ekle
▶ while (g1 != null) {
3      sonuc.sonaEkle(g1.veri);
4      g1 = g1.sonraki;
5  }
6  while (g2 != null) {
7      sonuc.sonaEkle(g2.veri);
8      g2 = g2.sonraki;
9  }
10 return sonuc;
```

DEĞİŞKENLER

`g1 = null` `g2 = düğüm(8)` `sonuc = 1→2→3→4→7`

Kod İzleme: `birlestir` (devam)



g2 = düğüm(8) null değil.

Adım 2 / 5

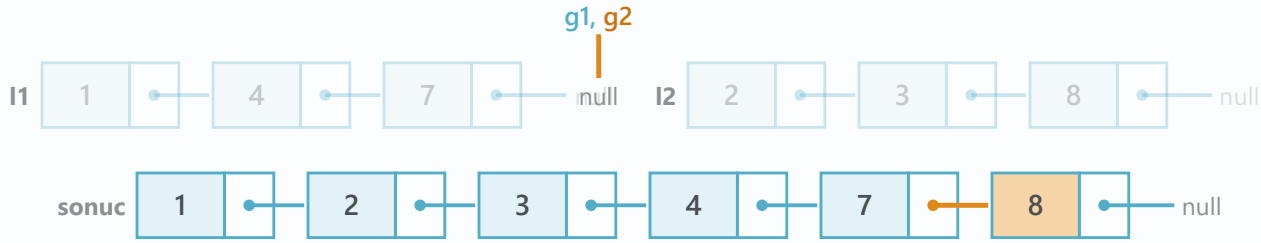
```
1  // kalan öğeleri ekle
2  while (g1 != null) {
3      sonuc.sonaEkle(g1.veri);
4      g1 = g1.sonraki;
5  }
▶  while (g2 != null) {
7      sonuc.sonaEkle(g2.veri);
8      g2 = g2.sonraki;
9  }
10 return sonuc;
```

DEĞİŞKENLER

`g1 = null` `g2 = düğüm(8)`

`sonuc = 1→2→3→4→7`

Kod İzleme: `birlestir` (devam)



8 sonuca eklenir, `g2` ilerler.

Adım 3 / 5

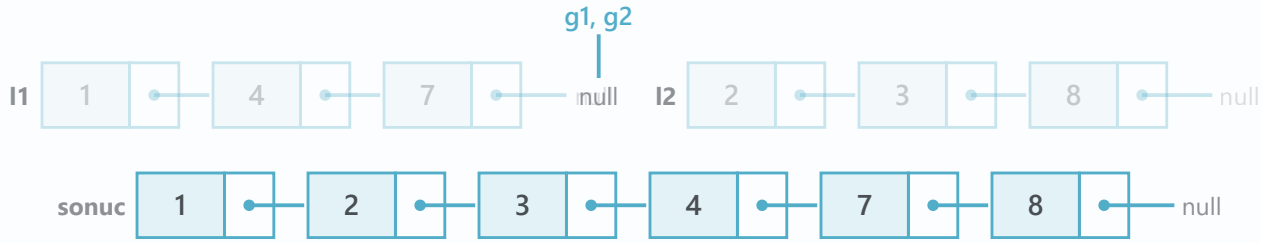
```
1  // kalan öğeleri ekle
2  while (g1 != null) {
3      sonuc.sonaEkle(g1.veri);
4      g1 = g1.sonraki;
5  }
6  while (g2 != null) {
▶      sonuc.sonaEkle(g2.veri);
▶      g2 = g2.sonraki;
9  }
10 return sonuc;
```

DEĞİŞKENLER

`g1 = null` `g2 = null`

`sonuc = 1→2→3→4→7→8`

Kod İzleme: `birlestir` (devam)



g2 == null → biter.

Adım 4 / 5

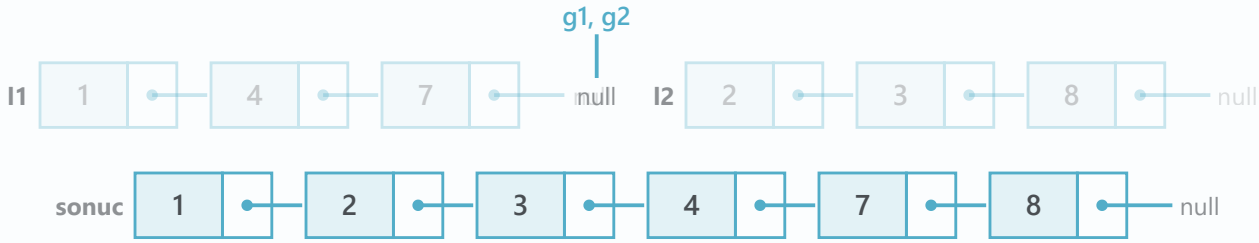
```
1  // kalan öğeleri ekle
2  while (g1 != null) {
3      sonuc.sonaEkle(g1.veri);
4      g1 = g1.sonraki;
5  }
▶  while (g2 != null) {
7      sonuc.sonaEkle(g2.veri);
8      g2 = g2.sonraki;
9  }
10 return sonuc;
```

DEĞİŞKENLER

`g1 = null` `g2 = null`

`sonuc = 1→2→3→4→7→8`

Kod İzleme: `birlestir` (devam)



`return sonuc` : 1 → 2 → 3 → 4 → 7 → 8. sonaEkle O(1) ise (son işaretçisi) toplam **O(n + m)**.

Adım 5 / 5

```
1 // kalan öğeleri ekle
2 while (g1 != null) {
3     sonuc.sonaEkle(g1.veri);
4     g1 = g1.sonraki;
5 }
6 while (g2 != null) {
7     sonuc.sonaEkle(g2.veri);
8     g2 = g2.sonraki;
9 }
▶ return sonuc;
```

DEĞİŞKENLER

`g1 = null` `g2 = null`

`sonuc = 1→2→3→4→7→8`

Tek Yönlü Bağlı Listeyi Sıralama

```
void sirala() {
    Dugum suanki = bas;
    while (suanki != null) {
        Dugum diger = suanki.sonraki;
        while (diger != null) {
            if (suanki.veri > diger.veri) {
                int gecici = suanki.veri;
                suanki.veri = diger.veri;
                diger.veri = gecici;
            }
            diger = diger.sonraki;
        }
        suanki = suanki.sonraki;
    }
}
```

Değer takası

Düğümler yerinde kalır, yalnızca `veri` alanları değiştirilir.

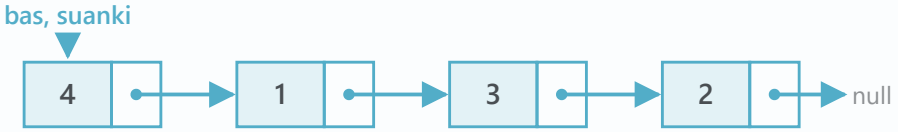
$O(n^2)$

Her düğüm, kendinden sonraki tüm düğümlerle karşılaştırılır: $n(n-1)/2$.

Daha iyisi

Birleştirme sıralaması (merge sort) bağlı listede O(n log n) çalışır ve ek dizi gerektirmez.

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



Liste 4 → 1 → 3 → 2. `suanki` baştan başlar; düğümler yerinde kalacak, yalnızca `veri` alanları değişecek.

Adım 1 / 15

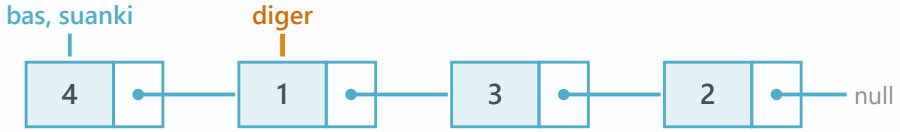
```
1  void sirala() {
▶       Dugum suanki = bas;
3       while (suanki != null) {
4           Dugum diger = suanki.sonraki;
5           while (diger != null) {
6               if (suanki.veri > diger.veri) {
7                   int gecici = suanki.veri;
8                   suanki.veri = diger.veri;
9                   diger.veri = gecici;
10              }
11              diger = diger.sonraki;
12          }
13          suanki = suanki.sonraki;
14      }
15 }
```

DEĞİŞKENLER

`bas` = **düğüm(4)**

`suanki` = **düğüm(4)**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



`diger = suanki.sonraki` → düğüm(1). suanki (4) kendinden sonrakilerle karşılaştırılacak.

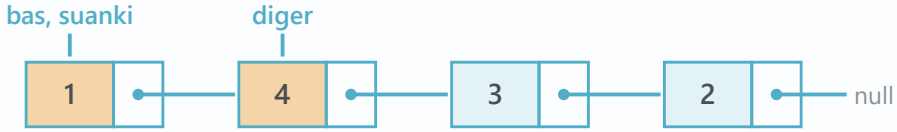
Adım 2 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
▶          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(4)**
- `suanki` = **düğüm(4)**
- `diger` = **düğüm(1)**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



4 > 1 → değerler takas edilir (`gecici` = 4). Liste: 1 → 4 → 3 → 2.

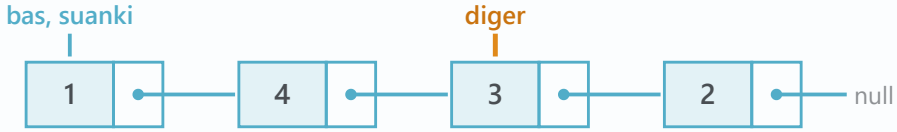
Adım 3 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
▶              if (suanki.veri > diger.veri) {
▶                  int gecici = suanki.veri;
▶                  suanki.veri = diger.veri;
▶                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(1)**
- `diger` = **düğüm(4)**
- `gecici` = **4**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



1 > 3? Hayır → takas yok; `diger` ilerler.

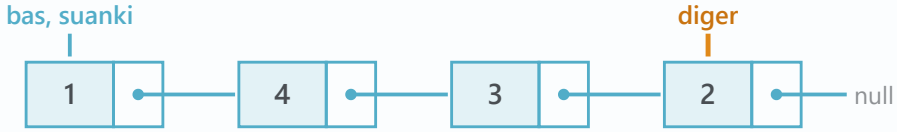
Adım 4 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
▶              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(1)**
- `diger` = **düğüm(3)**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



1 > 2? Hayır → takas yok; `diger` ilerler.

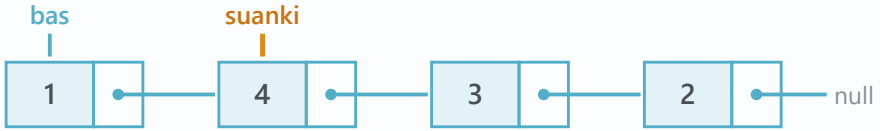
Adım 5 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
▶              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(1)**
- `diger` = **düğüm(2)**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



İç döngü bitti: en küçük kalan değer suanki'nin hücresine yerleşti. `suanki` ilerler → düğüm(4).

Adım 6 / 15

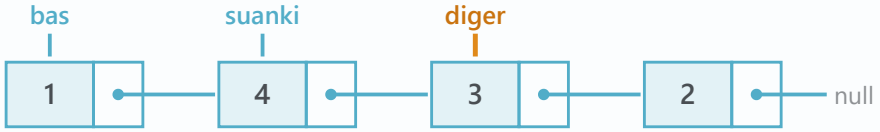
```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
▶          suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

`bas = düğüm(1)`

`suanki = düğüm(4)`

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



`diger = suanki.sonraki` → düğüm(3). suanki (4) kendinden sonrakilerle karşılaştırılacak.

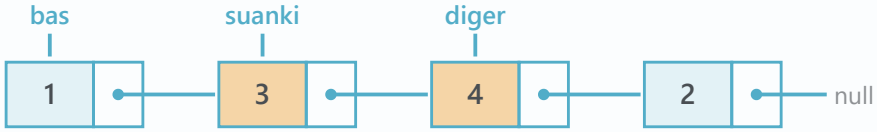
Adım 7 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
▶          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(4)**
- `diger` = **düğüm(3)**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



4 > 3 → değerler takas edilir (`gecici` = 4). Liste: 1 → 3 → 4 → 2.

Adım 8 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
▶              if (suanki.veri > diger.veri) {
▶                  int gecici = suanki.veri;
▶                  suanki.veri = diger.veri;
▶                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(3)**
- `diger` = **düğüm(4)**
- `gecici` = **4**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



3 > 2 → değerler takas edilir (`gecici` = 3). Liste: 1 → 2 → 4 → 3.

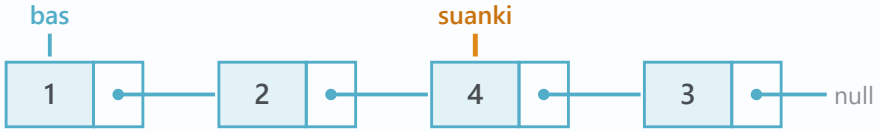
Adım 9 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
▶              if (suanki.veri > diger.veri) {
▶                  int gecici = suanki.veri;
▶                  suanki.veri = diger.veri;
▶                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(2)**
- `diger` = **düğüm(3)**
- `gecici` = **3**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



İç döngü bitti: en küçük kalan değer suanki'nin hücresine yerleşti. `suanki` ilerler → düğüm(4).

Adım 10 / 15

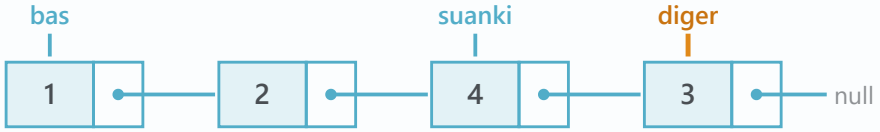
```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
▶          suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

`bas` = **düğüm(1)**

`suanki` = **düğüm(4)**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



`diger = suanki.sonraki` → düğüm(3). suanki (4) kendinden sonrakilerle karşılaştırılacak.

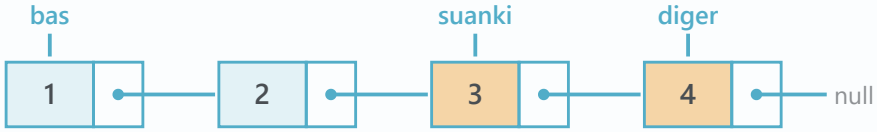
Adım 11 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
▶          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(4)**
- `diger` = **düğüm(3)**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



4 > 3 → değerler takas edilir (`gecici` = 4). Liste: 1 → 2 → 3 → 4.

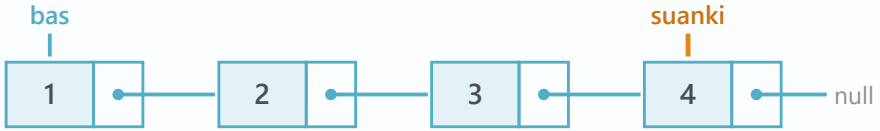
Adım 12 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
▶              if (suanki.veri > diger.veri) {
▶                  int gecici = suanki.veri;
▶                  suanki.veri = diger.veri;
▶                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(3)**
- `diger` = **düğüm(4)**
- `gecici` = **4**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



İç döngü bitti: en küçük kalan değer suanki'nin hücresine yerleşti. `suanki` ilerler → düğüm(4).

Adım 13 / 15

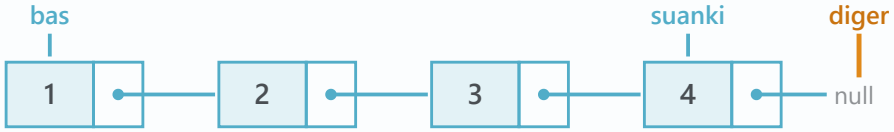
```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
▶          suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

`bas = düğüm(1)`

`suanki = düğüm(4)`

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



`diger = suanki.sonraki` = null: son düğümle karşılaştırılacak kimse yok.

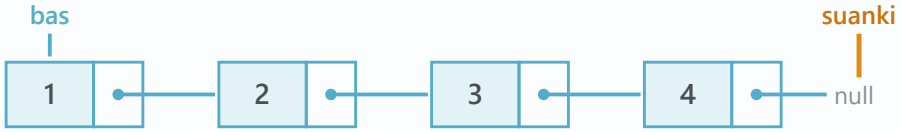
Adım 14 / 15

```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
▶          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
13         suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `suanki` = **düğüm(4)**
- `diger` = **null**

Kod İzleme: `sirala()` (4 → 1 → 3 → 2)



`suanki` = null → dış döngü biter. Sonuç: 1 → 2 → 3 → 4; 6 karşılaştırma = n(n−1)/2, 4 takas.

Adım 15 / 15

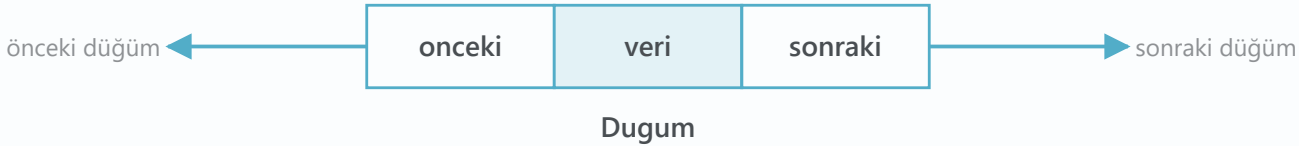
```
1  void sirala() {
2      Dugum suanki = bas;
3      while (suanki != null) {
4          Dugum diger = suanki.sonraki;
5          while (diger != null) {
6              if (suanki.veri > diger.veri) {
7                  int gecici = suanki.veri;
8                  suanki.veri = diger.veri;
9                  diger.veri = gecici;
10             }
11             diger = diger.sonraki;
12         }
▶          suanki = suanki.sonraki;
14     }
15 }
```

DEĞİŞKENLER

`bas = düğüm(1)` `suanki = null`

Çift Yönlü Bağlı Liste (Doubly Linked List)

- Düğümlerin birbirine **iki yönde** bağlandığı veri yapısıdır.
- Her düğüm iki işaretçi tutar: kendinden **önceki** ve kendinden **sonraki** düğüm.
- Liste her iki yönde gezilebilir; genellikle hem baş (`bas`) hem son (`son`) tutulur.



Dugum

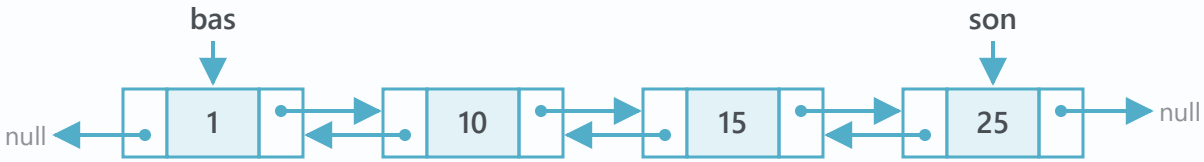
Kazanç

Geri gezinme; bilinen bir düğümü ve son düğümü O(1)'de silme.

Bedel

Düğüm başına ikinci referans; her ekleme/silmede iki yönün de güncellenmesi.

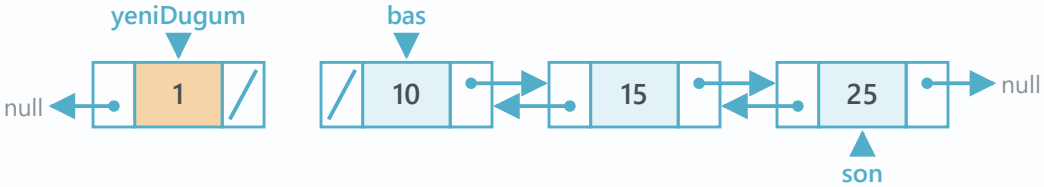
Çift Yönlü Liste: Düğüm ve Yapı




```
class Dugum {
    int veri;
    Dugum onceki;
    Dugum sonraki;
    Dugum(int veri) {
        this.veri = veri;
        this.onceki = null;
        this.sonraki = null;
    }
}
```

- Öğelere hem ileri hem geri yönde erişilir.
- Düğüm eklenince işaretçiler **her iki yönde** güncellenmelidir.
- Düğüm çıkarılınca hem önceki hem sonraki düğümün işaretçileri güncellenmelidir.
- Arama ileri ya da geri yönde gezinerek yapılabilir.
- Listede `Dugum bas, son;` alanları tutulur.

Çift Yönlü Listede Başa Ekleme — O(1)



`basaEkle(1)` : yeni düğümün `onceki` ve `sonraki` alanları null (/).

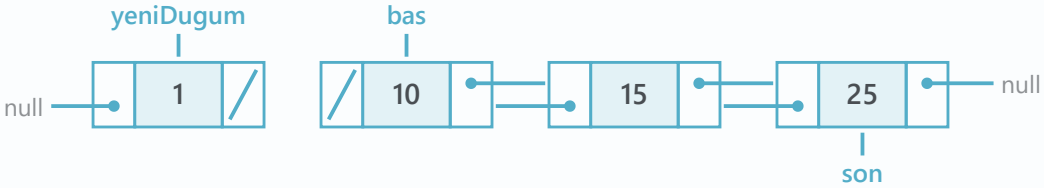
Adım 1 / 5

```
1  public void basaEkle(int veri) {
▶      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.sonraki = bas;
8          bas.onceki = yeniDugum;
9          bas = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(25)**
- `veri` = **1**
- `yeniDugum` = **düğüm(1)**

Çift Yönlü Listede Başa Ekleme — O(1)



`bas` null değil → `else` dalı.

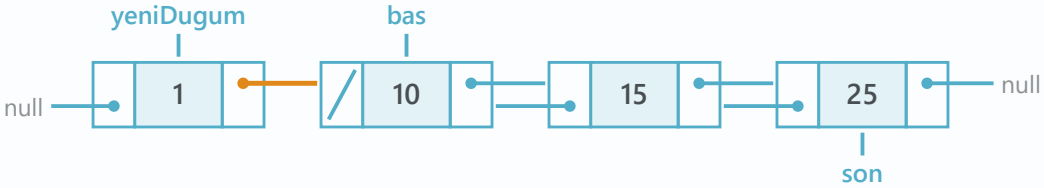
Adım 2 / 5

```
1  public void basaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
▶      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.sonraki = bas;
8          bas.onceki = yeniDugum;
9          bas = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(25)**
- `veri` = **1**
- `yeniDugum` = **düğüm(1)**

Çift Yönlü Listede Başa Ekleme — O(1)



`yeniDugum.sonraki = bas` : 1 → 10 ileri bağı.

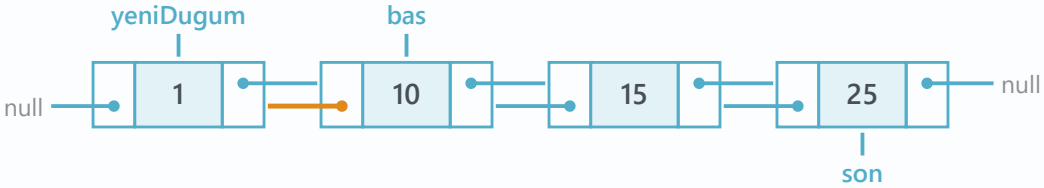
Adım 3 / 5

```
1  public void basaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
▶          yeniDugum.sonraki = bas;
8          bas.onceki = yeniDugum;
9          bas = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(25)**
- `veri` = **1**
- `yeniDugum` = **düğüm(1)**

Çift Yönlü Listede Başa Ekleme — O(1)



`bas.onceki = yeniDugum` : 10 → 1 geri bağı (10'un null'u gitti).

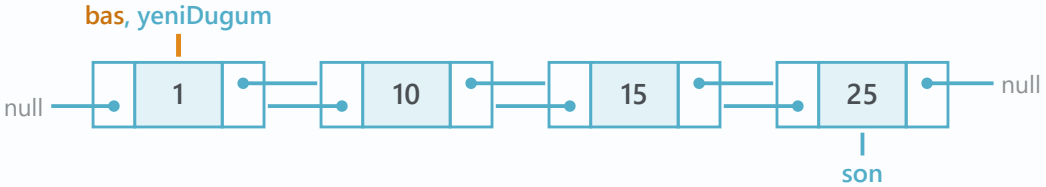
Adım 4 / 5

```
1  public void basaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.sonraki = bas;
▶          bas.onceki = yeniDugum;
9          bas = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(10)**
- `son` = **düğüm(25)**
- `veri` = **1**
- `yeniDugum` = **düğüm(1)**

Çift Yönlü Listede Başa Ekleme — O(1)



`bas = yeniDugum` : 1 ⇄ 10 ⇄ 15 ⇄ 25. **O(1)**. Liste boş olsaydı bas ve son birlikte yeni düğüm olurdu.

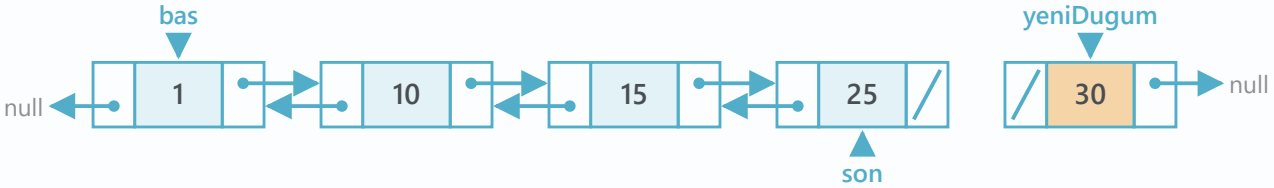
Adım 5 / 5

```
1  public void basaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.sonraki = bas;
8          bas.onceki = yeniDugum;
▶          bas = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas = düğüm(1)`
- `son = düğüm(25)`
- `veri = 1`
- `yeniDugum = düğüm(1)`

Çift Yönlü Listede Sona Ekleme — O(1)



`sonaEkle(30)` : yeni düğüm oluşturulur.

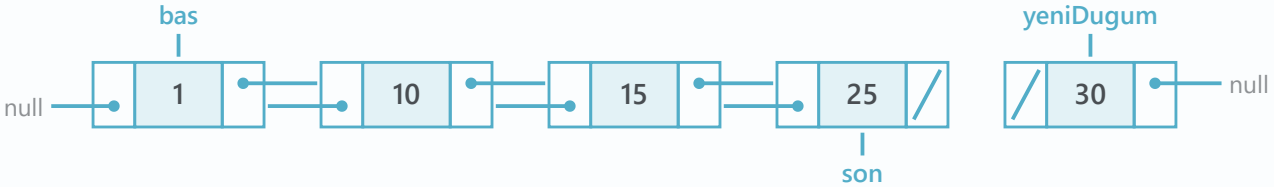
Adım 1 / 5

```
1  public void sonaEkle(int veri) {
▶      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.onceki = son;
8          son.sonraki = yeniDugum;
9          son = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **30**
- `yeniDugum` = **düğüm(30)**

Çift Yönlü Listede Sona Ekleme — O(1)



Liste boş değil → `else`. Son düğüm `son` ile doğrudan biliniyor.

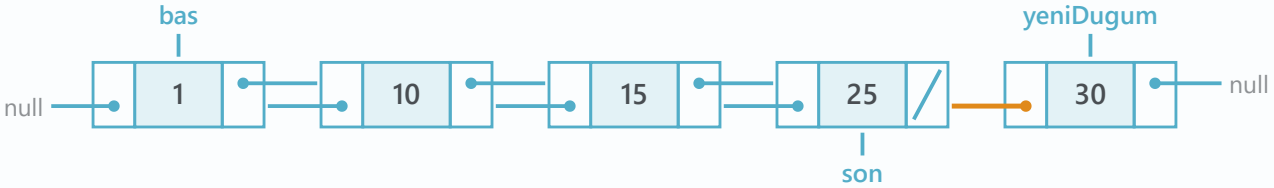
Adım 2 / 5

```
1  public void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
▶      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.onceki = son;
8          son.sonraki = yeniDugum;
9          son = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **30**
- `yeniDugum` = **düğüm(30)**

Çift Yönlü Listede Sona Ekleme — O(1)



`yeniDugum.onceki = son` : 30 → 25 geri bağı.

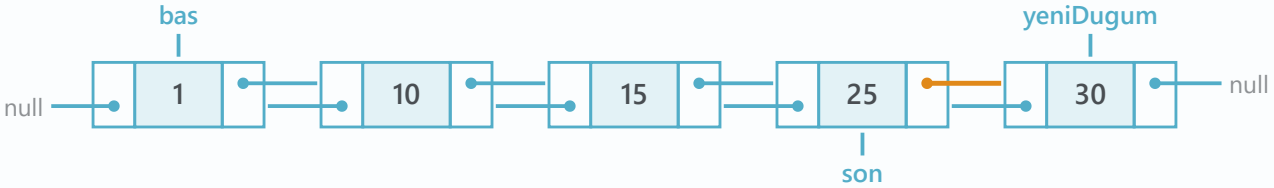
Adım 3 / 5

```
1  public void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
▶          yeniDugum.onceki = son;
8          son.sonraki = yeniDugum;
9          son = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **30**
- `yeniDugum` = **düğüm(30)**

Çift Yönlü Listede Sona Ekleme — O(1)



`son.sonraki = yeniDugum` : 25 → 30 ileri bağı.

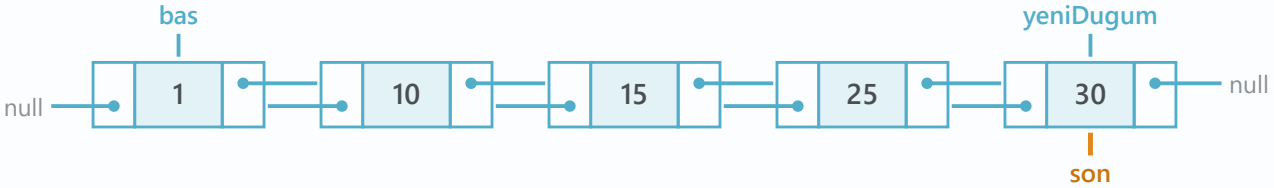
Adım 4 / 5

```
1  public void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.onceki = son;
▶          son.sonraki = yeniDugum;
9          son = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **30**
- `yeniDugum` = **düğüm(30)**

Çift Yönlü Listede Sona Ekleme — O(1)



`son = yeniDugum` : 1 ⇄ 10 ⇄ 15 ⇄ 25 ⇄ 30. Gezinme yok: **O(1)**.

Adım 5 / 5

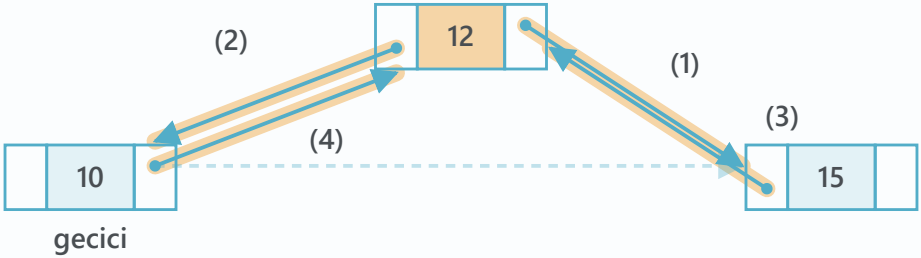
```
1  public void sonaEkle(int veri) {
2      Dugum yeniDugum = new Dugum(veri);
3      if (bas == null) {
4          bas = yeniDugum;
5          son = yeniDugum;
6      } else {
7          yeniDugum.onceki = son;
8          son.sonraki = yeniDugum;
▶          son = yeniDugum;
10     }
11 }
```

DEĞİŞKENLER

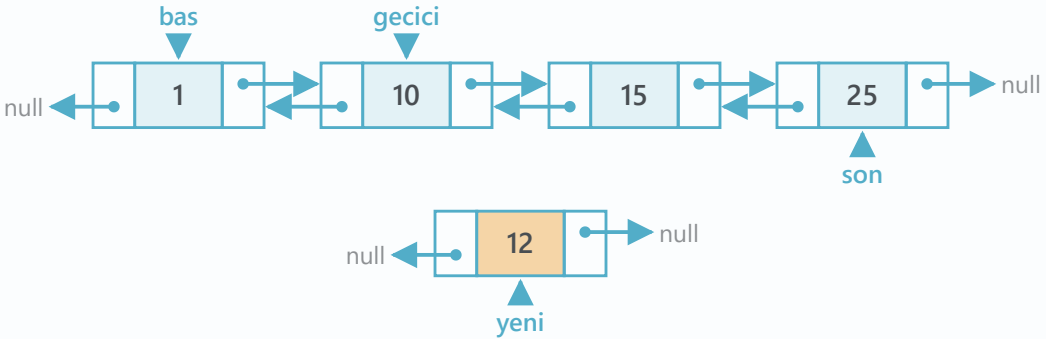
- `bas = düğüm(1)`
- `son = düğüm(30)`
- `veri = 30`
- `yeniDugum = düğüm(30)`

Çift Yönlü Listede Konuma Ekleme

```
void konumaEkle(int veri, int indis) {   // indis 0'dan başlar
    if (indis == 0) { basaEkle(veri); return; }
    Dugum gecici = bas;
    for (int i = 0; i < indis - 1 && gecici != null; i++)
        gecici = gecici.sonraki;
    if (gecici == null) { System.out.println("Geçersiz konum."); return; }
    if (gecici == son) { sonaEkle(veri); return; }   // son da güncellenir
    Dugum yeni = new Dugum(veri);
    yeni.sonraki = gecici.sonraki;      // (1)
    yeni.onceki = gecici;               // (2)
    gecici.sonraki.onceki = yeni;       // (3)
    gecici.sonraki = yeni;              // (4)
}
```



Kod İzleme: `konumaEkle(12, 2)` — bağlama kısmı



`konumaEkle(12, 2)` : gecici, tek yönlüdeki gibi indis − 1 = 1'deki düğüme (10) yürüdü. Yeni düğüm oluşturulur.

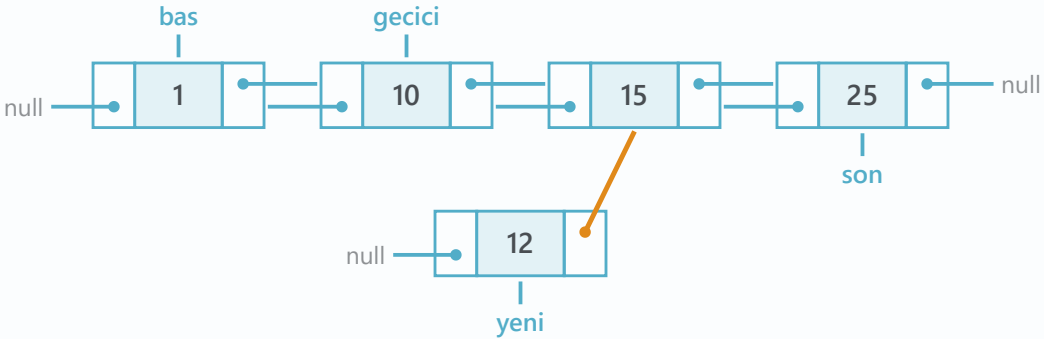
Adım 1 / 6

```
▶ Dugum yeni = new Dugum(veri);
2 yeni.sonraki = gecici.sonraki;      // (1)
3 yeni.onceki = gecici;               // (2)
4 gecici.sonraki.onceki = yeni;       // (3)
5 gecici.sonraki = yeni;              // (4)
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **12**
- `indis` = **2**
- `gecici` = **düğüm(10)**
- `yeni` = **düğüm(12)**

Kod İzleme: `konumaEkle(12, 2)` — bağlama kısmı



(1) `yeni.sonraki = gecici.sonraki` : 12 → 15. Liste henüz değişmedi.

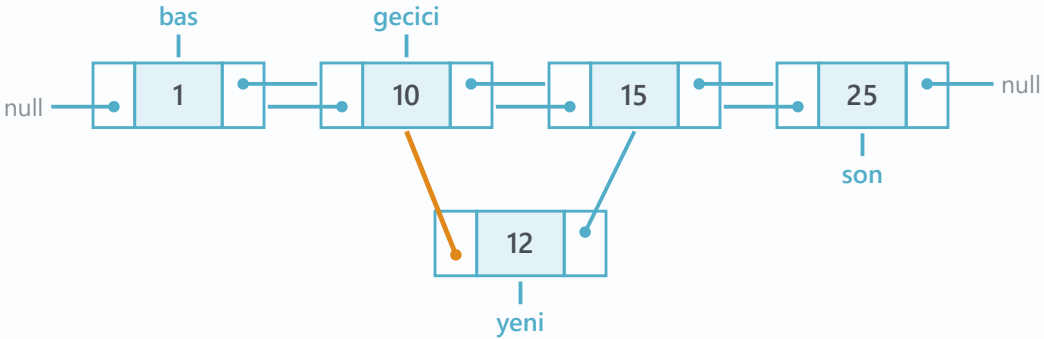
Adım 2 / 6

```
1 Dugum yeni = new Dugum(veri);
▶ yeni.sonraki = gecici.sonraki;      // (1)
3 yeni.onceki = gecici;               // (2)
4 gecici.sonraki.onceki = yeni;       // (3)
5 gecici.sonraki = yeni;              // (4)
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **12**
- `indis` = **2**
- `gecici` = **düğüm(10)**
- `yeni` = **düğüm(12)**

Kod İzleme: `konumaEkle(12, 2)` — bağlama kısmı



(2) `yeni.onceki = gecici` : 12 → 10 geri bağı.

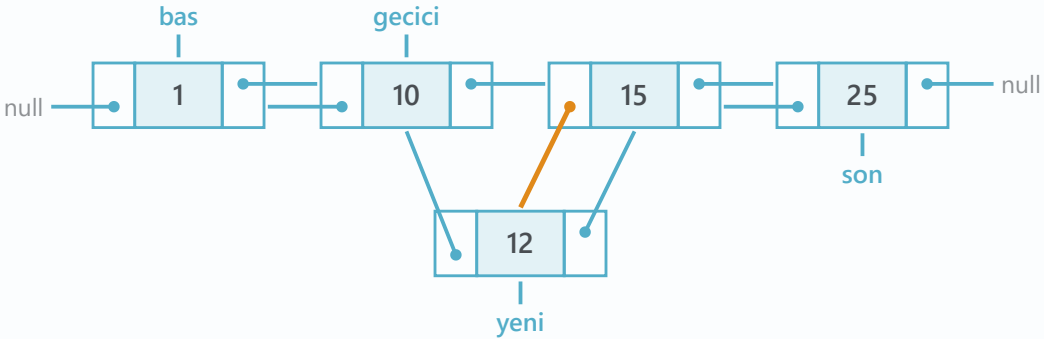
Adım 3 / 6

```
1 Dugum yeni = new Dugum(veri);
2 yeni.sonraki = gecici.sonraki;      // (1)
▶ yeni.onceki = gecici;               // (2)
4 gecici.sonraki.onceki = yeni;       // (3)
5 gecici.sonraki = yeni;              // (4)
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **12**
- `indis` = **2**
- `gecici` = **düğüm(10)**
- `yeni` = **düğüm(12)**

Kod İzleme: `konumaEkle(12, 2)` — bağlama kısmı



(3) `gecici.sonraki.onceki = yeni` : 15'in geri bağı artık 12'yi gösterir. gecici.sonraki hâlâ 15 olduğu için (4)'ten önce yapılmalı.

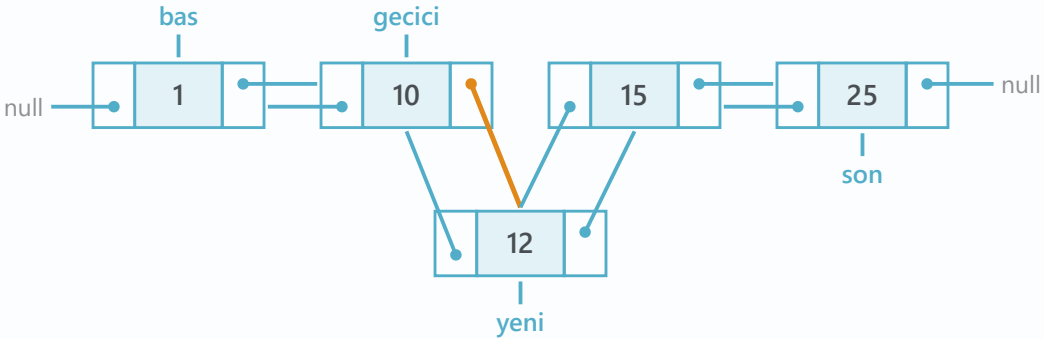
Adım 4 / 6

```
1 Dugum yeni = new Dugum(veri);
2 yeni.sonraki = gecici.sonraki;      // (1)
3 yeni.onceki = gecici;               // (2)
▶ gecici.sonraki.onceki = yeni;       // (3)
5 gecici.sonraki = yeni;              // (4)
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **12**
- `indis` = **2**
- `gecici` = **düğüm(10)**
- `yeni` = **düğüm(12)**

Kod İzleme: `konumaEkle(12, 2)` — bağlama kısmı



(4) `gecici.sonraki = yeni` : 10 → 12. Dört bağ değişti; sıra (3) ↔ (4) değişseydi 15'e ulaşılamazdı.

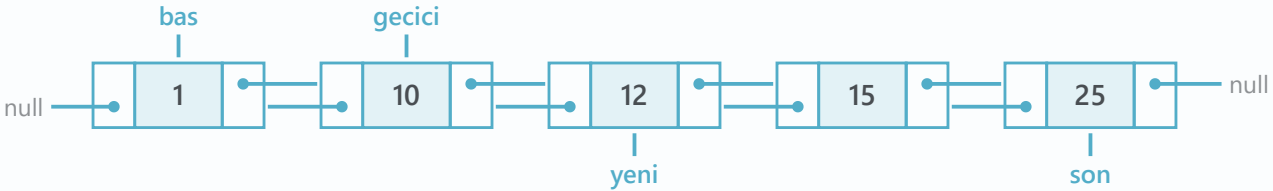
Adım 5 / 6

```
1 Dugum yeni = new Dugum(veri);
2 yeni.sonraki = gecici.sonraki;      // (1)
3 yeni.onceki = gecici;               // (2)
4 gecici.sonraki.onceki = yeni;       // (3)
▶ gecici.sonraki = yeni;              // (4)
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **12**
- `indis` = **2**
- `gecici` = **düğüm(10)**
- `yeni` = **düğüm(12)**

Kod İzleme: `konumaEkle(12, 2)` — bağlama kısmı



Sonuç (çizim düzenlendi): 1 ⇄ 10 ⇄ 12 ⇄ 15 ⇄ 25. Bağlama O(1); konuma yürümek O(n).

Adım 6 / 6

```
1 Dugum yeni = new Dugum(veri);
2 yeni.sonraki = gecici.sonraki;      // (1)
3 yeni.onceki = gecici;               // (2)
4 gecici.sonraki.onceki = yeni;       // (3)
▶ gecici.sonraki = yeni;              // (4)
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **12**
- `indis` = **2**
- `gecici` = **düğüm(10)**
- `yeni` = **düğüm(12)**

Çift Yönlü Listede Baştan ve Sondan Silme

O(1)

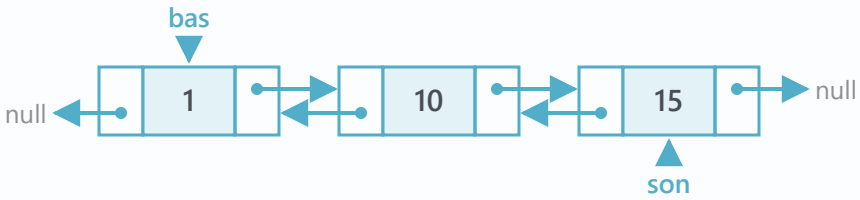
```
void ilkOgeyiSil() {
    if (bas == null) return;  // boş
    if (bas == son) {  // tek düğüm
        bas = son = null;
    } else {
        bas = bas.sonraki;
        bas.onceki = null;
    }
}
```

O(1)

```
void sonOgeyiSil() {
    if (son == null) return;  // boş
    if (bas == son) {  // tek düğüm
        bas = son = null;
    } else {
        son = son.onceki;
        son.sonraki = null;
    }
}
```

Sondan silmede önceki düğüme `son.onceki` ile tek adımda ulaşılır; **gezinme gerekmez**. Tek yönlü listede aynı işlem O(n)'dir. Tek düğüm silinirken `bas` ve `son` birlikte null yapılmalıdır.

Kod İzleme: Çift Yönlü `ilkOgeyiSil()`



Liste: 1 ⇄ 10 ⇄ 15. Boş değil.

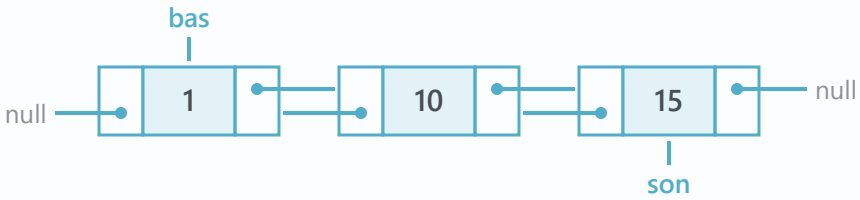
Adım 1 / 4

```
1 void ilkOgeyiSil() {
▶       if (bas == null) return;  // boş
3       if (bas == son) {  // tek düğüm
4           bas = son = null;
5       } else {
6           bas = bas.sonraki;
7           bas.onceki = null;
8       }
9   }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ilkOgeyiSil()`



`bas != son` → birden çok düğüm var, `else` dalı.

Adım 2 / 4

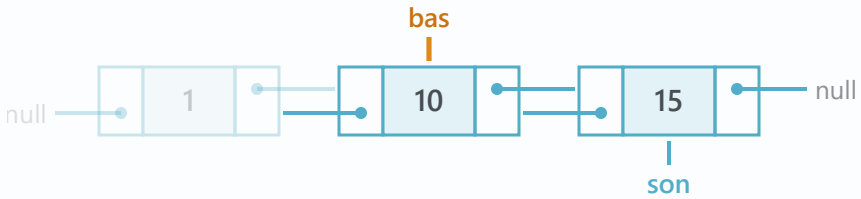
```
1 void ilkOgeyiSil() {
2     if (bas == null) return;  // boş
▶     if (bas == son) {  // tek düğüm
4         bas = son = null;
5     } else {
6         bas = bas.sonraki;
7         bas.onceki = null;
8     }
9 }
```

DEĞİŞKENLER

`bas` = **düğüm(1)**

`son` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ilkOgeyiSil()`



`bas = bas.sonraki` : baş 10'a kaydı; 1'e listeden ulaşılamaz.

Adım 3 / 4

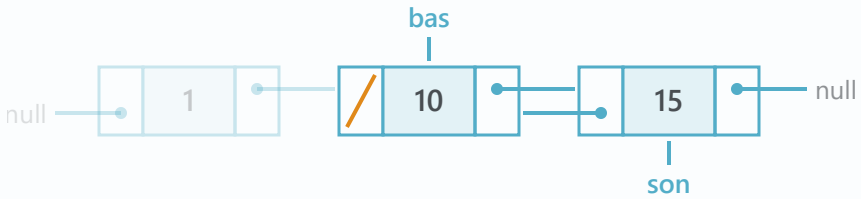
```
1 void ilkOgeyiSil() {
2     if (bas == null) return;  // boş
3     if (bas == son) {  // tek düğüm
4         bas = son = null;
5     } else {
▶         bas = bas.sonraki;
7         bas.onceki = null;
8     }
9 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)**

`son` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ilkOgeyiSil()`



`bas.onceki = null` : 10'un geri bağı da temizlenir; yoksa silinen düğüm hâlâ gösterilirdi. **O(1)**.

Adım 4 / 4

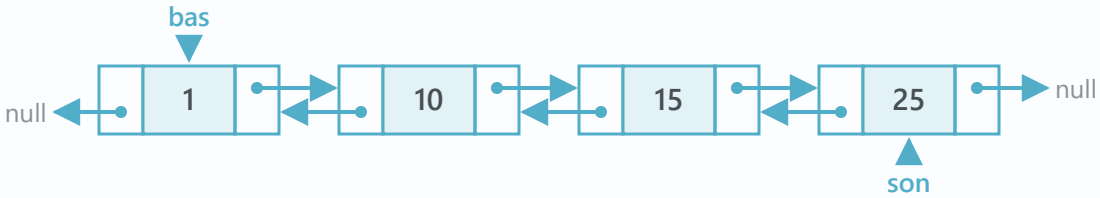
```
1 void ilkOgeyiSil() {
2     if (bas == null) return;  // boş
3     if (bas == son) {  // tek düğüm
4         bas = son = null;
5     } else {
6         bas = bas.sonraki;
▶         bas.onceki = null;
8     }
9 }
```

DEĞİŞKENLER

`bas` = **düğüm(10)**

`son` = **düğüm(15)**

Sondan Silme Adım Adım



Liste: 1 ⇄ 10 ⇄ 15 ⇄ 25; `son` 25'i gösteriyor.

Adım 1 / 4

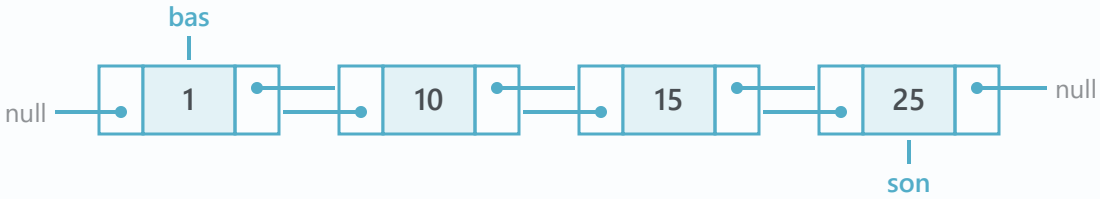
```
1 void sonOgeyiSil() {
▶       if (son == null) return;
3       if (bas == son) {
4           bas = son = null;
5       } else {
6           son = son.onceki;
7           son.sonraki = null;
8       }
9   }
```

DEĞİŞKENLER

`bas` = **düğüm(1)**

`son` = **düğüm(25)**

Sondan Silme Adım Adım



`bas != son` → `else` dalı.

Adım 2 / 4

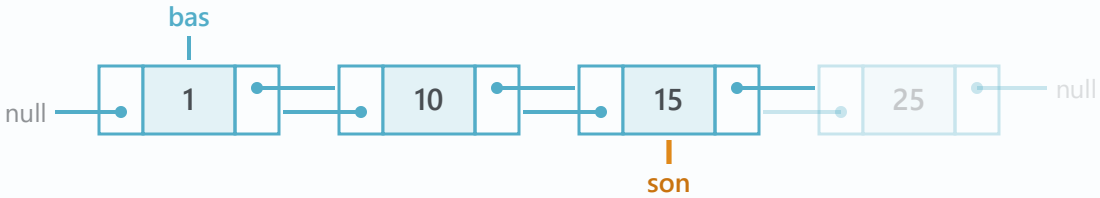
```
1 void sonOgeyiSil() {
2     if (son == null) return;
▶     if (bas == son) {
4         bas = son = null;
5     } else {
6         son = son.onceki;
7         son.sonraki = null;
8     }
9 }
```

DEĞİŞKENLER

`bas` = **düğüm(1)**

`son` = **düğüm(25)**

Sondan Silme Adım Adım



`son = son.onceki` : sondan bir önceki düğüme **tek adımda** ulaşıldı (tek yönlüde gezinme gerekirdi).

Adım 3 / 4

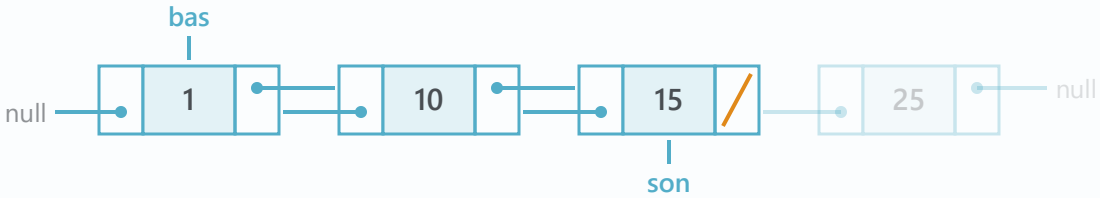
```
1 void sonOgeyiSil() {
2     if (son == null) return;
3     if (bas == son) {
4         bas = son = null;
5     } else {
▶         son = son.onceki;
7         son.sonraki = null;
8     }
9 }
```

DEĞİŞKENLER

`bas = düğüm(1)`

`son = düğüm(15)`

Sondan Silme Adım Adım



`son.sonraki = null` : 25 listeden ayrıldı. Sonuç: 1 ⇄ 10 ⇄ 15. Gezinme yok: **O(1)**.

Adım 4 / 4

```
1 void sonOgeyiSil() {
2     if (son == null) return;
3     if (bas == son) {
4         bas = son = null;
5     } else {
6         son = son.onceki;
▶         son.sonraki = null;
8     }
9 }
```

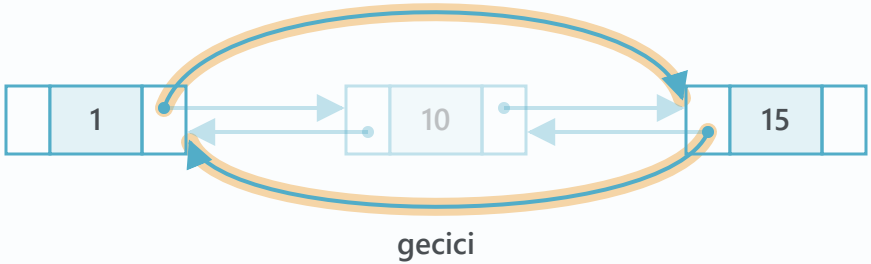
DEĞİŞKENLER

`bas` = **düğüm(1)**

`son` = **düğüm(15)**

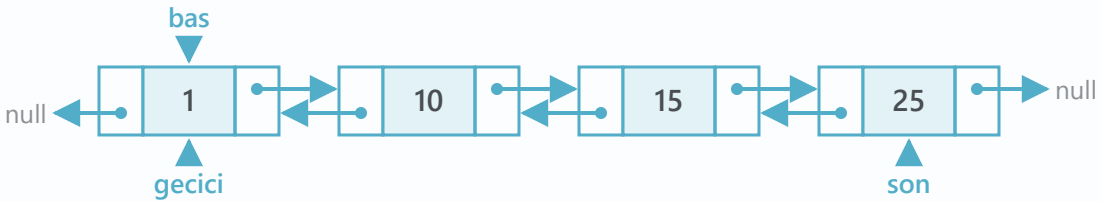
Çift Yönlü Listede Belirli Bir Öğeyi Silme

```
void ogeyiSil(int veri) {
    Dugum gecici = bas;
    while (gecici != null && gecici.veri != veri)
        gecici = gecici.sonraki;
    if (gecici == null) return;  // bulunamadı
    if (gecici.onceki != null)
        gecici.onceki.sonraki = gecici.sonraki;
    else bas = gecici.sonraki;   // baş siliniyor
    if (gecici.sonraki != null)
        gecici.sonraki.onceki = gecici.onceki;
    else son = gecici.onceki;    // son siliniyor
}
```



Silinen düğümün **komşuları birbirine bağlanır**. Baş veya son siliniyorsa `bas` / `son` güncellenir. Arama O(n), silme O(1).

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



`ogeyiSil(15)` : `gecici = bas` ; önce silinecek düğüm aranır.

Adım 1 / 11

```
▶ Dugum gecici = bas;
2 while (gecici != null && gecici.veri != veri)
3     gecici = gecici.sonraki;
4 if (gecici == null) return;  // bulunamadı
5 if (gecici.onceki != null)
6     gecici.onceki.sonraki = gecici.sonraki;
7 else bas = gecici.sonraki;   // baş siliniyor
8 if (gecici.sonraki != null)
9     gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

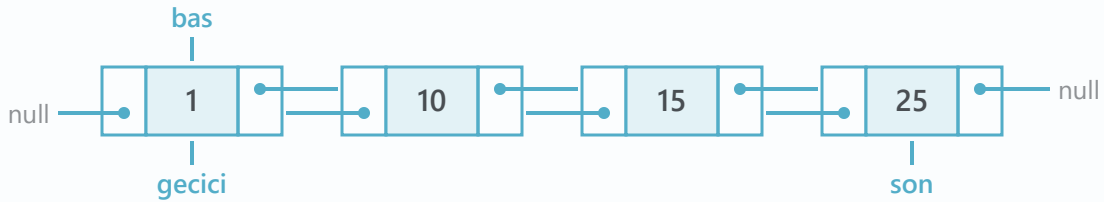
DEĞİŞKENLER

`bas = düğüm(1)`

`son = düğüm(25)` `veri = 15`

`gecici = düğüm(1)`

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



düğüm(1) null değil ve 1 ≠ 15 → ilerle.

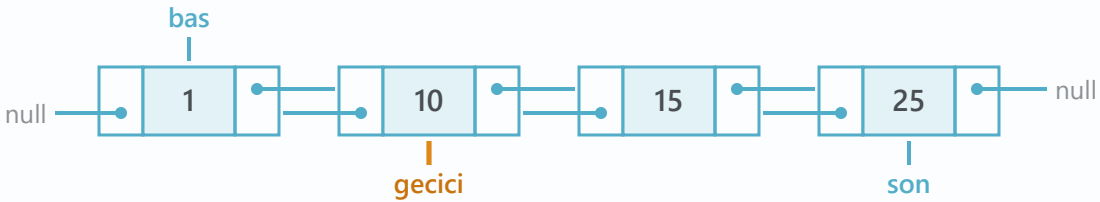
Adım 2 / 11

```
1  Dugum gecici = bas;
▶ while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(1)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



`gecici = gecici.sonraki` → düğüm(10).

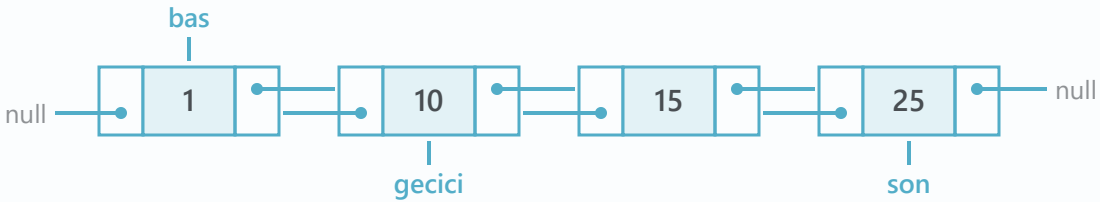
Adım 3 / 11

```
1  Dugum gecici = bas;
2  while (gecici != null && gecici.veri != veri)
▶      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(10)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



düğüm(10) null değil ve 10 ≠ 15 → ilerle.

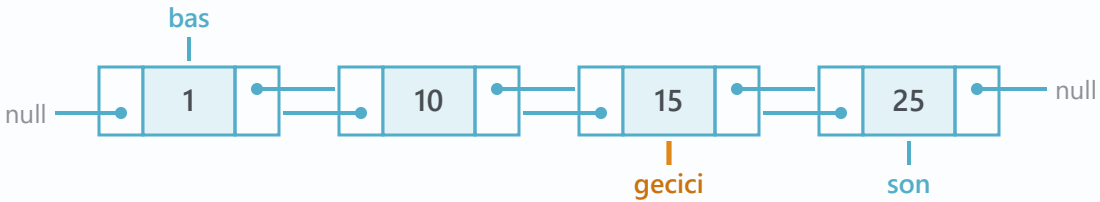
Adım 4 / 11

```
1  Dugum gecici = bas;
▶ while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(10)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



`gecici = gecici.sonraki` → düğüm(15).

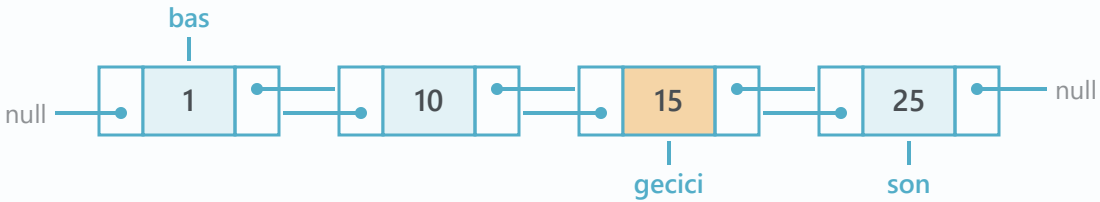
Adım 5 / 11

```
1  Dugum gecici = bas;
2  while (gecici != null && gecici.veri != veri)
▶      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



gecici.veri == 15 → döngü biter; silinecek düğüm bulundu.

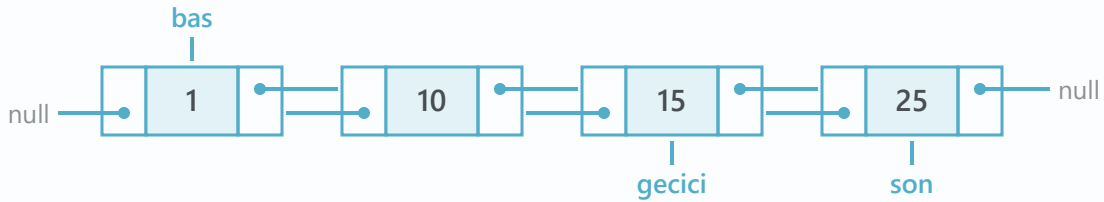
Adım 6 / 11

```
1  Dugum gecici = bas;
▶ while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



gecici null değil.

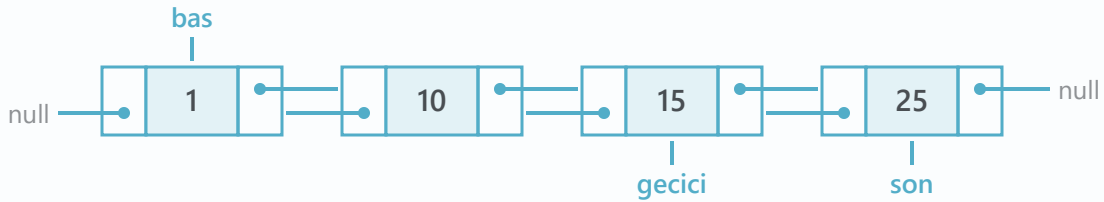
Adım 7 / 11

```
1  Dugum gecici = bas;
2  while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
▶  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



`gecici.onceki` (10) null değil → silinen baş değil.

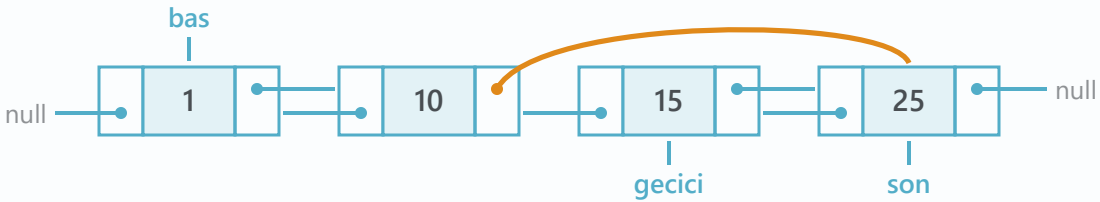
Adım 8 / 11

```
1  Dugum gecici = bas;
2  while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
▶  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



`gecici.onceki.sonraki = gecici.sonraki` : 10 → 25 ileri bağı; 15 atlanır.

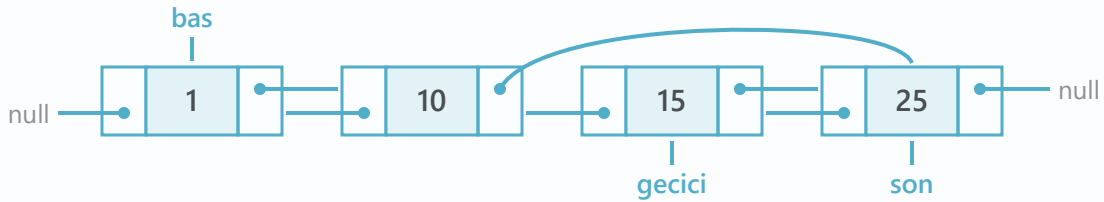
Adım 9 / 11

```
1  Dugum gecici = bas;
2  while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
▶      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



`gecici.sonraki` (25) null değil → silinen son değil.

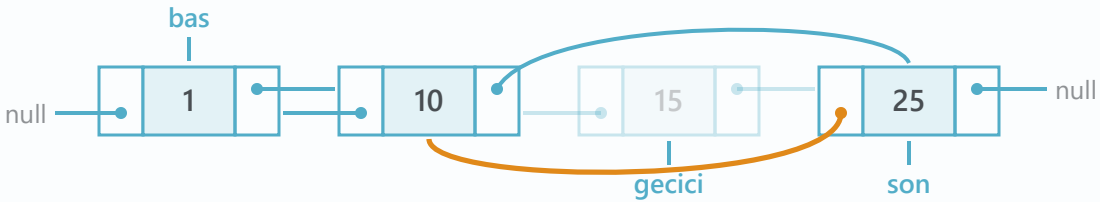
Adım 10 / 11

```
1  Dugum gecici = bas;
2  while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
▶  if (gecici.sonraki != null)
9      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(15)**

Kod İzleme: Çift Yönlü `ogeyiSil(15)`



`gecici.sonraki.onceki = gecici.onceki` : 25 → 10 geri bağı. 1 ⇄ 10 ⇄ 25. Arama O(n), bağlama O(1).

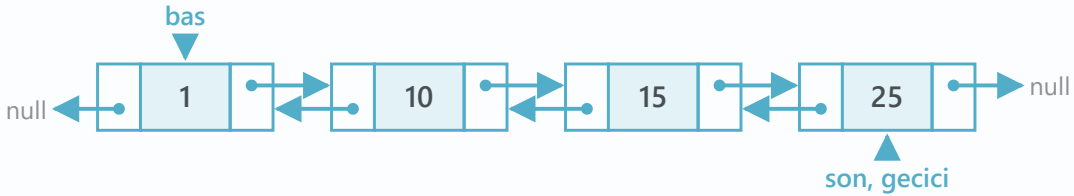
Adım 11 / 11

```
1  Dugum gecici = bas;
2  while (gecici != null && gecici.veri != veri)
3      gecici = gecici.sonraki;
4  if (gecici == null) return;  // bulunamadı
5  if (gecici.onceki != null)
6      gecici.onceki.sonraki = gecici.sonraki;
7  else bas = gecici.sonraki;   // baş siliniyor
8  if (gecici.sonraki != null)
▶      gecici.sonraki.onceki = gecici.onceki;
10 else son = gecici.onceki;    // son siliniyor
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `veri` = **15**
- `gecici` = **düğüm(15)**

Listeyi Geri Yönde Yazdırma



`gecici = son` : son düğüm doğrudan bilinir; sona gitmek için gezinmeye gerek yok.

Adım 1 / 16

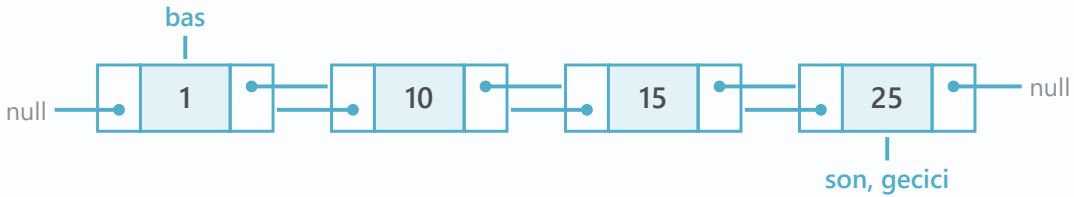
```
1 void listeyiGeriYazdir() {
▶       Dugum gecici = son;
3       System.out.print("Geri Yönde: ");
4       while (gecici != null) {
5           System.out.print(gecici.veri + " <-> ");
6           gecici = gecici.onceki;
7       }
8       System.out.println("null");
9   }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(25)**

ÇIKTI

Listeyi Geri Yönde Yazdırma



Başlık yazdırılır.

Adım 2 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
▶     System.out.print("Geri Yönde: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

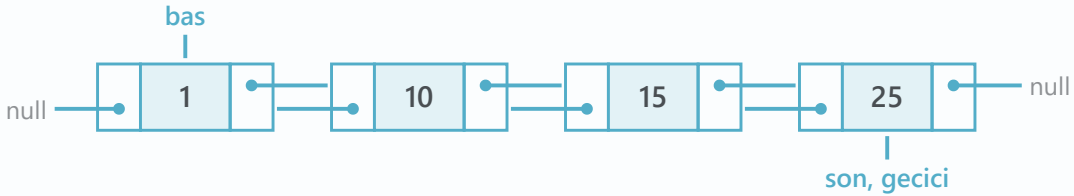
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(25)**

ÇIKTI

```
Geri Yönde:
```

Listeyi Geri Yönde Yazdırma



gecici = düğüm(25) null değil.

Adım 3 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

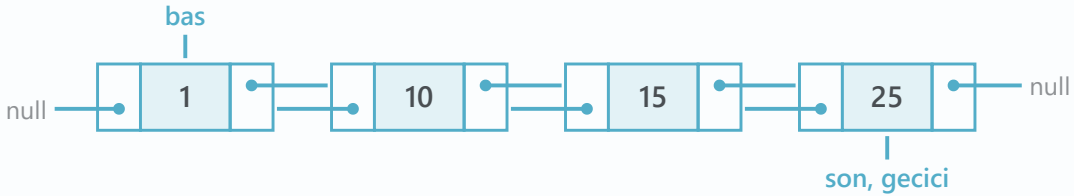
DEĞİŞKENLER

- `bas = düğüm(1)`
- `son = düğüm(25)`
- `gecici = düğüm(25)`

ÇIKTI

```
Geri Yönde:
```

Listeyi Geri Yönde Yazdırma



25 yazdırılır.

Adım 4 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
4     while (gecici != null) {
▶         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

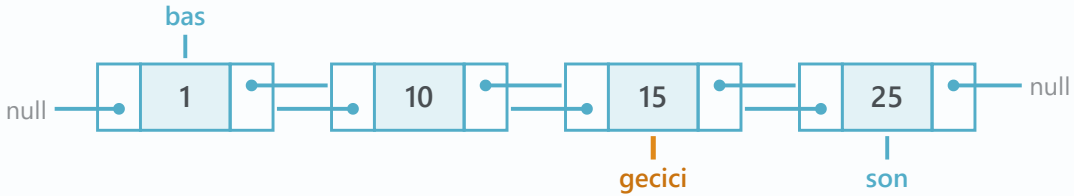
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(25)**

ÇIKTI

```
Geri Yönde: 25 <->
```

Listeyi Geri Yönde Yazdırma



`gecici = gecici.onceki` → düğüm(15).

Adım 5 / 16

```
1  void listeyiGeriYazdir() {
2      Dugum gecici = son;
3      System.out.print("Geri Yönde: ");
4      while (gecici != null) {
5          System.out.print(gecici.veri + " <-> ");
▶          gecici = gecici.onceki;
7      }
8      System.out.println("null");
9  }
```

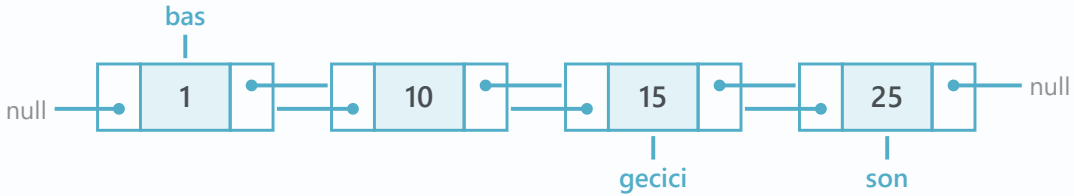
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(15)**

ÇIKTI

```
Geri Yönde: 25 <-
```

Listeyi Geri Yönde Yazdırma



gecici = düğüm(15) null değil.

Adım 6 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

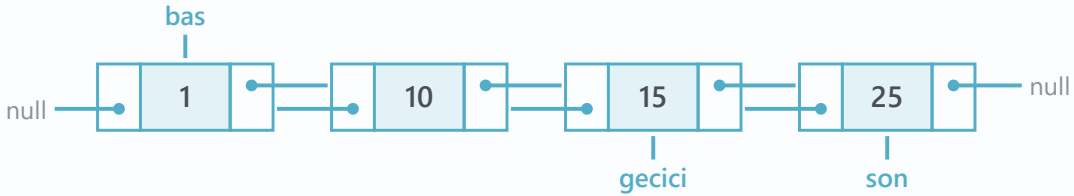
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(15)**

ÇIKTI

```
Geri Yönde: 25 <->
```

Listeyi Geri Yönde Yazdırma



15 yazdırılır.

Adım 7 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
4     while (gecici != null) {
▶         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

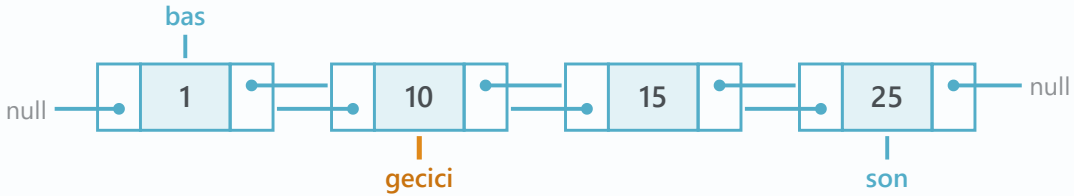
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(15)**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <->
```

Listeyi Geri Yönde Yazdırma



`gecici = gecici.onceki` → düğüm(10).

Adım 8 / 16

```
1  void listeyiGeriYazdir() {
2      Dugum gecici = son;
3      System.out.print("Geri Yönde: ");
4      while (gecici != null) {
5          System.out.print(gecici.veri + " <-> ");
▶          gecici = gecici.onceki;
7      }
8      System.out.println("null");
9  }
```

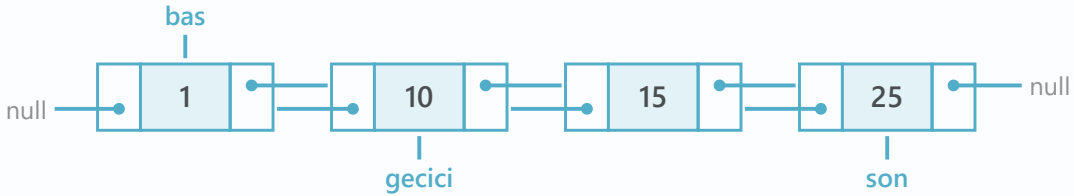
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(10)**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <->
```

Listeyi Geri Yönde Yazdırma



gecici = düğüm(10) null değil.


```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

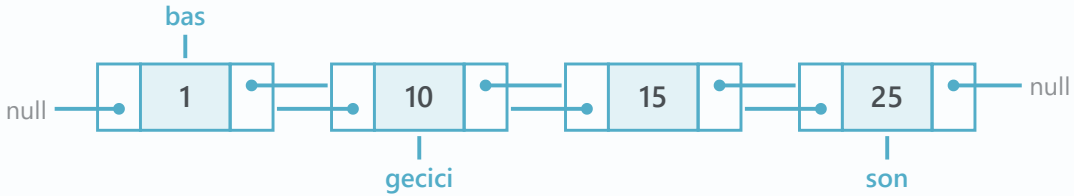
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(10)**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <->
```

Listeyi Geri Yönde Yazdırma



10 yazdırılır.

Adım 10 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
4     while (gecici != null) {
▶         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

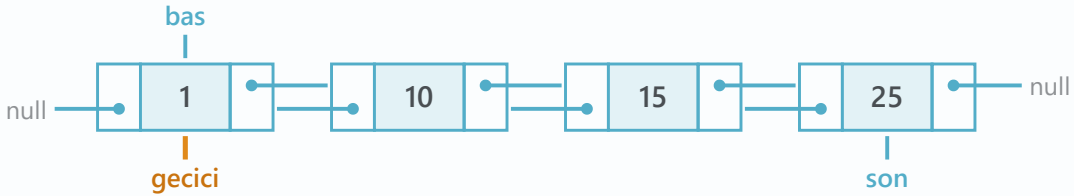
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(10)**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <-> 10 <->
```

Listeyi Geri Yönde Yazdırma



`gecici = gecici.onceki` → düğüm(1).

Adım 11 / 16

```
1  void listeyiGeriYazdir() {
2      Dugum gecici = son;
3      System.out.print("Geri Yönde: ");
4      while (gecici != null) {
5          System.out.print(gecici.veri + " <-> ");
▶          gecici = gecici.onceki;
7      }
8      System.out.println("null");
9  }
```

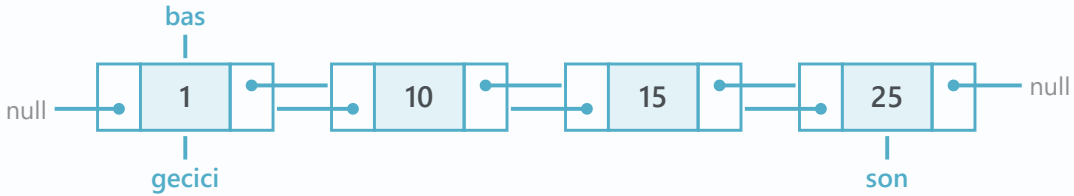
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(1)**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <-> 10 <->
```

Listeyi Geri Yönde Yazdırma



gecici = düğüm(1) null değil.


```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
▶     while (gecici != null) {
5         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

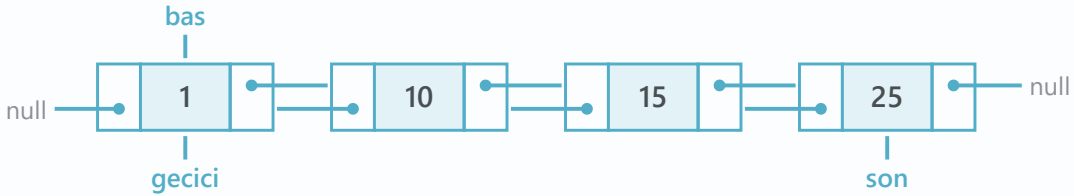
DEĞİŞKENLER

- bas = **düğüm(1)**
- son = **düğüm(25)**
- gecici = **düğüm(1)**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <-> 10 <->
```

Listeyi Geri Yönde Yazdırma



1 yazdırılır.

Adım 13 / 16

```
1  void listeyiGeriYazdir() {
2      Dugum gecici = son;
3      System.out.print("Geri Yönde: ");
4      while (gecici != null) {
▶          System.out.print(gecici.veri + " <-> ");
6          gecici = gecici.onceki;
7      }
8      System.out.println("null");
9  }
```

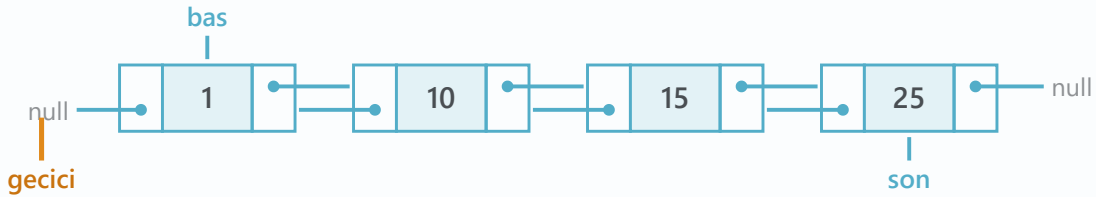
DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `son` = **düğüm(25)**
- `gecici` = **düğüm(1)**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <-> 10 <-> 1 <->
```

Listeyi Geri Yönde Yazdırma



`gecici = gecici.onceki` → **null** (1'in onceki alanı).

Adım 14 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " <-> ");
▶         gecici = gecici.onceki;
7     }
8     System.out.println("null");
9 }
```

DEĞİŞKENLER

`bas` = **düğüm(1)**

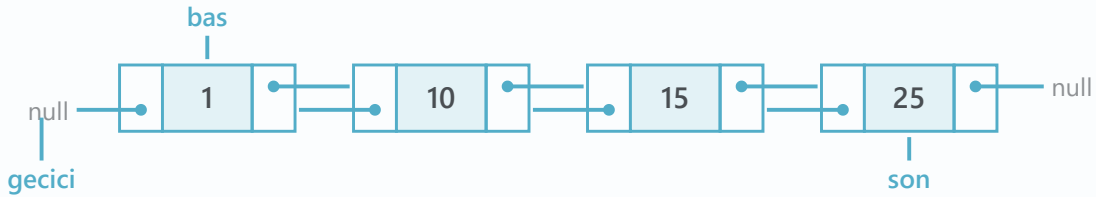
`son` = **düğüm(25)**

`gecici` = **null**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <-> 10 <-> 1 <->
```

Listeyi Geri Yönde Yazdırma



gecici == null → döngü biter.

Adım 15 / 16

```
1  void listeyiGeriYazdir() {
2      Dugum gecici = son;
3      System.out.print("Geri Yönde: ");
▶      while (gecici != null) {
5          System.out.print(gecici.veri + " <-> ");
6          gecici = gecici.onceki;
7      }
8      System.out.println("null");
9  }
```

DEĞİŞKENLER

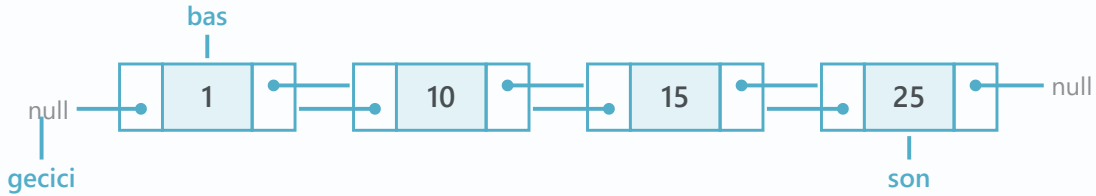
`bas = düğüm(1)`

`son = düğüm(25)` `gecici = null`

ÇIKTI

```
Geri Yönde: 25 <-> 15 <-> 10 <-> 1 <->
```

Listeyi Geri Yönde Yazdırma



"null" yazılır. İleri yazdırma aynıdır: `bas` 'tan başlayıp `sonraki` izlenir. İkisi de O(n).

Adım 16 / 16

```
1 void listeyiGeriYazdir() {
2     Dugum gecici = son;
3     System.out.print("Geri Yönde: ");
4     while (gecici != null) {
5         System.out.print(gecici.veri + " <-> ");
6         gecici = gecici.onceki;
7     }
▶     System.out.println("null");
9 }
```

DEĞİŞKENLER

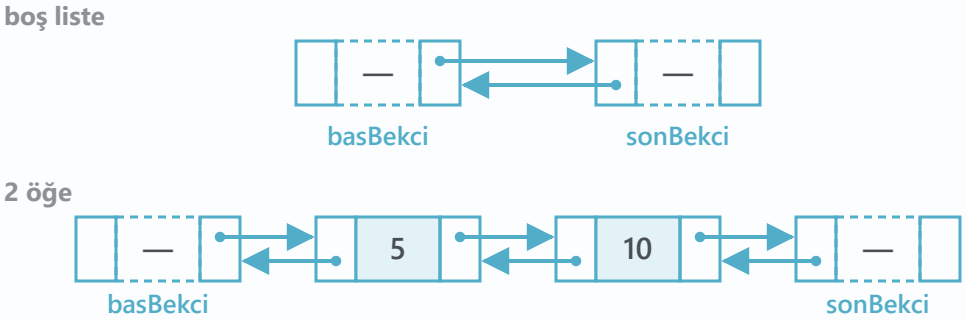
`bas` = **düğüm(1)**

`son` = **düğüm(25)** `gecici` = **null**

ÇIKTI

```
Geri Yönde: 25 <-> 15 <-> 10 <-> 1 <-> null
```

Bekçi (Sentinel) Düğümler: Başlık ve Bitiş



Fikir

Listenin iki ucuna veri taşımayan iki sabit düğüm konur (Goodrich–Tamassia: *header* / *trailer*).

Özel durum yok

Her gerçek düğümün **her zaman** bir öncesi ve sonrası vardır: `null` kontrolü, `bas` / `son` güncellemesi gerekmez.

Bedel

İki fazla düğüm; gezinme `basBekci.sonraki` 'den başlar, `sonBekci` 'de durur.

Kod İzleme: Bekçili Listede Ekleme ve Silme



Boş liste: yalnızca iki bekçi var ve birbirini gösteriyor. `boyut` = 0. Yalnızca bekçilerin dış uçları `null`.

Adım 1 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
▶  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

`basBekci` = **düğüm(–)**

`sonBekci` = **düğüm(–)** `boyut` = **0**

Kod İzleme: Bekçili Listede Ekleme ve Silme



`basaEkle(10)` : `arayaEkle(10, basBekci, basBekci.sonraki)`. Liste boşken `basBekci.sonraki` bitiş bekçisidir.

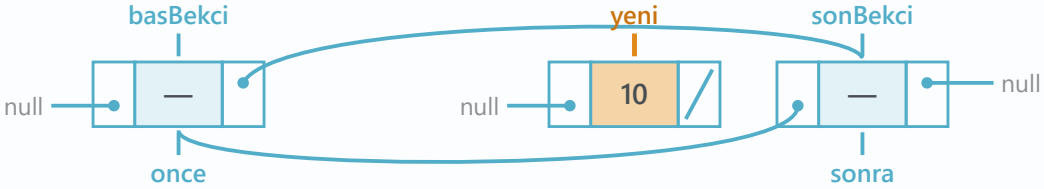
Adım 2 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
▶      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 0`
- `v = 10`
- `once = düğüm(–)`
- `sonra = düğüm(–)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`new Dugum(10)` : iki bağı da null olan yeni düğüm.

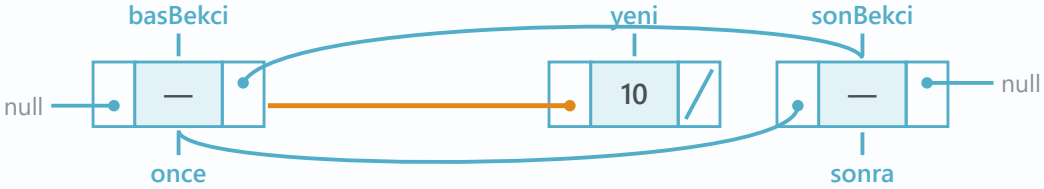
Adım 3 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
▶      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 0`
- `v = 10`
- `once = düğüm(–)`
- `sonra = düğüm(–)`
- `yeni = düğüm(10)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`yeni.onceki = once` → düğüm(—).

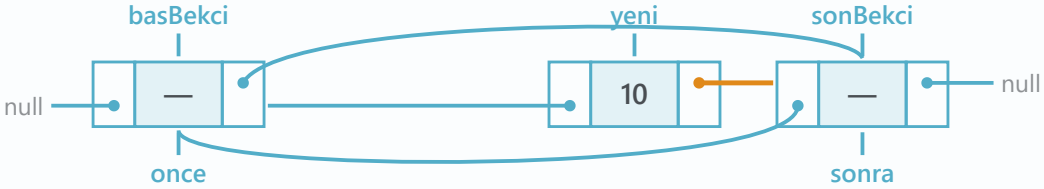
Adım 4 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
▶      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- basBekci = **düğüm(—)**
- sonBekci = **düğüm(—)**
- boyut = **0**
- v = **10**
- once = **düğüm(—)**
- sonra = **düğüm(—)**
- yeni = **düğüm(10)**

Kod İzleme: Bekçili Listede Ekleme ve Silme



`yeni.sonraki = sonra` → düğüm(—). Yeni düğüm hazır; liste henüz onu görmüyor.

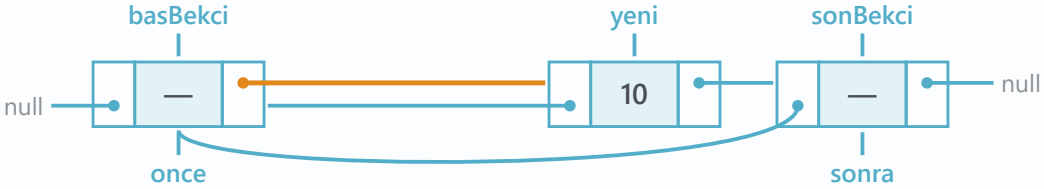
Adım 5 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
▶      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 0`
- `v = 10`
- `once = düğüm(–)`
- `sonra = düğüm(–)`
- `yeni = düğüm(10)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`once.sonraki = yeni` : ileri yönde — → 10.

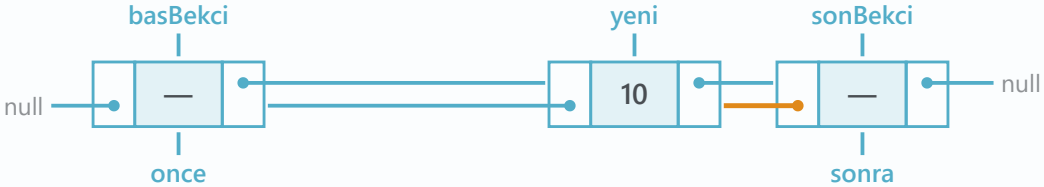
Adım 6 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
▶      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 0`
- `v = 10`
- `once = düğüm(–)`
- `sonra = düğüm(–)`
- `yeni = düğüm(10)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`sonra.onceki = yeni` : geri yönde de bağlandı; `boyut` = 1. **if yok**: once ve sonra her zaman var.

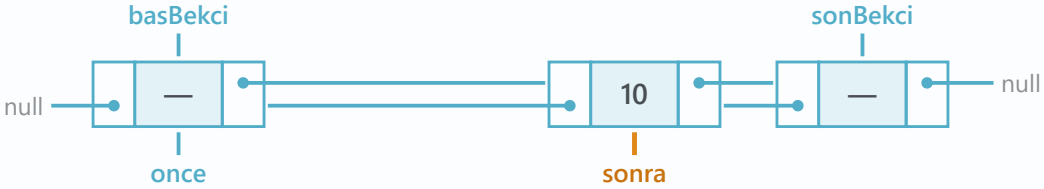
Adım 7 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
▶      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 1`
- `v = 10`
- `once = düğüm(–)`
- `sonra = düğüm(–)`
- `yeni = düğüm(10)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`basaEkle(5)` : `arayaEkle(5, basBekci, basBekci.sonraki)` — 5, başlık ile 10 arasına girer. Aynı kod, ayrı durum yok.

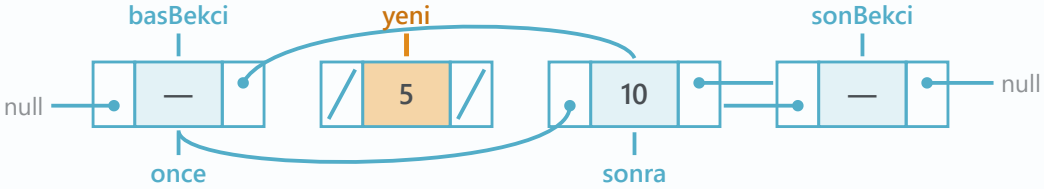
Adım 8 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
▶      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 1`
- `v = 5`
- `once = düğüm(–)`
- `sonra = düğüm(10)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`new Dugum(5)` : iki bağı da null olan yeni düğüm.

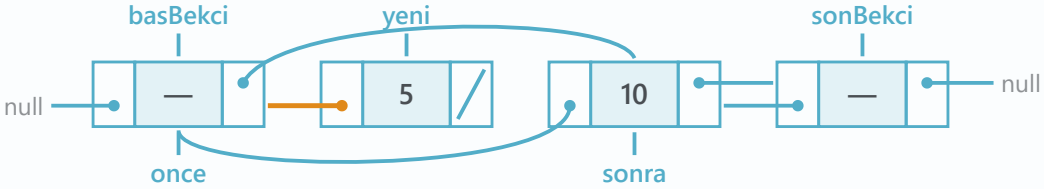
Adım 9 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
▶      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 1`
- `v = 5`
- `once = düğüm(–)`
- `sonra = düğüm(10)`
- `yeni = düğüm(5)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`yeni.onceki = once` → düğüm(—).

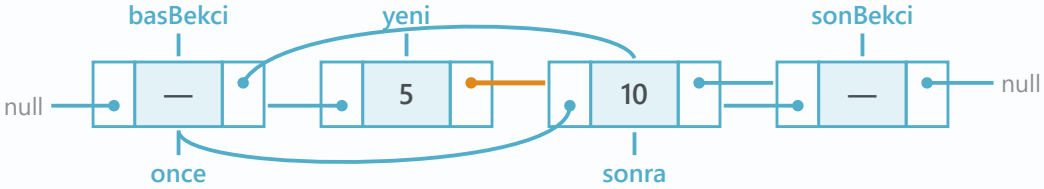
Adım 10 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
▶      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- basBekci = **düğüm(–)**
- sonBekci = **düğüm(–)**
- boyut = **1**
- v = **5**
- once = **düğüm(–)**
- sonra = **düğüm(10)**
- yeni = **düğüm(5)**

Kod İzleme: Bekçili Listede Ekleme ve Silme



`yeni.sonraki = sonra` → düğüm(10). Yeni düğüm hazır; liste henüz onu görmüyor.

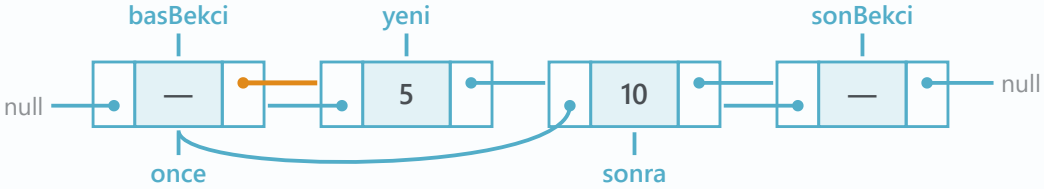
Adım 11 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
▶      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci` = **düğüm(–)**
- `sonBekci` = **düğüm(–)**
- `boyut` = **1**
- `v` = **5**
- `once` = **düğüm(–)**
- `sonra` = **düğüm(10)**
- `yeni` = **düğüm(5)**

Kod İzleme: Bekçili Listede Ekleme ve Silme



`once.sonraki = yeni` : ileri yönde — → 5.

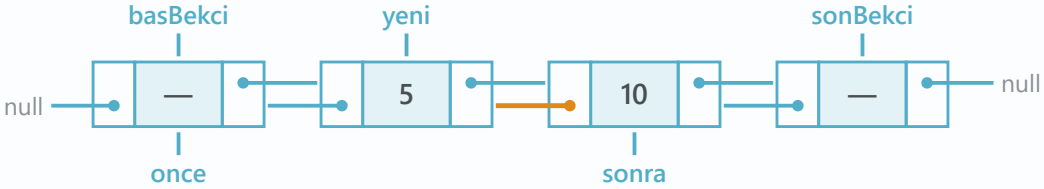
Adım 12 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
▶      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 1`
- `v = 5`
- `once = düğüm(–)`
- `sonra = düğüm(10)`
- `yeni = düğüm(5)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`sonra.onceki = yeni` : geri yönde de bağlandı; `boyut` = 2. **if yok**: once ve sonra her zaman var.

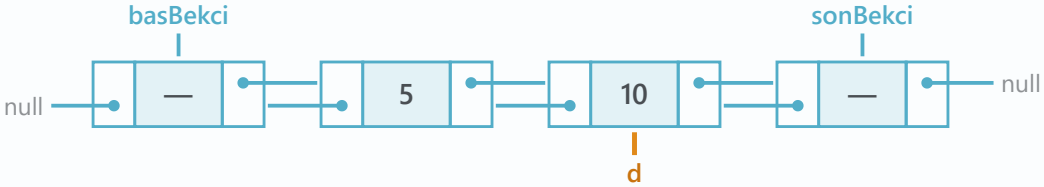
Adım 13 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
▶      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci` = **düğüm(–)**
- `sonBekci` = **düğüm(–)**
- `boyut` = **2**
- `v` = **5**
- `once` = **düğüm(–)**
- `sonra` = **düğüm(10)**
- `yeni` = **düğüm(5)**

Kod İzleme: Bekçili Listede Ekleme ve Silme



`sil(d)`, d = düğüm(10) (son gerçek düğüm). Bekçiler sayesinde komşuları kesin var.

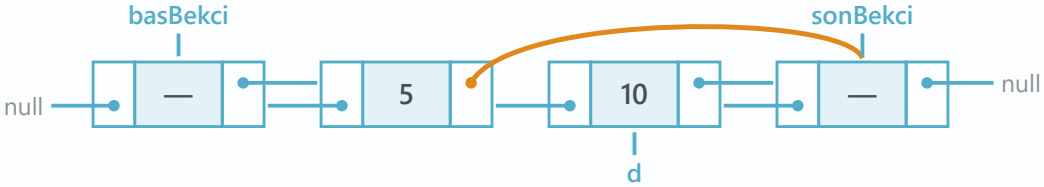
Adım 14 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
▶  void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 2`
- `d = düğüm(10)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`d.onceki.sonraki = d.sonraki` : 5 artık doğrudan sonBekci'yi gösterir.

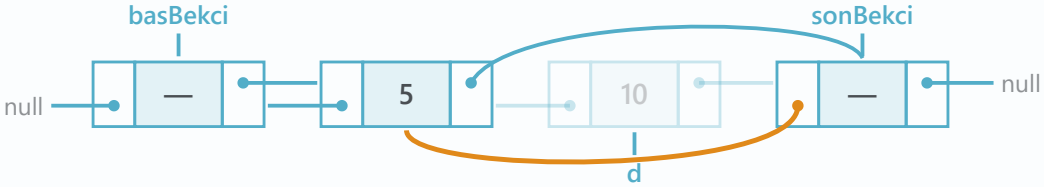
Adım 15 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
▶      d.onceki.sonraki = d.sonraki;
12     d.sonraki.onceki = d.onceki;
13     boyut--; }
```

DEĞİŞKENLER

- `basBekci = düğüm(–)`
- `sonBekci = düğüm(–)`
- `boyut = 2`
- `d = düğüm(10)`

Kod İzleme: Bekçili Listede Ekleme ve Silme



`d.sonraki.onceki = d.onceki` : 10 iki yönden de atlandı; `boyut` = 1. Son düğüm silinirken bile `son` güncellemesi yok → her işlem **O(1)**.

Adım 16 / 16

```
1  void arayaEkle(int v, Dugum once, Dugum sonra) {
2      Dugum yeni = new Dugum(v);
3      yeni.onceki = once;
4      yeni.sonraki = sonra;
5      once.sonraki = yeni;
6      sonra.onceki = yeni;  boyut++;
7  }
8  void basaEkle(int v) {
9      arayaEkle(v, basBekci, basBekci.sonraki); }
10 void sil(Dugum d) {
11     d.onceki.sonraki = d.sonraki;
▶      d.sonraki.onceki = d.onceki;
▶      boyut--; }
```

DEĞİŞKENLER

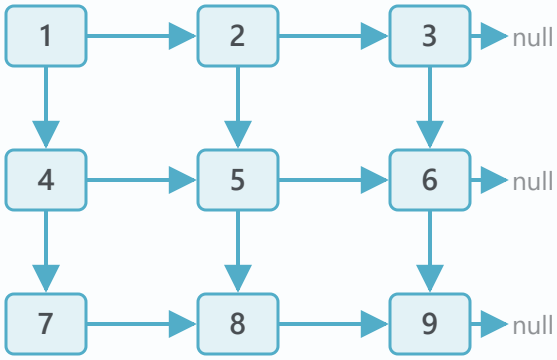
`basBekci = düğüm(–)`

`sonBekci = düğüm(–)` `boyut = 1`

`d = düğüm(10)`

Çok Yönlü Bağlantı: Matrisi Bağlı Yapıya Çevirme

Düğüm iki bağlantı tutar: `class Dugum { int veri; Dugum sag, asagi; }` (kurucu: `Dugum(int veri)`)




```
void goruntule(Dugum baslangic) {
    Dugum asagi = baslangic;
    while (asagi != null) {        // satırlar
        Dugum sag = asagi;
        while (sag != null) {      // sütunlar
            System.out.print(sag.veri + " ");
            sag = sag.sag;
        }
        System.out.println();
        asagi = asagi.asagi;
    }
}
```

Çıktı: satır satır 1 2 3 / 4 5 6 / 7 8 9. Her düğüm bir kez ziyaret edilir: O(satır · sütun).

Matristen Bağlı Yapıyı Oluşturma

```
Dugum olustur(int[][] m) {
    int r = m.length, c = m[0].length;
    Dugum[][] d = new Dugum[r][c];
    for (int i = r - 1; i >= 0; i--)
        for (int j = c - 1; j >= 0; j--) {
            Dugum n = new Dugum(m[i][j]);
            if (j + 1 < c) n.sag = d[i][j+1];
            if (i + 1 < r) n.asagi = d[i+1][j];
            d[i][j] = n;
        }
    return d[0][0];
}
```

Sağ alttan başla

Sağdaki ve alttaki düğüm önceden oluşturulmuş olur; her hücre için **tek** düğüm.

O(satır · sütun)

Her hücre bir kez işlenir.

Neden `d` tablosu? Doğrudan `olustur(i, j+1)` ve `olustur(i+1, j)` çağıran özyinelemeli bir sürüm aynı hücreyi defalarca yeniden oluşturur (üstel sayıda düğüm; sağ ve alt yollar ortak düğümü paylaşmaz).

Kod İzleme: `olustur(m)` — 2×3 matris



`d` : 2×3'lük **düğüm referansı** dizisi, hepsi null. Düğümler sağ alttan başlayarak oluşturulacak.

Adım 1 / 8

```
1  Dugum olustur(int[][] m) {
▶      int r = m.length, c = m[0].length;
▶      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
6              Dugum n = new Dugum(m[i][j]);
7              if (j + 1 < c) n.sag = d[i][j+1];
8              if (i + 1 < r) n.asagi = d[i+1][j];
9              d[i][j] = n;
10         }
11     return d[0][0];
12 }
```

DEĞİŞKENLER

`r = 2` `c = 3`

`d = [[null, null, null], [null, null, null]]`

Kod İzleme: `olustur(m)` — 2×3 matris



(i=1, j=2): `new Dugum(6)` ; `sag` yok (son sütun), `asagi` yok (son satır). Komşular zaten hazır, hiçbir hücre iki kez oluşmaz.

Adım 2 / 8

```
1  Dugum olustur(int[][] m) {
2      int r = m.length, c = m[0].length;
3      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
▶              Dugum n = new Dugum(m[i][j]);
▶              if (j + 1 < c) n.sag = d[i][j+1];
▶              if (i + 1 < r) n.asagi = d[i+1][j];
▶              d[i][j] = n;
10         }
11     return d[0][0];
12 }
```

DEĞİŞKENLER

`r = 2` `c = 3` `i = 1` `j = 2`

`n = düğüm(6)`

`d = [[null, null, null], [null, null, düğüm(6)]]`

Kod İzleme: `olustur(m)` — 2×3 matris



(i=1, j=1): `new Dugum(5)`; `sag` → 6, `asagi` yok (son satır). Komşular zaten hazır, hiçbir hücre iki kez oluşmaz.

Adım 3 / 8

```
1  Dugum olustur(int[][] m) {
2      int r = m.length, c = m[0].length;
3      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
▶              Dugum n = new Dugum(m[i][j]);
▶              if (j + 1 < c) n.sag = d[i][j+1];
▶              if (i + 1 < r) n.asagi = d[i+1][j];
▶              d[i][j] = n;
10         }
11     return d[0][0];
12 }
```

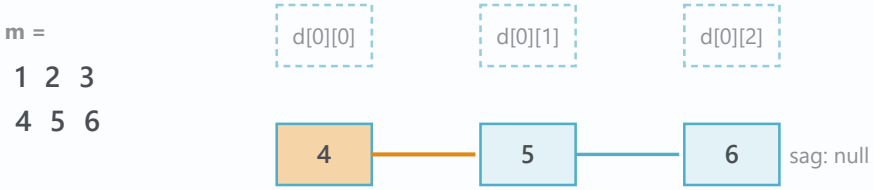
DEĞİŞKENLER

`r = 2` `c = 3` `i = 1` `j = 1`

`n = düğüm(5)`

`d = [[null, null, null], [null, düğüm(5), düğüm(6)]]`

Kod İzleme: `olustur(m)` — 2×3 matris



(i=1, j=0): `new Dugum(4)` ; `sag` → 5, `asagi` yok (son satır). Komşular zaten hazır, hiçbir hücre iki kez oluşmaz.

Adım 4 / 8

```
1  Dugum olustur(int[][] m) {
2      int r = m.length, c = m[0].length;
3      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
▶              Dugum n = new Dugum(m[i][j]);
▶              if (j + 1 < c) n.sag = d[i][j+1];
▶              if (i + 1 < r) n.asagi = d[i+1][j];
▶              d[i][j] = n;
10         }
11     return d[0][0];
12 }
```

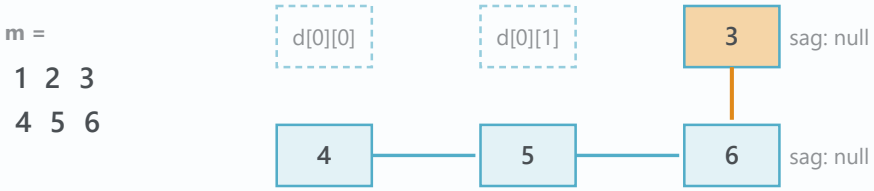
DEĞİŞKENLER

`r = 2` `c = 3` `i = 1` `j = 0`

`n = düğüm(4)`

`d = [[null, null, null], [düğüm(4), düğüm(5), düğüm(6)]]`

Kod İzleme: `olustur(m)` — 2×3 matris



(i=0, j=2): `new Dugum(3)` ; `sag` yok (son sütun), `asagi` → 6. Komşular zaten hazır, hiçbir hücre iki kez oluşmaz.

Adım 5 / 8

```
1  Dugum olustur(int[][] m) {
2      int r = m.length, c = m[0].length;
3      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
▶              Dugum n = new Dugum(m[i][j]);
▶              if (j + 1 < c) n.sag = d[i][j+1];
▶              if (i + 1 < r) n.asagi = d[i+1][j];
▶              d[i][j] = n;
10         }
11     return d[0][0];
12 }
```

DEĞİŞKENLER

`r = 2` `c = 3` `i = 0` `j = 2`

`n = düğüm(3)`

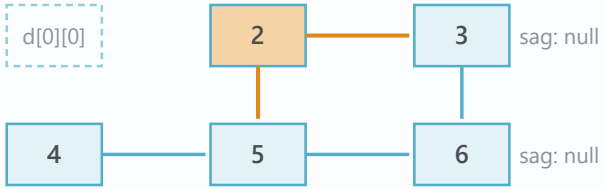
`d = [[null, null, düğüm(3)], [düğüm(4), düğüm(5), düğüm(6)]]`

Kod İzleme: `olustur(m)` — 2×3 matris

m =

1 2 3

4 5 6



(i=0, j=1): `new Dugum(2)`; `sag` → 3, `asagi` → 5. Komşular zaten hazır, hiçbir hücre iki kez oluşmaz.

Adım 6 / 8

```
1  Dugum olustur(int[][] m) {
2      int r = m.length, c = m[0].length;
3      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
▶              Dugum n = new Dugum(m[i][j]);
▶              if (j + 1 < c) n.sag = d[i][j+1];
▶              if (i + 1 < r) n.asagi = d[i+1][j];
▶              d[i][j] = n;
10         }
11     return d[0][0];
12 }
```

DEĞİŞKENLER

`r = 2` `c = 3` `i = 0` `j = 1`

`n = düğüm(2)`

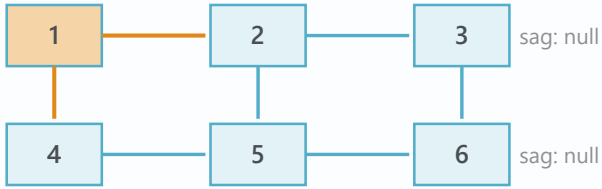
`d = [[null, düğüm(2), düğüm(3)], [düğüm(4), düğüm(5), düğüm(6)]]`

Kod İzleme: `olustur(m)` — 2×3 matris

m =

1 2 3

4 5 6



(i=0, j=0): `new Dugum(1)`; `sag` → 2, `asagi` → 4. Komşular zaten hazır, hiçbir hücre iki kez oluşmaz.

Adım 7 / 8

```
1  Dugum olustur(int[][] m) {
2      int r = m.length, c = m[0].length;
3      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
▶              Dugum n = new Dugum(m[i][j]);
▶              if (j + 1 < c) n.sag = d[i][j+1];
▶              if (i + 1 < r) n.asagi = d[i+1][j];
▶              d[i][j] = n;
10         }
11     return d[0][0];
12 }
```

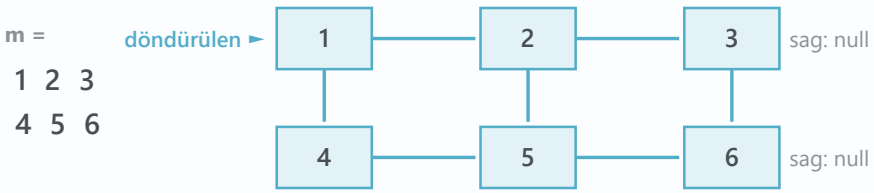
DEĞİŞKENLER

`r = 2` `c = 3` `i = 0` `j = 0`

`n = düğüm(1)`

`d = [[düğüm(1), düğüm(2), düğüm(3)], [düğüm(4), düğüm(5), düğüm(6)]]`

Kod İzleme: `olustur(m)` — 2×3 matris



`d[0][0]` (1) döndürülür: tüm yapıya sol üst köşeden `sag` / `asagi` ile ulaşılır. 6 hücre, 6 düğüm → O(r·c).

Adım 8 / 8

```
1  Dugum olustur(int[][] m) {
2      int r = m.length, c = m[0].length;
3      Dugum[][] d = new Dugum[r][c];
4      for (int i = r - 1; i >= 0; i--)
5          for (int j = c - 1; j >= 0; j--) {
6              Dugum n = new Dugum(m[i][j]);
7              if (j + 1 < c) n.sag = d[i][j+1];
8              if (i + 1 < r) n.asagi = d[i+1][j];
9              d[i][j] = n;
10         }
▶      return d[0][0];
12 }
```

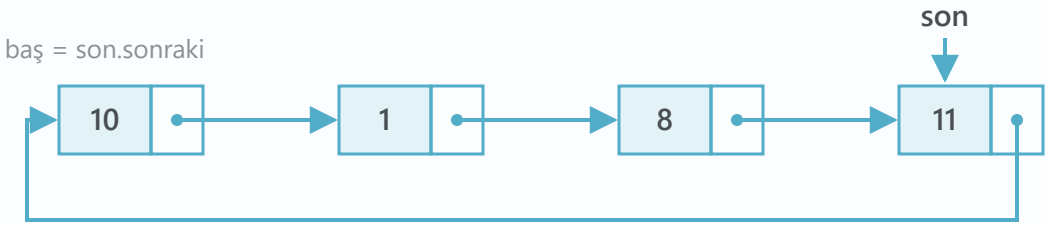
DEĞİŞKENLER

`r = 2` `c = 3`

`d = [[düğüm(1), düğüm(2), düğüm(3)], [düğüm(4), düğüm(5), düğüm(6)]]`

Dairesel Bağlı Liste (Circularly Linked List)

- Son düğüm `null` yerine **ilk düğümü** işaret eder; liste bir döngü (halka) oluşturur.
- Her düğüm yine kendisinden sonraki düğümü gösterir; listede `null` bağlantı yoktur.
- Yalnızca `son` tutmak yeterlidir: baş düğüm `son.sonraki`'dir.
- Kullanım: sırayla dönen işler (round-robin çizelgeleme), çok oyunculu oyunlarda sıra, dairesel çalma listesi.



Dairesel Liste Sınıfı

```java
public class DaireselBagliListe {
    class Dugum {
        int veri;
        Dugum sonraki;
        Dugum(int veri) {
            this.veri = veri;
            this.sonraki = null;
        }
    }
    private Dugum son;  // baş = son.sonraki
    private int boyut;
}
```

Neden yalnızca son?

Baş = `son.sonraki`. Böylece hem başa hem sona O(1) erişilir.

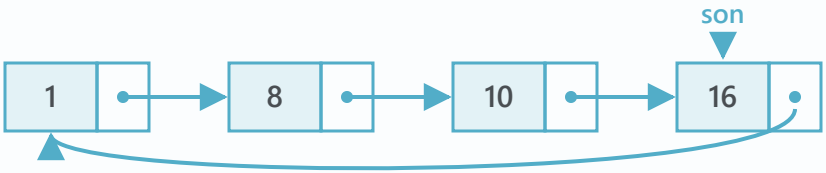
Ayrıca bas tutulursa

Her işlemde iki işaretçi güncellenmeli; gereksiz.

Özel durumlar

Boş: `son == null`. Tek düğüm: `son.sonraki == son`.

Dairesel Listeyi Yazdırma



Liste boş değil. Yalnızca `son` tutulur; son düğüm başı gösterir (alttaki bağ).

Adım 1 / 13

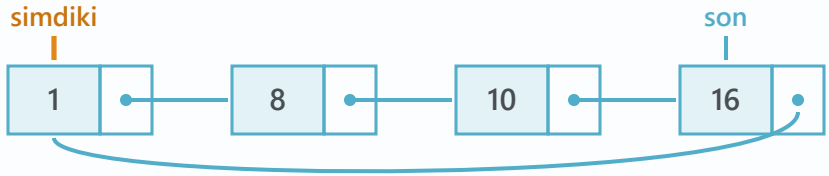
```
1 void yazdir() {
▶       if (son == null) return;
3       Dugum simdiki = son.sonraki;
4       while (simdiki != son) {
5           System.out.print(simdiki.veri + " ");
6           simdiki = simdiki.sonraki;
7       }
8       System.out.print(simdiki.veri + " ");
9   }
```

DEĞİŞKENLER

`son = düğüm(16)`

ÇIKTI

Dairesel Listeyi Yazdırma



Baş düğüm `son.sonraki` : `simdiki` = 1.

Adım 2 / 13

```
1 void yazdir() {
2     if (son == null) return;
▶     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

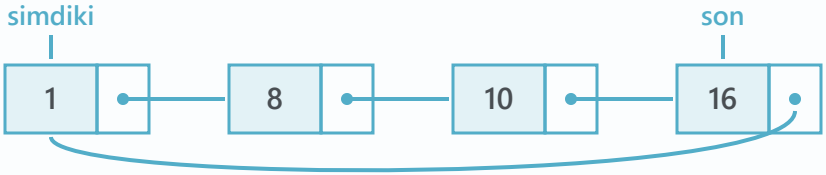
DEĞİŞKENLER

`son = düğüm(16)`

`simdiki = düğüm(1)`

ÇIKTI

Dairesel Listeyi Yazdırma



simdiki (1) != son (16) → döngü.

Adım 3 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
▶     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

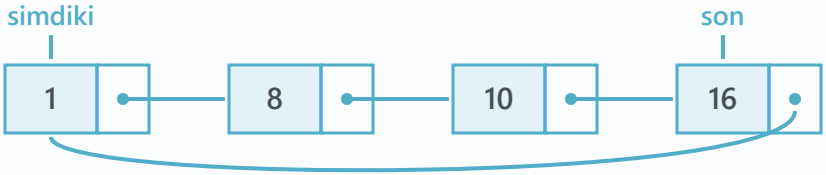
DEĞİŞKENLER

`son = düğüm(16)`

`simdiki = düğüm(1)`

ÇIKTI

Dairesel Listeyi Yazdırma



1 yazdırılır.

Adım 4 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
▶         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

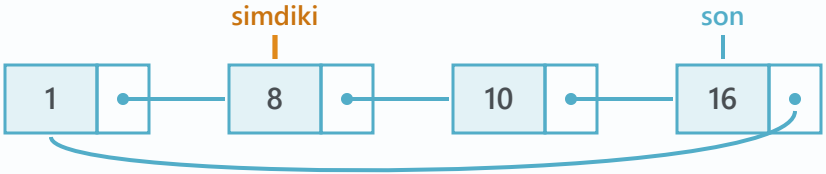
`son = düğüm(16)`

`simdiki = düğüm(1)`

ÇIKTI

```
1
```

Dairesel Listeyi Yazdırma



`simdiki = simdiki.sonraki` → düğüm(8).

Adım 5 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
▶         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

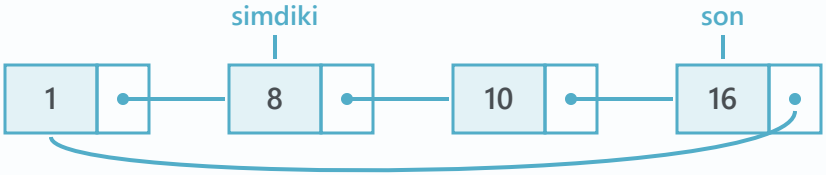
`son = düğüm(16)`

`simdiki = düğüm(8)`

ÇIKTI

```
1
```

Dairesel Listeyi Yazdırma



simdiki (8) != son (16) → döngü.

Adım 6 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
▶     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

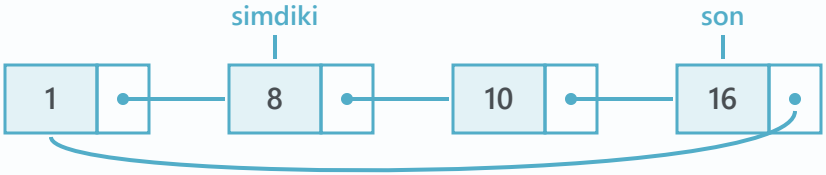
`son = düğüm(16)`

`simdiki = düğüm(8)`

ÇIKTI

```
1
```

Dairesel Listeyi Yazdırma



8 yazdırılır.

Adım 7 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
▶         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

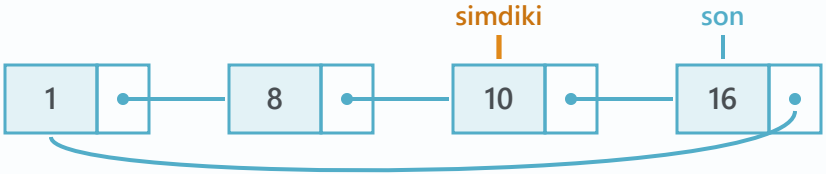
`son = düğüm(16)`

`simdiki = düğüm(8)`

ÇIKTI

```
1 8
```

Dairesel Listeyi Yazdırma



`simdiki = simdiki.sonraki` → düğüm(10).

Adım 8 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
▶         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

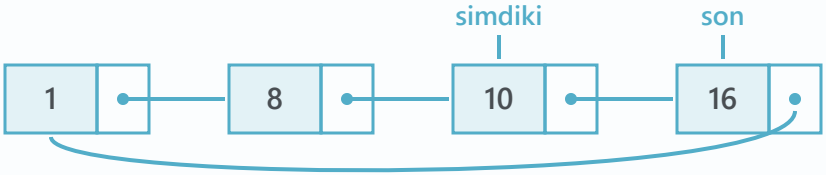
`son = düğüm(16)`

`simdiki = düğüm(10)`

ÇIKTI

```
1 8
```

Dairesel Listeyi Yazdırma



simdiki (10) != son (16) → döngü.

Adım 9 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
▶     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

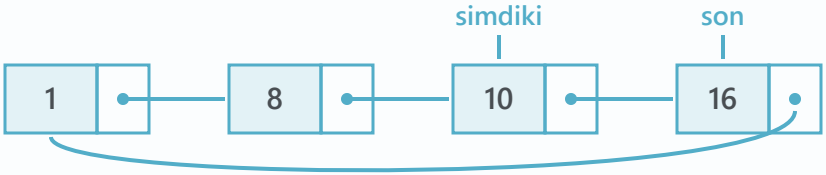
`son = düğüm(16)`

`simdiki = düğüm(10)`

ÇIKTI

```
1 8
```

Dairesel Listeyi Yazdırma



10 yazdırılır.

Adım 10 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
▶         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

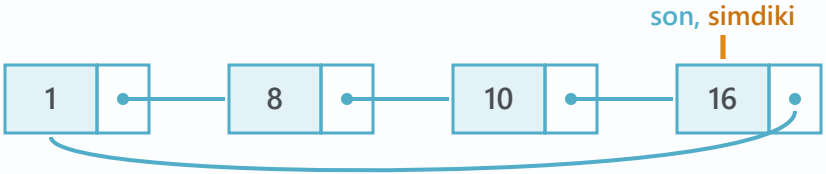
`son = düğüm(16)`

`simdiki = düğüm(10)`

ÇIKTI

```
1 8 10
```

Dairesel Listeyi Yazdırma



`simdiki = simdiki.sonraki` → düğüm(16).

Adım 11 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
▶         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

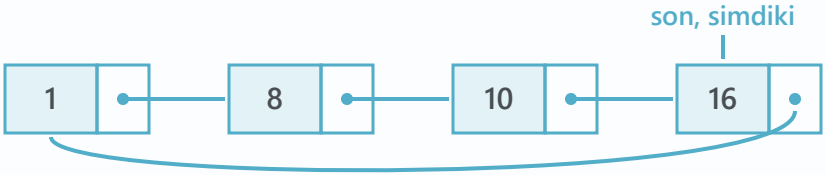
`son = düğüm(16)`

`simdiki = düğüm(16)`

ÇIKTI

```
1 8 10
```

Dairesel Listeyi Yazdırma



`simdiki == son` → döngü biter (null'a hiç ulaşılmaz; koşul `!= son`).

Adım 12 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
▶     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
8     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

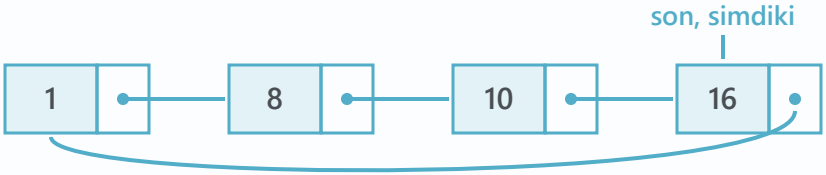
`son` = **düğüm(16)**

`simdiki` = **düğüm(16)**

ÇIKTI

```
1 8 10
```

Dairesel Listeyi Yazdırma



Son düğüm döngüden sonra ayrıca yazdırılır. Her düğüm bir kez: **O(n)**.

Adım 13 / 13

```
1 void yazdir() {
2     if (son == null) return;
3     Dugum simdiki = son.sonraki;
4     while (simdiki != son) {
5         System.out.print(simdiki.veri + " ");
6         simdiki = simdiki.sonraki;
7     }
▶     System.out.print(simdiki.veri + " ");
9 }
```

DEĞİŞKENLER

`son = düğüm(16)`

`simdiki = düğüm(16)`

ÇIKTI

```
1 8 10 16
```

Genel Döngü Tespiti: Floyd Algoritması

```
boolean donguVarMi(Dugum bas) {
    Dugum yavas = bas, hizli = bas;
    while (hizli != null &&
           hizli.sonraki != null) {
        yavas = yavas.sonraki;
        hizli = hizli.sonraki.sonraki;
        if (yavas == hizli) return true;
    }
    return false;
}
```

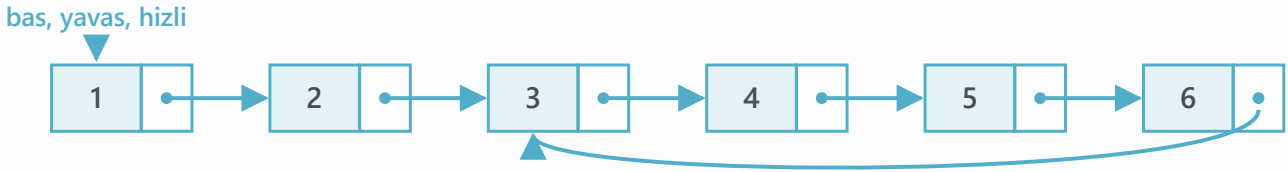
- Yavaş işaretçi her turda bir, hızlı işaretçi iki düğüm ilerler.
- Döngü varsa iki işaretçi sonunda döngünün içinde buluşur.
- Döngü yoksa hızlı işaretçi `null` 'a ulaşır.
- Döngünün mutlaka `bas` düğümüne dönmesi gerekmez.

Zaman
O(n)

Ek bellek
O(1)

Sadece düğümleri `bas` 'a dönene kadar izlemek yeterli değildir: döngü listenin ortasındaki bir düğüme de bağlanabilir.

Kod İzleme: `donguVarMi` (Floyd)



Liste 1 → … → 6 ve 6 tekrar 3'ü gösteriyor (döngü, baş düğüme dönmüyor). İki işaretçi de baştan başlar.

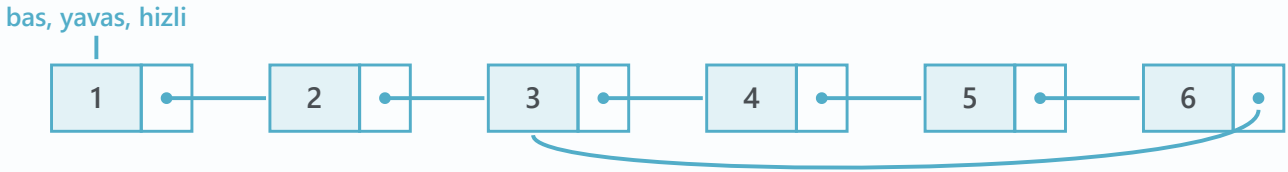
Adım 1 / 17

```
1  boolean donguVarMi(Dugum bas) {
▶      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(1)**
- `hizli` = **düğüm(1)**

Kod İzleme: `donguVarMi` (Floyd)



1. tur: hizli (1) ve hizli.sonraki (2) null değil.

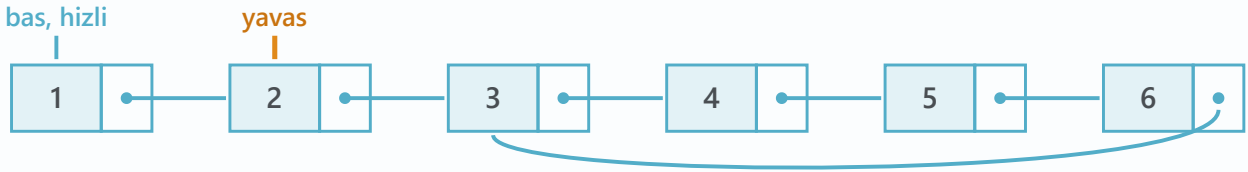
Adım 2 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
▶      while (hizli != null &&
▶             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(1)**
- `hizli` = **düğüm(1)**

Kod İzleme: `donguVarMi` (Floyd)



`yavas` 1 adım → düğüm(2).

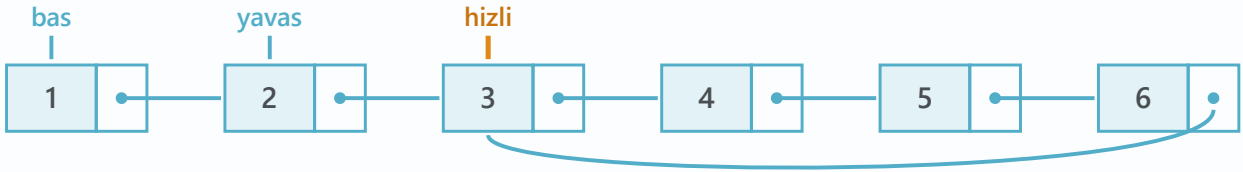
Adım 3 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
▶          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(2)**
- `hizli` = **düğüm(1)**

Kod İzleme: `donguVarMi` (Floyd)



`hizli` 2 adım → düğüm(3).

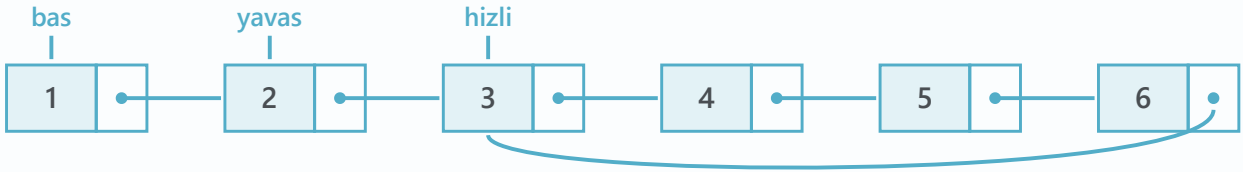
Adım 4 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
▶          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(2)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



yavas (2) ≠ hizli (3) → devam.

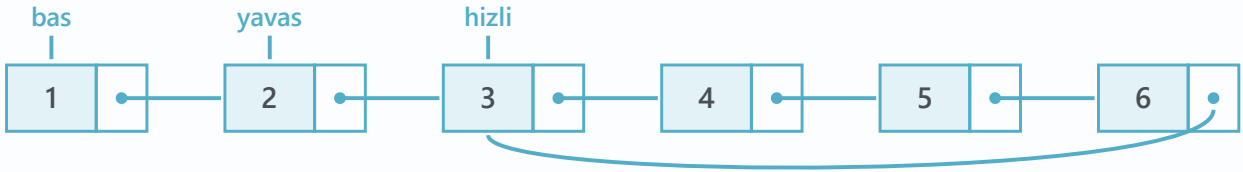
Adım 5 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
▶          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(2)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



2. tur: hizli (3) ve hizli.sonraki (4) null değil.

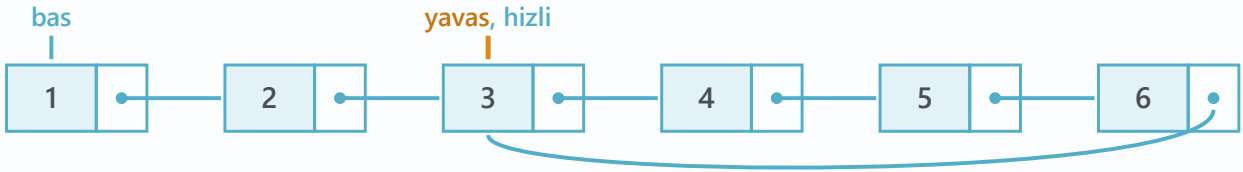
Adım 6 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
▶      while (hizli != null &&
▶             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(2)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



`yavas` 1 adım → düğüm(3).

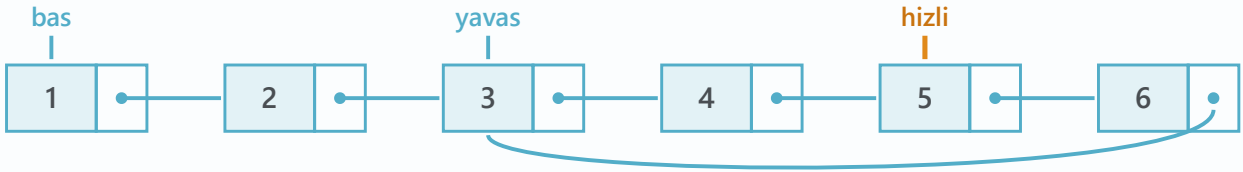
Adım 7 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
▶          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(3)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



`hizli` 2 adım → düğüm(5).

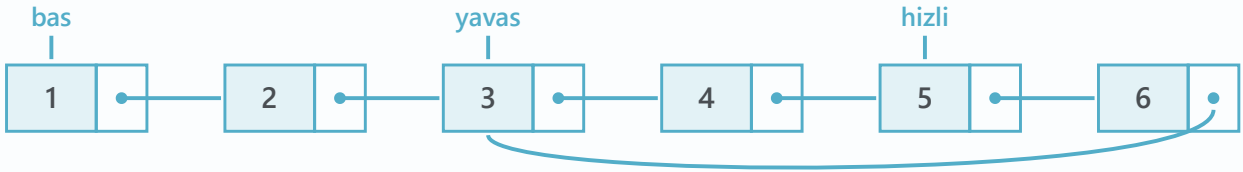
Adım 8 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
▶          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(3)**
- `hizli` = **düğüm(5)**

Kod İzleme: `donguVarMi` (Floyd)



yavas (3) ≠ hizli (5) → devam.

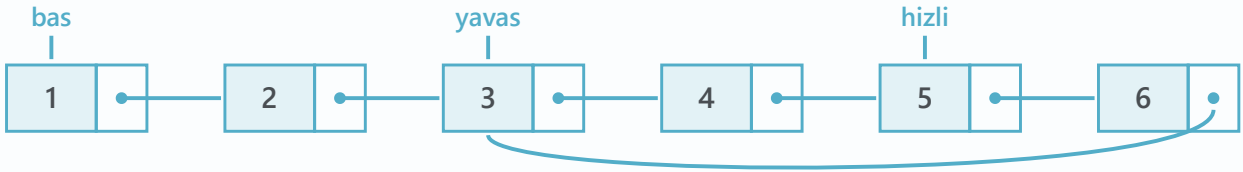
Adım 9 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
▶          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(3)**
- `hizli` = **düğüm(5)**

Kod İzleme: `donguVarMi` (Floyd)



3. tur: hizli (5) ve hizli.sonraki (6) null değil.

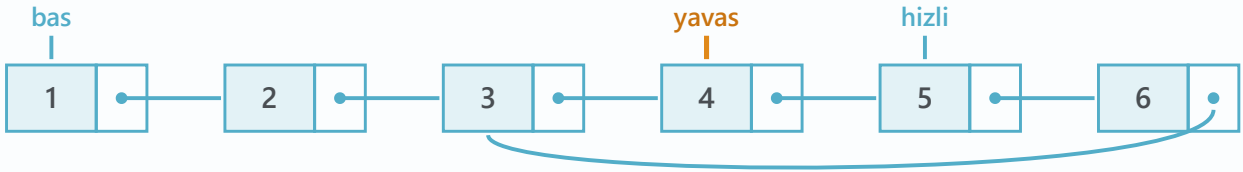
Adım 10 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
▶      while (hizli != null &&
▶             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(3)**
- `hizli` = **düğüm(5)**

Kod İzleme: `donguVarMi` (Floyd)



`yavas` 1 adım → düğüm(4).

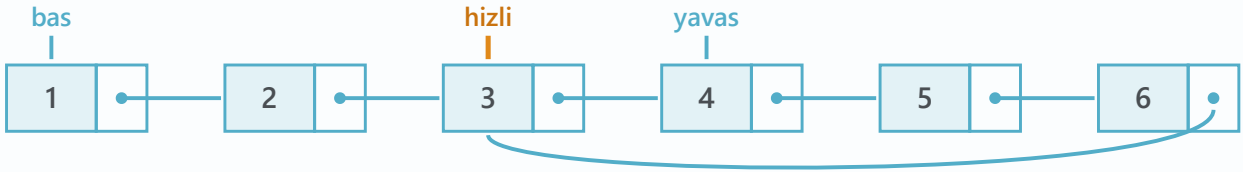
Adım 11 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
▶          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(4)**
- `hizli` = **düğüm(5)**

Kod İzleme: `donguVarMi` (Floyd)



`hizli` 2 adım → düğüm(3).

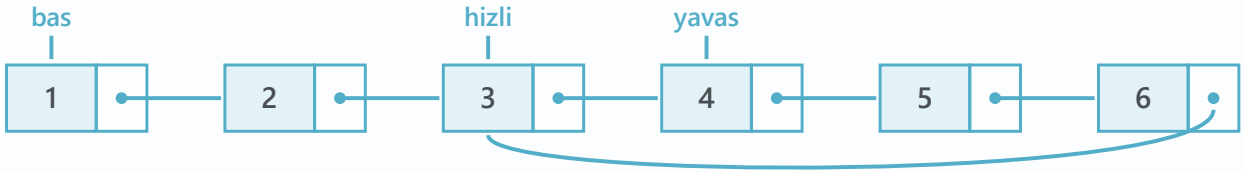
Adım 12 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
▶          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(4)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



yavas (4) ≠ hizli (3) → devam. hizli döngüye girdi, yavaşın arkasından yaklaşıyor.

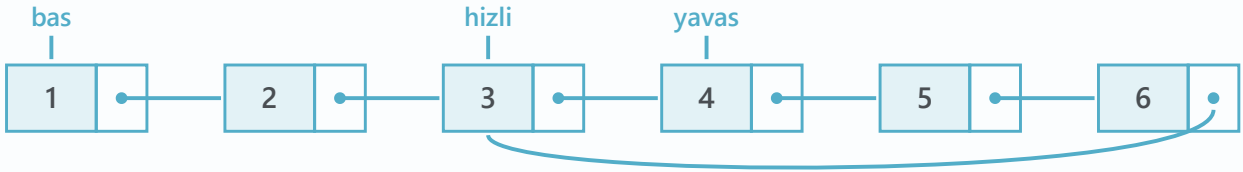
Adım 13 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
▶          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(4)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



4. tur: hizli (3) ve hizli.sonraki (4) null değil.

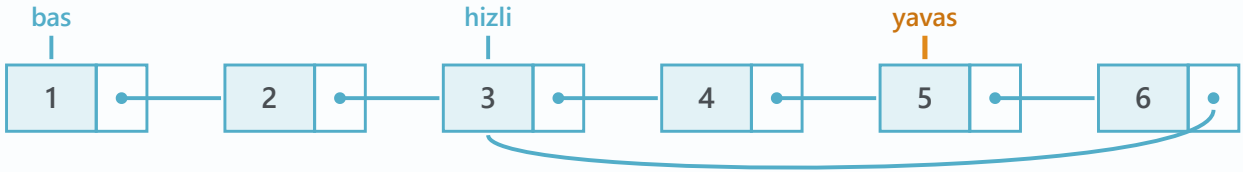
Adım 14 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
▶      while (hizli != null &&
▶             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(4)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



`yavas` 1 adım → düğüm(5).

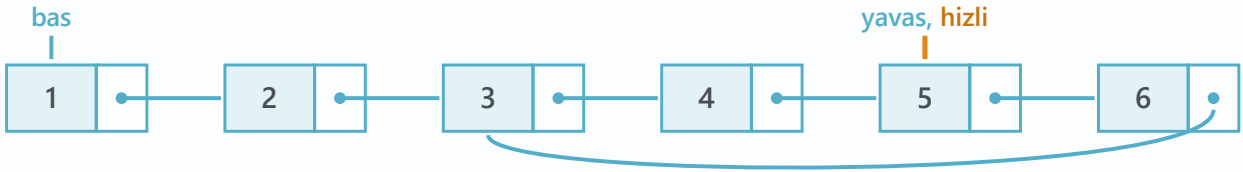
Adım 15 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
▶          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(5)**
- `hizli` = **düğüm(3)**

Kod İzleme: `donguVarMi` (Floyd)



`hizli` 2 adım → düğüm(5).

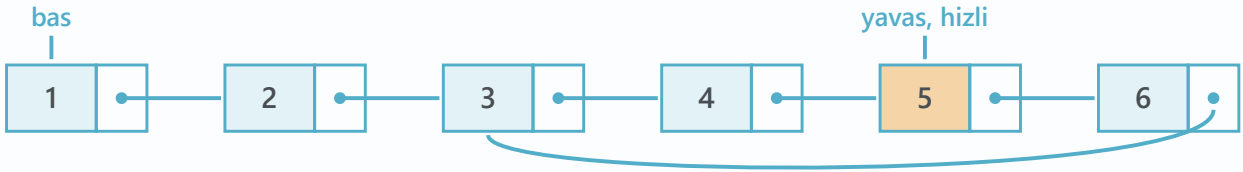
Adım 16 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
▶          hizli = hizli.sonraki.sonraki;
7          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(5)**
- `hizli` = **düğüm(5)**

Kod İzleme: `donguVarMi` (Floyd)



yavas == hizli (düğüm 5) → **return true**. Döngüde hızlı, yavaşa her tur 1 düğüm yaklaşır; O(n) zaman, O(1) bellek.

Adım 17 / 17

```
1  boolean donguVarMi(Dugum bas) {
2      Dugum yavas = bas, hizli = bas;
3      while (hizli != null &&
4             hizli.sonraki != null) {
5          yavas = yavas.sonraki;
6          hizli = hizli.sonraki.sonraki;
▶          if (yavas == hizli) return true;
8      }
9      return false;
10 }
```

DEĞİŞKENLER

- `bas` = **düğüm(1)**
- `yavas` = **düğüm(5)**
- `hizli` = **düğüm(5)**

Dairesel Listenin Sonuna Ekleme — O(1)



`sonaEkle(1)` : yeni düğüm oluşturulur.

Adım 1 / 17

```
Dugum gecici = new Dugum(deger);
if (son == null) {
    son = gecici;
    son.sonraki = son;
} else {
    gecici.sonraki = son.sonraki;
    son.sonraki = gecici;
    son = gecici;
}
boyut++;
```

DEĞİŞKENLER

`son = null` `boyut = 0` `deger = 1` `gecici = düğüm(1)`

Dairesel Listenin Sonuna Ekleme — O(1)



`son == null` → liste boş.

Adım 2 / 17

```
1  Dugum gecici = new Dugum(deger);
▶  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = null` `boyut = 0` `deger = 1` `gecici = düğüm(1)`

Dairesel Listenin Sonuna Ekleme — O(1)



`son = gecici` .

Adım 3 / 17

```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
▶      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(1)` `boyut = 0` `deger = 1` `gecici = düğüm(1)`

Dairesel Listenin Sonuna Ekleme — O(1)



`son.sonraki = son` : tek düğüm kendini gösterir; halka hemen kurulur.

Adım 4 / 17

```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
3      son = gecici;
▶      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(1)` `boyut = 0` `deger = 1` `gecici = düğüm(1)`

Dairesel Listenin Sonuna Ekleme — O(1)



`boyut++` .

Adım 5 / 17

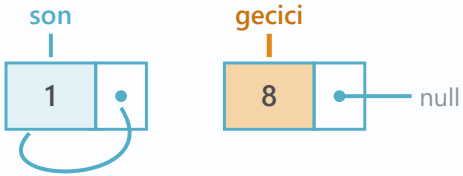
```
1 Dugum gecici = new Dugum(deger);
2 if (son == null) {
3     son = gecici;
4     son.sonraki = son;
5 } else {
6     gecici.sonraki = son.sonraki;
7     son.sonraki = gecici;
8     son = gecici;
9 }
▶ boyut++;
```

DEĞİŞKENLER

`son = düğüm(1)` `boyut = 1`

`deger = 1` `gecici = düğüm(1)`

Dairesel Listenin Sonuna Ekleme — O(1)



`sonaEkle(8)` : yeni düğüm oluşturulur.

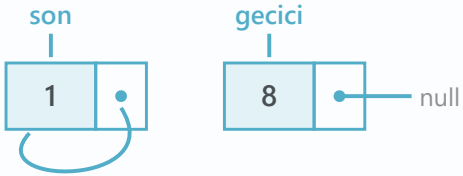
Adım 6 / 17

```
Dugum gecici = new Dugum(deger);
if (son == null) {
    son = gecici;
    son.sonraki = son;
} else {
    gecici.sonraki = son.sonraki;
    son.sonraki = gecici;
    son = gecici;
}
boyut++;
```

DEĞİŞKENLER

`son = düğüm(1)` `boyut = 1` `deger = 8` `gecici = düğüm(8)`

Dairesel Listenin Sonuna Ekleme — O(1)



Liste boş değil → `else`.

Adım 7 / 17

```
1  Dugum gecici = new Dugum(deger);
▶  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(1)` `boyut = 1`

`deger = 8` `gecici = düğüm(8)`

Dairesel Listenin Sonuna Ekleme — O(1)



`gecici.sonraki = son.sonraki` : yeni düğüm başı (1) gösterir.

Adım 8 / 17

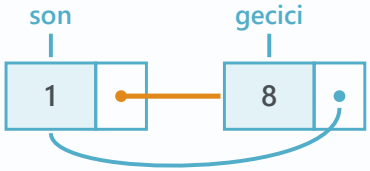
```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
▶      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(1)` `boyut = 1`

`deger = 8` `gecici = düğüm(8)`

Dairesel Listenin Sonuna Ekleme — O(1)



`son.sonraki = gecici` : eski son (1) yeni düğümü gösterir.

Adım 9 / 17

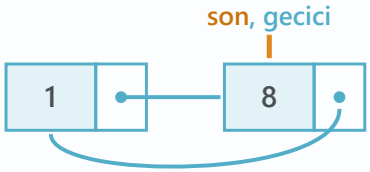
```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
▶      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(1)` `boyut = 1`

`deger = 8` `gecici = düğüm(8)`

Dairesel Listenin Sonuna Ekleme — O(1)



`son = gecici` : son artık 8.

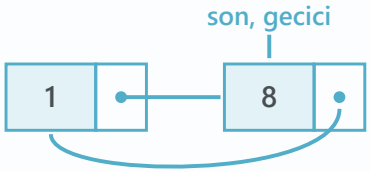
Adım 10 / 17

```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
▶      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(8)` `boyut = 1` `deger = 8` `gecici = düğüm(8)`

Dairesel Listenin Sonuna Ekleme — O(1)



`boyut++` .

Adım 11 / 17

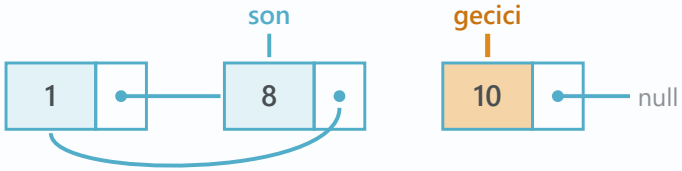
```
1 Dugum gecici = new Dugum(deger);
2 if (son == null) {
3     son = gecici;
4     son.sonraki = son;
5 } else {
6     gecici.sonraki = son.sonraki;
7     son.sonraki = gecici;
8     son = gecici;
9 }
▶ boyut++;
```

DEĞİŞKENLER

`son = düğüm(8)` `boyut = 2`

`deger = 8` `gecici = düğüm(8)`

Dairesel Listenin Sonuna Ekleme — O(1)



`sonaEkle(10)` : yeni düğüm oluşturulur.

Adım 12 / 17

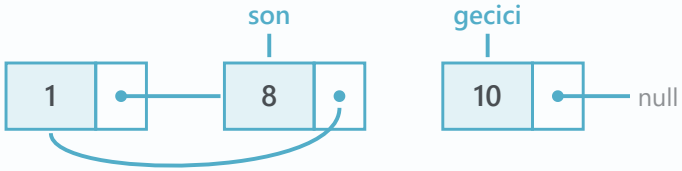
```
▶ Dugum gecici = new Dugum(deger);
2 if (son == null) {
3     son = gecici;
4     son.sonraki = son;
5 } else {
6     gecici.sonraki = son.sonraki;
7     son.sonraki = gecici;
8     son = gecici;
9 }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(8)` `boyut = 2`

`deger = 10` `gecici = düğüm(10)`

Dairesel Listenin Sonuna Ekleme — O(1)



Liste boş değil → `else`.

Adım 13 / 17

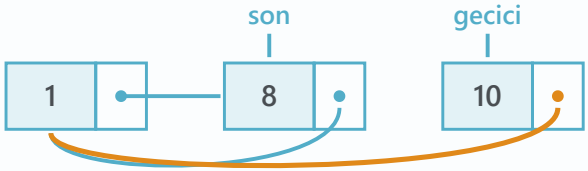
```
1  Dugum gecici = new Dugum(deger);
▶  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(8)` `boyut = 2`

`deger = 10` `gecici = düğüm(10)`

Dairesel Listenin Sonuna Ekleme — O(1)



`gecici.sonraki = son.sonraki` : yeni düğüm başı (1) gösterir.

Adım 14 / 17

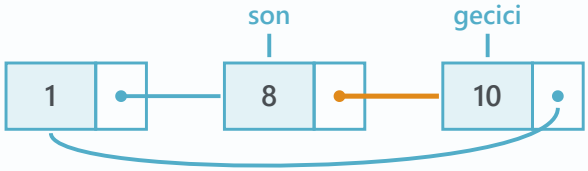
```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
▶      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(8)` `boyut = 2`

`deger = 10` `gecici = düğüm(10)`

Dairesel Listenin Sonuna Ekleme — O(1)



`son.sonraki = gecici` : eski son (8) yeni düğümü gösterir.

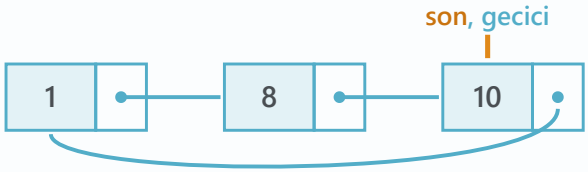
Adım 15 / 17

```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
▶      son.sonraki = gecici;
8      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(8)` `boyut = 2` `deger = 10` `gecici = düğüm(10)`

Dairesel Listenin Sonuna Ekleme — O(1)



`son = gecici` : son artık 10.

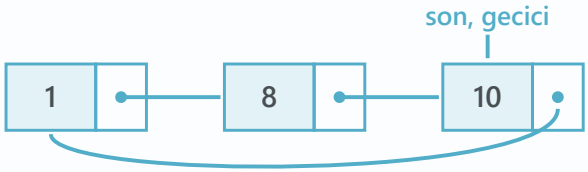
Adım 16 / 17

```
1  Dugum gecici = new Dugum(deger);
2  if (son == null) {
3      son = gecici;
4      son.sonraki = son;
5  } else {
6      gecici.sonraki = son.sonraki;
7      son.sonraki = gecici;
▶      son = gecici;
9  }
10 boyut++;
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 2` `deger = 10` `gecici = düğüm(10)`

Dairesel Listenin Sonuna Ekleme — O(1)



`boyut++`. Sonuç: 1 → 8 → 10 → (1). Gezinme yok: **O(1)**.

Adım 17 / 17

```
1 Dugum gecici = new Dugum(deger);
2 if (son == null) {
3     son = gecici;
4     son.sonraki = son;
5 } else {
6     gecici.sonraki = son.sonraki;
7     son.sonraki = gecici;
8     son = gecici;
9 }
▶ boyut++;
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3`

`deger = 10` `gecici = düğüm(10)`

Dairesel Listede Başa Ekleme ve Baştan Silme

O(1)

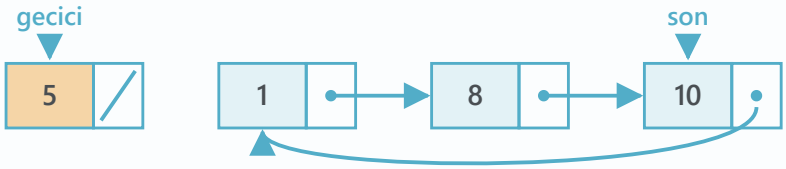
```
void basaEkle(int deger) {
    Dugum gecici = new Dugum(deger);
    if (son == null) {
        son = gecici;
        son.sonraki = son;
    } else {
        gecici.sonraki = son.sonraki;
        son.sonraki = gecici;
    }   // son değişmez
    boyut++;
}
```

O(1)

```
void bastanSil() {
    if (son == null) return;
    Dugum bas = son.sonraki;
    if (bas == son)   // tek düğüm
        son = null;
    else
        son.sonraki = bas.sonraki;
    boyut--;
}
```

Başa ekleme, sona eklemenin `son = gecici` satırı çıkarılmış hâlidir: yeni düğüm halkaya aynı yere girer, yalnızca kimin "son" sayıldığı değişir.

Kod İzleme: Dairesel `basaEkle(5)`



`basaEkle(5)` : yeni düğüm. Halka: 1 → 8 → 10 → (1), son = 10.

Adım 1 / 5

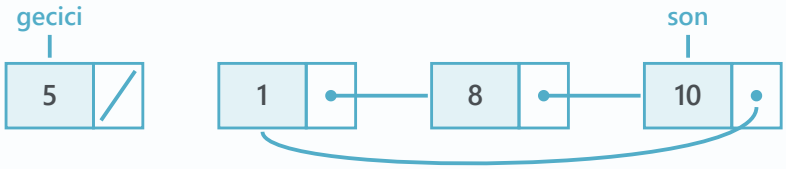
```
1  void basaEkle(int deger) {
▶      Dugum gecici = new Dugum(deger);
3      if (son == null) {
4          son = gecici;
5          son.sonraki = son;
6      } else {
7          gecici.sonraki = son.sonraki;
8          son.sonraki = gecici;
9      }   // son değişmez
10     boyut++;
11 }
```

DEĞİŞKENLER

`son` = **düğüm(10)** `boyut` = **3**

`deger` = **5** `gecici` = **düğüm(5)**

Kod İzleme: Dairesel `basaEkle(5)`



Liste boş değil → `else`.

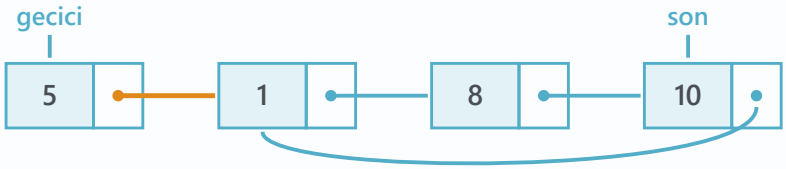
Adım 2 / 5

```
1  void basaEkle(int deger) {
2      Dugum gecici = new Dugum(deger);
▶      if (son == null) {
4          son = gecici;
5          son.sonraki = son;
6      } else {
7          gecici.sonraki = son.sonraki;
8          son.sonraki = gecici;
9      }   // son değişmez
10     boyut++;
11 }
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3` `deger = 5` `gecici = düğüm(5)`

Kod İzleme: Dairesel `basaEkle(5)`



`gecici.sonraki = son.sonraki` : 5, eski başı (1) gösterir.

Adım 3 / 5

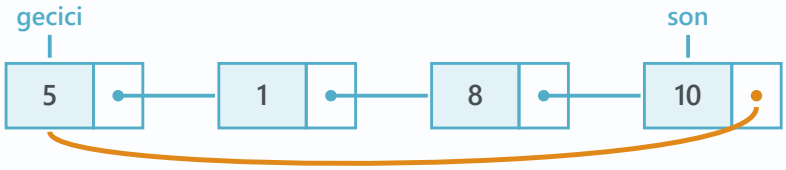
```
1  void basaEkle(int deger) {
2      Dugum gecici = new Dugum(deger);
3      if (son == null) {
4          son = gecici;
5          son.sonraki = son;
6      } else {
▶          gecici.sonraki = son.sonraki;
8          son.sonraki = gecici;
9      }   // son değişmez
10     boyut++;
11 }
```

DEĞİŞKENLER

`son` = **düğüm(10)** `boyut` = **3**

`deger` = **5** `gecici` = **düğüm(5)**

Kod İzleme: Dairesel `basaEkle(5)`



`son.sonraki = gecici` : 10 artık 5'i gösterir → 5 yeni baş. `son` değişmez.

Adım 4 / 5

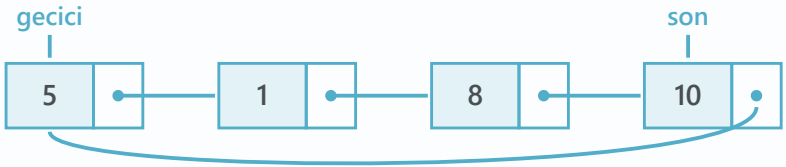
```
1  void basaEkle(int deger) {
2      Dugum gecici = new Dugum(deger);
3      if (son == null) {
4          son = gecici;
5          son.sonraki = son;
6      } else {
7          gecici.sonraki = son.sonraki;
▶          son.sonraki = gecici;
9      }   // son değişmez
10     boyut++;
11 }
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3`

`deger = 5` `gecici = düğüm(5)`

Kod İzleme: Dairesel `basaEkle(5)`



`boyut++`. Halka: 5 → 1 → 8 → 10 → (5). **O(1)**; sona eklemeden tek farkı `son = gecici` olmaması.

Adım 5 / 5

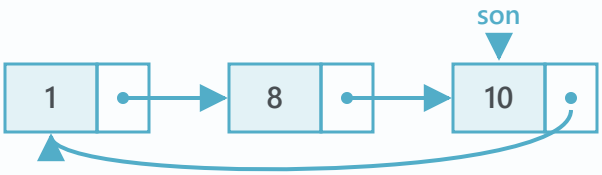
```
1  void basaEkle(int deger) {
2      Dugum gecici = new Dugum(deger);
3      if (son == null) {
4          son = gecici;
5          son.sonraki = son;
6      } else {
7          gecici.sonraki = son.sonraki;
8          son.sonraki = gecici;
9      }   // son değişmez
▶      boyut++;
11 }
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 4`

`deger = 5` `gecici = düğüm(5)`

Kod İzleme: Dairesel `bastanSil()`



Halka: 1 → 8 → 10 → (1), son = 10. Boş değil.

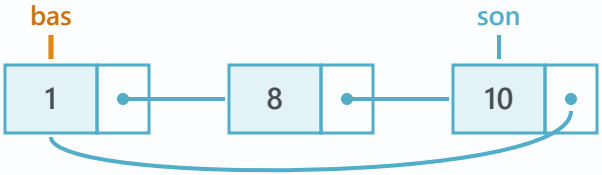
Adım 1 / 5

```
1 void bastanSil() {
▶       if (son == null) return;
3       Dugum bas = son.sonraki;
4       if (bas == son)   // tek düğüm
5           son = null;
6       else
7           son.sonraki = bas.sonraki;
8       boyut--;
9   }
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3`

Kod İzleme: Dairesel `bastanSil()`



`bas = son.sonraki` : silinecek baş düğüm (1).

Adım 2 / 5

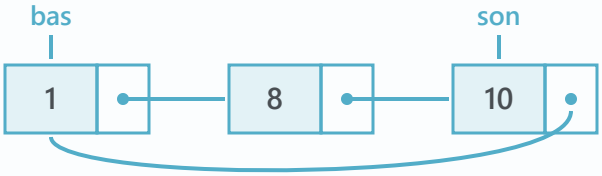
```
1 void bastanSil() {
2     if (son == null) return;
▶     Dugum bas = son.sonraki;
4     if (bas == son)   // tek düğüm
5         son = null;
6     else
7         son.sonraki = bas.sonraki;
8     boyut--;
9 }
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3`

`bas = düğüm(1)`

Kod İzleme: Dairesel `bastanSil()`



`bas != son` → birden çok düğüm.

Adım 3 / 5

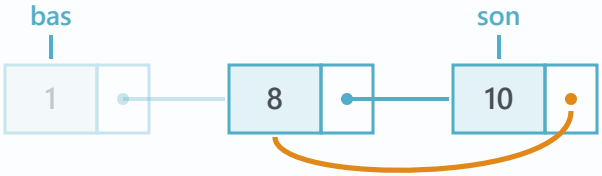
```
1 void bastanSil() {
2     if (son == null) return;
3     Dugum bas = son.sonraki;
▶     if (bas == son)   // tek düğüm
5         son = null;
6     else
7         son.sonraki = bas.sonraki;
8     boyut--;
9 }
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3`

`bas = düğüm(1)`

Kod İzleme: Dairesel `bastanSil()`



`son.sonraki = bas.sonraki` : 10 artık 8'i gösterir; 1 halkadan çıktı.

Adım 4 / 5

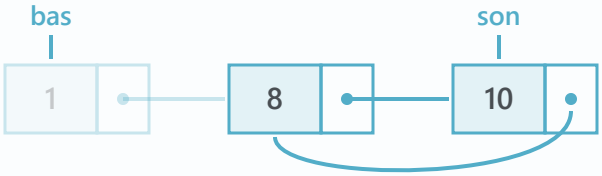
```
1 void bastanSil() {
2     if (son == null) return;
3     Dugum bas = son.sonraki;
4     if (bas == son)   // tek düğüm
5         son = null;
6     else
▶         son.sonraki = bas.sonraki;
8     boyut--;
9 }
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3`

`bas = düğüm(1)`

Kod İzleme: Dairesel `bastanSil()`



`boyut--`. Halka: 8 → 10 → (8). **O(1)**.

Adım 5 / 5

```
1 void bastanSil() {
2     if (son == null) return;
3     Dugum bas = son.sonraki;
4     if (bas == son)   // tek düğüm
5         son = null;
6     else
7         son.sonraki = bas.sonraki;
▶     boyut--;
9 }
```

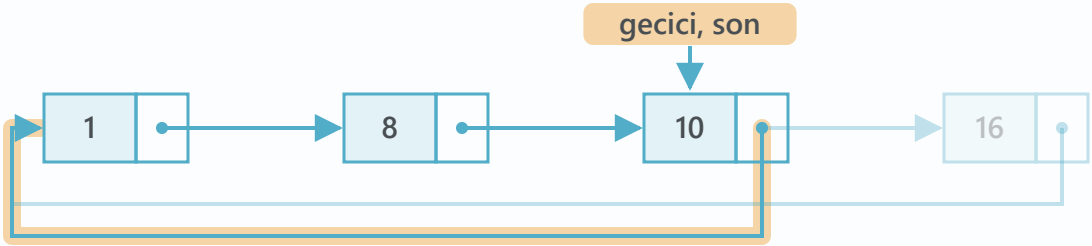
DEĞİŞKENLER

`son = düğüm(10)` `boyut = 2`

`bas = düğüm(1)`

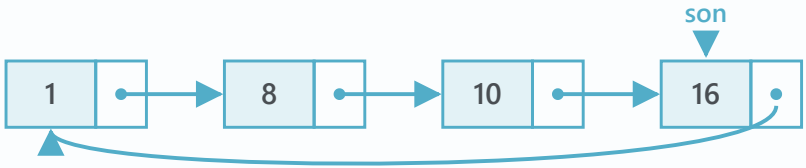
Dairesel Listede Sondan Silme — O(n)

```
void sondanSil() {
    if (son == null) return;
    if (son.sonraki == son) son = null;    // tek düğüm
    else {
        Dugum gecici = son.sonraki;
        while (gecici.sonraki != son)      // sondan bir önceki
            gecici = gecici.sonraki;
        gecici.sonraki = son.sonraki;      // başa bağla
        son = gecici;
    }
    boyut--;
}
```



Sondan bir önceki düğümü bulmak için halkada gezinmek gerekir: O(n). Çift yönlü dairesel listede `son.onceki` ile O(1) olur.

Kod İzleme: Dairesel `sondanSil()`



`sondanSil()` : halka 1 → 8 → 10 → 16 → (1), son = 16. Boş değil.

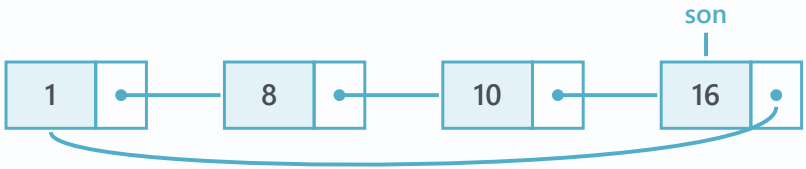
Adım 1 / 11

```
▶ if (son == null) return;
2 if (son.sonraki == son) son = null;   // tek düğüm
3 else {
4     Dugum gecici = son.sonraki;
5     while (gecici.sonraki != son)  // sondan bir önceki
6         gecici = gecici.sonraki;
7     gecici.sonraki = son.sonraki;  // başa bağla
8     son = gecici;
9 }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4`

Kod İzleme: Dairesel `sondanSil()`



`son.sonraki != son` → tek düğüm değil.

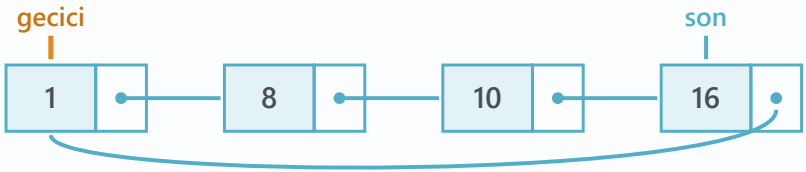
Adım 2 / 11

```
1  if (son == null) return;
▶  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
5      while (gecici.sonraki != son)  // sondan bir önceki
6          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4`

Kod İzleme: Dairesel `sondanSil()`



`gecici = son.sonraki` : baştan (1) başlanır.

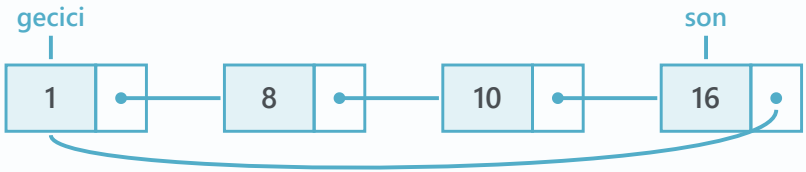
Adım 3 / 11

```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
▶      Dugum gecici = son.sonraki;
5      while (gecici.sonraki != son)  // sondan bir önceki
6          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4` `gecici = düğüm(1)`

Kod İzleme: Dairesel `sondanSil()`



düğüm(1).sonraki = 8 ≠ son → ilerle.

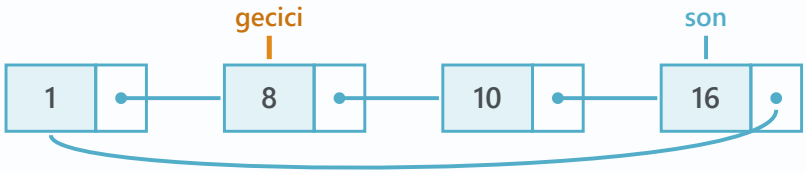
Adım 4 / 11

```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
▶      while (gecici.sonraki != son)  // sondan bir önceki
6          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4` `gecici = düğüm(1)`

Kod İzleme: Dairesel `sondanSil()`



`gecici = gecici.sonraki` → düğüm(8).

Adım 5 / 11

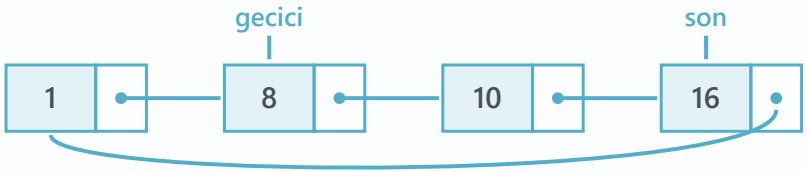
```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
5      while (gecici.sonraki != son)  // sondan bir önceki
▶          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4`

`gecici = düğüm(8)`

Kod İzleme: Dairesel `sondanSil()`



düğüm(8).sonraki = 10 ≠ son → ilerle.

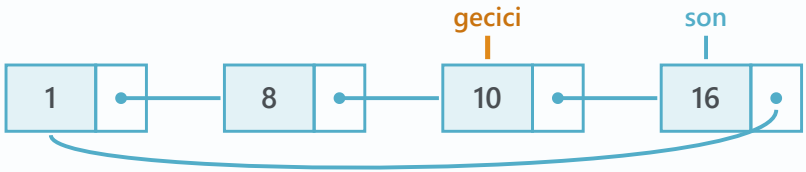
Adım 6 / 11

```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
▶      while (gecici.sonraki != son)  // sondan bir önceki
6          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4` `gecici = düğüm(8)`

Kod İzleme: Dairesel `sondanSil()`



`gecici = gecici.sonraki` → düğüm(10).

Adım 7 / 11

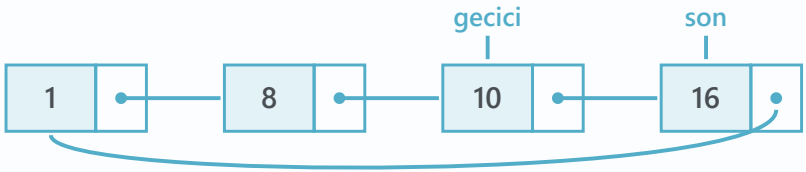
```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
5      while (gecici.sonraki != son)  // sondan bir önceki
▶          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4`

`gecici = düğüm(10)`

Kod İzleme: Dairesel `sondanSil()`



düğüm(10).sonraki == son → gecici sondan bir önceki düğümde.

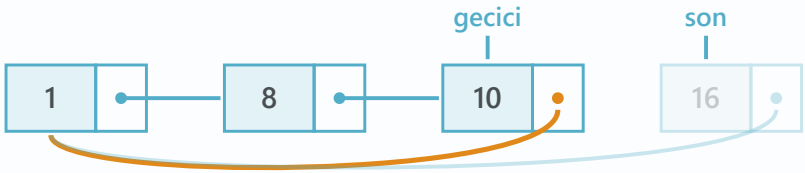
Adım 8 / 11

```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
▶      while (gecici.sonraki != son)  // sondan bir önceki
6          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4` `gecici = düğüm(10)`

Kod İzleme: Dairesel `sondanSil()`



`gecici.sonraki = son.sonraki` : 10 başı (1) gösterir; 16 halkadan çıktı.

Adım 9 / 11

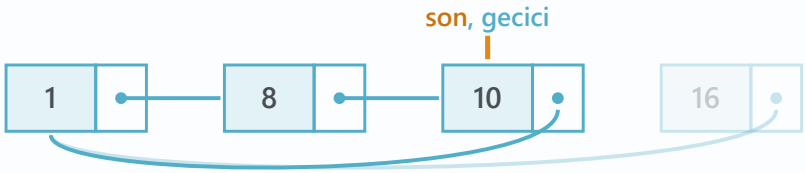
```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
5      while (gecici.sonraki != son)  // sondan bir önceki
6          gecici = gecici.sonraki;
▶      gecici.sonraki = son.sonraki;  // başa bağla
8      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(16)` `boyut = 4`

`gecici = düğüm(10)`

Kod İzleme: Dairesel `sondanSil()`



`son = gecici` : son artık 10.

Adım 10 / 11

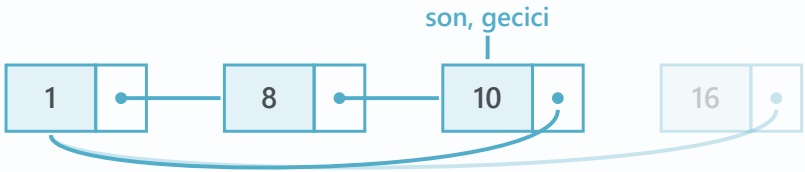
```
1  if (son == null) return;
2  if (son.sonraki == son) son = null;   // tek düğüm
3  else {
4      Dugum gecici = son.sonraki;
5      while (gecici.sonraki != son)  // sondan bir önceki
6          gecici = gecici.sonraki;
7      gecici.sonraki = son.sonraki;  // başa bağla
▶      son = gecici;
9  }
10 boyut--;
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 4`

`gecici = düğüm(10)`

Kod İzleme: Dairesel `sondanSil()`



`boyut--`. Halka: 1 → 8 → 10 → (1). Öncekini bulmak için gezinildi: **O(n)**.

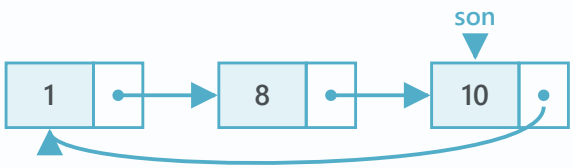
Adım 11 / 11

```
1 if (son == null) return;
2 if (son.sonraki == son) son = null;   // tek düğüm
3 else {
4     Dugum gecici = son.sonraki;
5     while (gecici.sonraki != son)  // sondan bir önceki
6         gecici = gecici.sonraki;
7     gecici.sonraki = son.sonraki;  // başa bağla
8     son = gecici;
9 }
▶ boyut--;
```

DEĞİŞKENLER

`son = düğüm(10)` `boyut = 3` `gecici = düğüm(10)`

Kod İzleme: `dondur()` ile Round-Robin



Halka 1 → 8 → 10 → (1), `son` = 10, baş = `son.sonraki` = 1. Üç iş sırayla işlemciyi kullanacak.

Adım 1 / 9

```
1 void dondur() {                // Goodrich: rotate()
2     if (son != null)
3         son = son.sonraki;     // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
▶ for (int tur = 1; tur <= 4; tur++) {
7     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

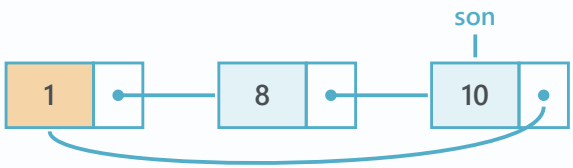
`son = düğüm(10)`

ÇAĞRI YIĞINI

`main()`

ÇIKTI

Kod İzleme: `dondur()` ile Round-Robin



Tur 1: baştaki iş **1** hizmet alır (yazdırılır).

Adım 2 / 9

```
1 void dondur() {              // Goodrich: rotate()
2     if (son != null)
3         son = son.sonraki;   // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
▶     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(10)` `tur = 1`

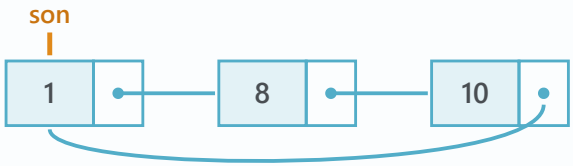
ÇAĞRI YIĞINI

`main()`

ÇIKTI

```
1
```

Kod İzleme: `dondur()` ile Round-Robin



`dondur()` : `son = son.sonraki` → son = 1. Hiçbir bağ değişmedi; yalnızca bir işaretçi ilerledi: **O(1)**. Yeni baş: 8.

Adım 3 / 9

```
1 void dondur() {              // Goodrich: rotate()
▶     if (son != null)
▶         son = son.sonraki;   // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
7     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(1)` `tur = 1`

ÇAĞRI YIĞINI

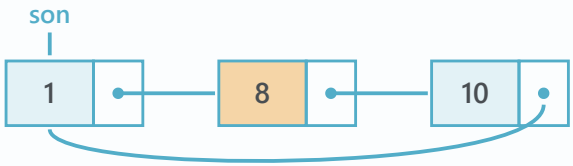
`dondur()`

`main()`

ÇIKTI

```
1
```

Kod İzleme: `dondur()` ile Round-Robin



Tur 2: baştaki iş **8** hizmet alır (yazdırılır).

Adım 4 / 9

```
1 void dondur() {                // Goodrich: rotate()
2     if (son != null)
3         son = son.sonraki;     // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
▶     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(1)` `tur = 2`

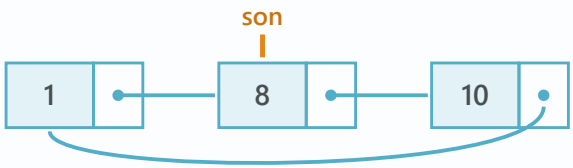
ÇAĞRI YIĞINI

`main()`

ÇIKTI

```
1 8
```

Kod İzleme: `dondur()` ile Round-Robin



`dondur()` : `son = son.sonraki` → son = 8. Hiçbir bağ değişmedi; yalnızca bir işaretçi ilerledi: **O(1)**. Yeni baş: 10.

Adım 5 / 9

```
1 void dondur() {              // Goodrich: rotate()
▶     if (son != null)
▶         son = son.sonraki;   // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
7     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(8)` `tur = 2`

ÇAĞRI YIĞINI

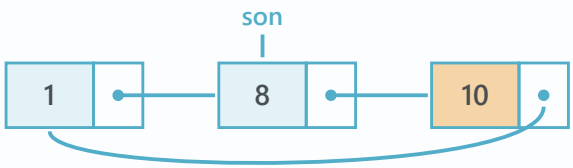
`dondur()`

`main()`

ÇIKTI

```
1 8
```

Kod İzleme: `dondur()` ile Round-Robin



Tur 3: baştaki iş **10** hizmet alır (yazdırılır).

Adım 6 / 9

```
1 void dondur() {              // Goodrich: rotate()
2     if (son != null)
3         son = son.sonraki;   // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
▶     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(8)` `tur = 3`

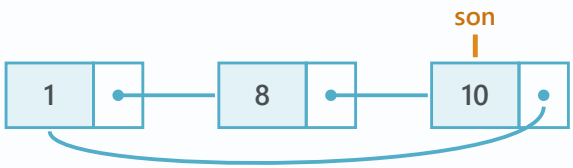
ÇAĞRI YIĞINI

`main()`

ÇIKTI

```
1 8 10
```

Kod İzleme: `dondur()` ile Round-Robin



`dondur()` : `son = son.sonraki` → son = 10. Hiçbir bağ değişmedi; yalnızca bir işaretçi ilerledi: **O(1)**. Yeni baş: 1.

Adım 7 / 9

```
1 void dondur() {              // Goodrich: rotate()
▶     if (son != null)
▶         son = son.sonraki;   // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
7     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(10)` `tur = 3`

ÇAĞRI YIĞINI

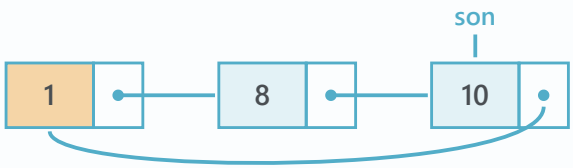
`dondur()`

`main()`

ÇIKTI

```
1 8 10
```

Kod İzleme: `dondur()` ile Round-Robin



Tur 4: baştaki iş **1** hizmet alır (yazdırılır).

Adım 8 / 9

```
1 void dondur() {                // Goodrich: rotate()
2     if (son != null)
3         son = son.sonraki;     // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
▶     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(10)` `tur = 4`

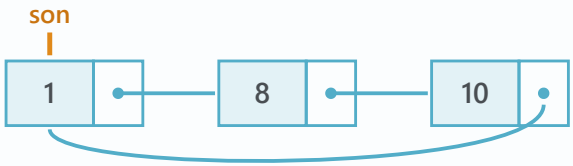
ÇAĞRI YIĞINI

`main()`

ÇIKTI

```
1 8 10 1
```

Kod İzleme: `dondur()` ile Round-Robin



`dondur()` : `son = son.sonraki` → son = 1. Hiçbir bağ değişmedi; yalnızca bir işaretçi ilerledi: **O(1)**. Çıktı **1 8 10 1**: işler adil biçimde dönüşümlü sıra alır.

Adım 9 / 9

```
1 void dondur() {              // Goodrich: rotate()
▶     if (son != null)
▶         son = son.sonraki;   // eski baş sona geçer
4 }
5 // round-robin: her tur baştakine hizmet et
6 for (int tur = 1; tur <= 4; tur++) {
7     System.out.print(son.sonraki.veri + " ");
8     liste.dondur();
9 }
```

DEĞİŞKENLER

`son = düğüm(1)` `tur = 4`

ÇAĞRI YIĞINI

`dondur()`

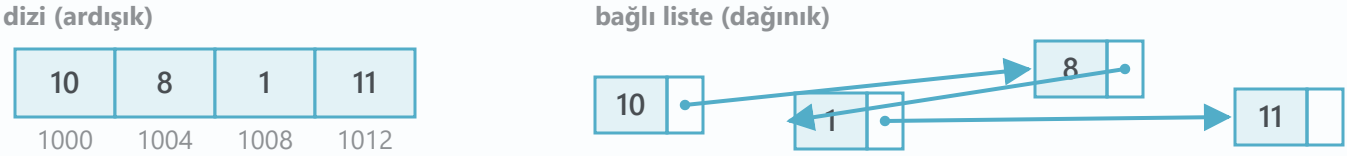
`main()`

ÇIKTI

```
1 8 10 1
```

ÖZET

Dizi mi, Bağlı Liste mi?



Ölçüt	Dizi	Bağlı liste
i. öğeye erişim	O(1): adres hesaplanır	O(n): baştan gezinme
Başa ekleme / silme	O(n): kaydırma	O(1)
Bilinen konumdan sonra ekleme	O(n)	O(1): iki bağ
Boyut	sabit; büyütmek O(n) kopya	dinamik, her ekleme `new`
Ek bellek / önbellek	boş kapasite; önbellek dostu	düğüm başına referans; dağınık erişim

Sık rastgele erişim ve sabit boyut → dizi. Uçlarda sık ekleme/silme, öngörülemeyen boyut → bağlı liste (yığın, kuyruk).

ÖZET

Bağlı Listelerde Sık Yapılan Hatalar

Boş liste kontrolü yok

`bas.veri`, `bas` null iken → `NullPointerException`.

```
if (bas == null) return;
```

Bağları yanlış sırada kurmak

Önce `bas` değişirse eski liste kaybolur.

```
yeni.sonraki = bas; bas = yeni;
```

Gezinirken `bas`'ı ilerletmek

Yazdırma sonrası liste erişilemez olur.

```
Dugum gecici = bas;
```

`son`'u unutmak

Tek düğüm silinince `son` eski düğümü göstermeye devam eder.

```
if (bas == null) son = null;
```

Çift yönlüde tek yön

`onceki` güncellenmezse geri gezinme bozulur.

```
sonraki.onceki = yeni;
```

Dairesel listede `!= null`

Halkada null yoktur → sonsuz döngü.

```
while (g != son) …
```

ÖZET

Bağlı Liste Türlerinde İşlem Maliyetleri

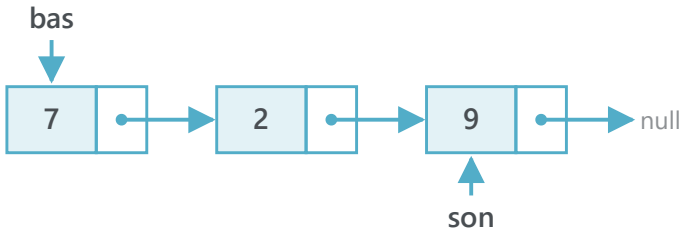
İşlem	Tek yönlü (yalnız bas)	Tek yönlü (bas + son)	Çift yönlü (bas + son)	Dairesel (yalnız son)
Başa ekleme	O(1)	O(1)	O(1)	O(1)
Sona ekleme	O(n)	O(1)	O(1)	O(1)
Baştan silme	O(1)	O(1)	O(1)	O(1)
Sondan silme	O(n)	O(n)	O(1)	O(n)
Arama / i. öğe	O(n)	O(n)	O(n)	O(n)
Bilinen düğümü silme	O(n)*	O(n)*	O(1)	O(n)*

* Önceki düğümü bulmak için gezinme gerekir. Bilinen bir düğümden *sonra* ekleme her türde O(1)'dir. Dizide ise i. öğeye erişim O(1), başa ekleme O(n).

ÖZET

Deneyin: Tek Yönlü Liste

basaEkle | sonaEkle | bastanSil | sondanSil | sıfırla | ☑ son işaretçisi



Butonlarla listeyi değiştirin; maliyetler durum satırında.

Daha fazlası için simülatör: Bağlı Liste Simülatörü (ekleme, silme, arama adım adım). Yığın ve kuyruk simülatörleri de bağlı listeyle uygulanabilir: Yığın · Kuyruk

ÖZET

Örnek Problemler

1. Tek yönlü listede sona eklemeyi O(1) yapmak için ne gerekir? Hangi işlem yine O(n) kalır?
2. `basaEkle`'de iki satırın sırası değişirse (`bas = yeniDugum;` önce) ne olur?
3. Bir listenin ortasını, uzunluğunu hesaplamadan tek geçişte nasıl bulursunuz?
4. Tek yönlü listede bir döngü (son düğümün önceki bir düğüme bağlanması) olup olmadığını nasıl anlarsınız?
5. Çift yönlü listede `son` düğümü silmenin maliyeti nedir? Tek yönlüde neden farklıdır?
6. Dairesel listede neden `bas` yerine `son` tutulur?

▶ **Cevapları göster**

ÖZET

Alıştırmalar

1. 3 → 5 → 7 listesinde `bas.sonraki = bas.sonraki.sonraki;` sonrası liste nedir? 5'e ne olur?
2. `int gizem(Dugum d) { return d == null ? 0 : d.veri + gizem(d.sonraki); }` 2 → 4 → 6 için ne döndürür?
3. Tek yönlü listede sondan k. öğeyi, uzunluğu bilmeden tek geçişte nasıl bulursunuz?
4. Bekçili çift yönlü listede boş listeye ekleme kaç bağ ataması yapar? Neden `if` gerekmez?
5. Dairesel liste 1 → 8 → 10 (son = 10). İki kez `dondur()` sonrası baştaki öğe hangisidir?
6. `sirala()` 3 → 1 → 2 listesinde kaç karşılaştırma ve kaç takas yapar?

▶ **Cevaplar / ipuçları**

Kaynaklar

- Goodrich, Tamassia ve Goldwasser, *Data Structures and Algorithms in Java*, 6. baskı, 2014.
- Lafore, *Data Structures and Algorithms in Java*, 2. baskı, 2002.
- Sedgewick ve Wayne, *Algorithms*, 4. baskı, 2011.

Son

Bölüm 3: Bağlı Listeler

Sonraki sunum: Bölüm 4: Yığınlar · Çalışma soruları · Simülatörler: Linked List Simulator, Doubly Linked List Simulator, Circular Linked List Simulator · Terim sözlüğü